



Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

26.08.2026

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Seite 2

Technology
Arts Sciences
TH Köln

Lernraum III: Graphen

- Graphen (un-/gerichtet, un-/gewichtet)
- Traversierung von Graphen
 - Tiefensuche
 - Breitensuche
- Gerichtete Graphen
 - Topologische Sortierung
- Kürzeste Pfade
 - Ungewichteter Graph: Breitensuche
 - Gewichteter Graph: Algorithmus von Dijkstra
- Minimale Spannbäume

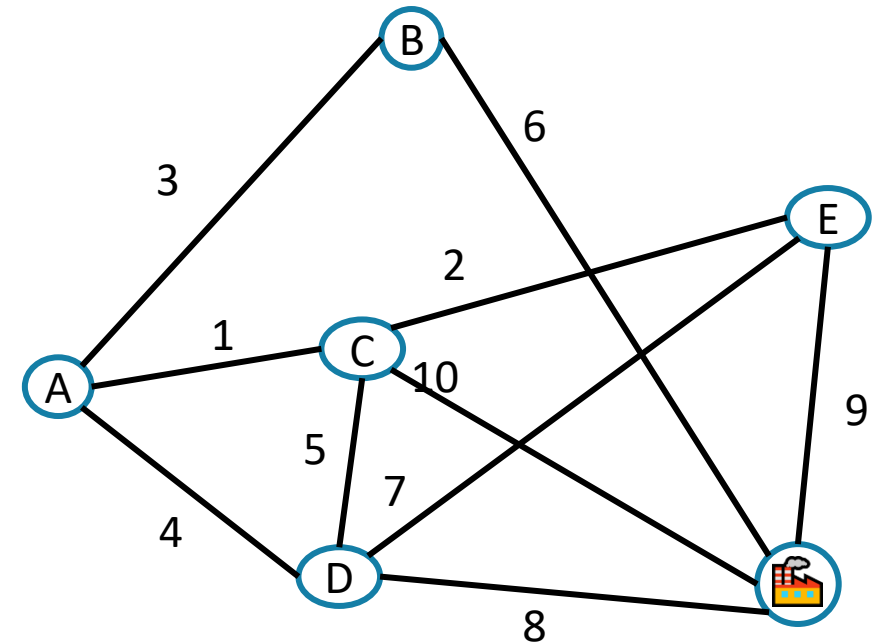
Übersicht für heute

- Minimale Spannbäume
 - Algorithmus von Prim
 - Algorithmus von Kruskal
- ADT Disjunkter Mengen
 - Menge
- Implementierung des ADT Disjunkter Mengen
 - Quick-Find
 - Wald disjunkter Mengen
 - Quick-Union, Weighted Quick-Union
 - Pfadverkürzung
- Laufzeit des Algorithmus von Kruskal mit optimaler Implementierung des ADT Disjunkter Mengen

Minimale Spannbäume

Es sind die 1920er Jahre. Ihre Freundin bei der Elektrizitätsgesellschaft muss entscheiden, wo Leitungen verlegt werden sollen, um all diese Städte mit dem Kraftwerk zu verbinden.

Sie weiß, wie viel es kosten würde, Stromleitungen zwischen zwei beliebigen Städten zu verlegen, und möchte die **günstigste Lösung** finden, um sicherzustellen, dass der Strom vom Kraftwerk in jede Stadt gelangt.



Minimale Spannbäume

Weitere Anwendungen:

- Wasserversorgungsnetze
- Telekommunikationsnetze
- Epidemiologie:
 - Rekonstruktion von Übertragungswegen von Krankheiten
- ...

(<https://pmc.ncbi.nlm.nih.gov/articles/PMC516344/>)



Minimale Spannbäume



„In seiner vegetativen Phase „schleimt“ der Schleimpilz *Physarum polycephalum* auf der Suche nach Nahrung durch die Welt. Bei seiner Erkundung verbindet er zuvor gefundene Nahrungsquellen mit röhrenförmigen Strukturen (tubular structures).“

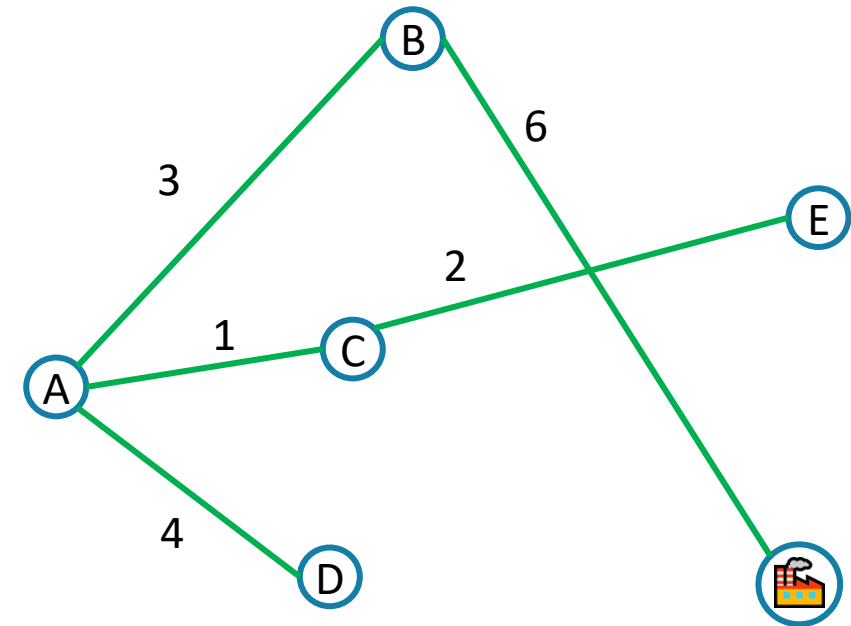
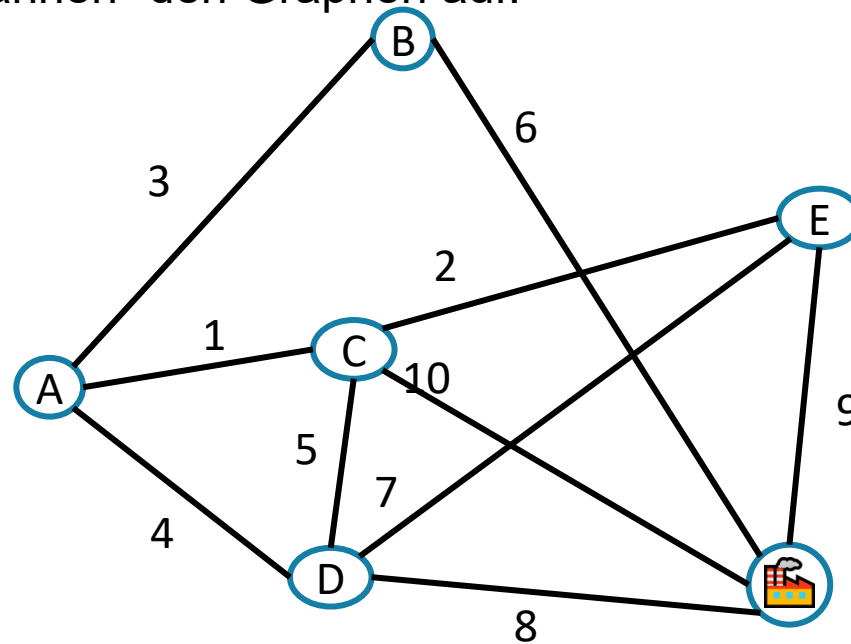
Quelle: s. unten

Minimaler-Spannbaum-Problem (Minimal Spanning Tree - MST)

Wir suchen eine Menge von Kanten, die:

- alle Knoten des Graphen miteinander verbindet.
- das geringste Gesamtgewicht hat, die diese Bedingung erfüllt.

Die Kanten „spannen“ den Graphen auf.



Minimaler-Spannbaum-Problem (Minimal Spanning Tree - MST)

Was suchen wir? Eine solche Menge von Kanten, dass:

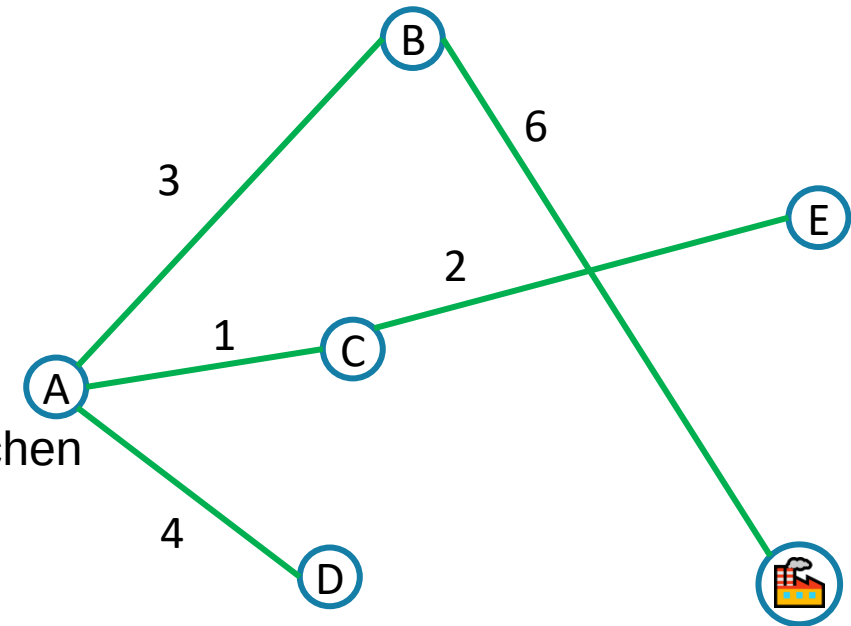
- alle Knoten des Graphen miteinander verbindet.
- das geringste Gesamtgewicht hat, die diese Bedingung erfüllt.

Die Kanten „spannen“ den Graphen auf.

Behauptung: Die von uns gewählte Menge von Kanten hat niemals einen Zyklus. Warum?

MST besteht aus genau den Kanten, die alle Knoten verbinden

- Wenn man 1 Kante wegnimmt, wird die Konnektivität unterbrochen
- Hinzufügen von 1 Kante ergibt einen Zyklus
- Enthält genau $|V| - 1$ Kanten
- MST ist ein Baum!



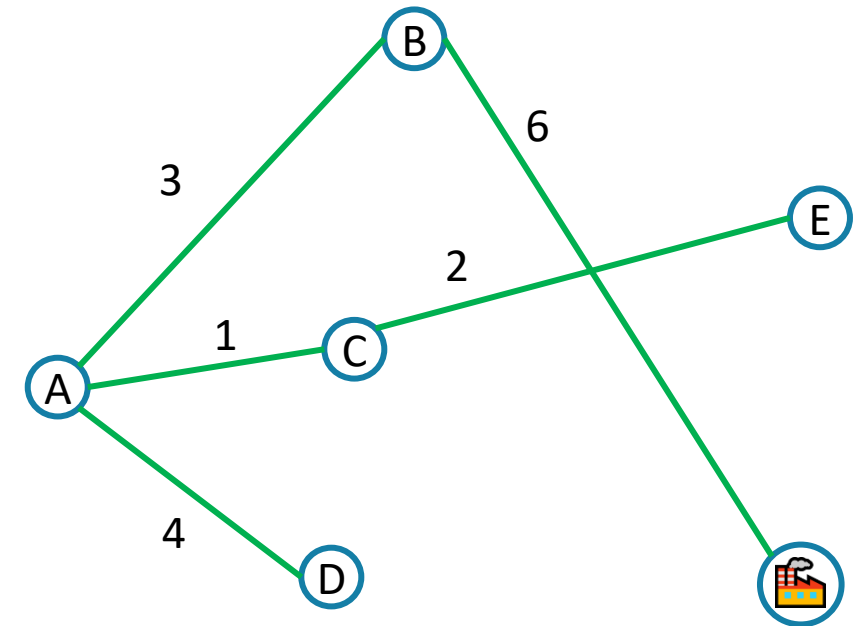
Minimaler-Spannbaum-Problem

Minimaler-Spannbaum-Problem

Gegeben: ein ungerichteter, gewichteter Graph G

Finde: Eine Menge von Kanten mit minimalem Gewicht, so dass man von jedem Knoten von G zu jedem anderen nur über diese Kanten gelangen kann.

Gibt es immer einen eindeutigen minimalen Spannbaum für einen gegebenen Graphen, ja oder nein?

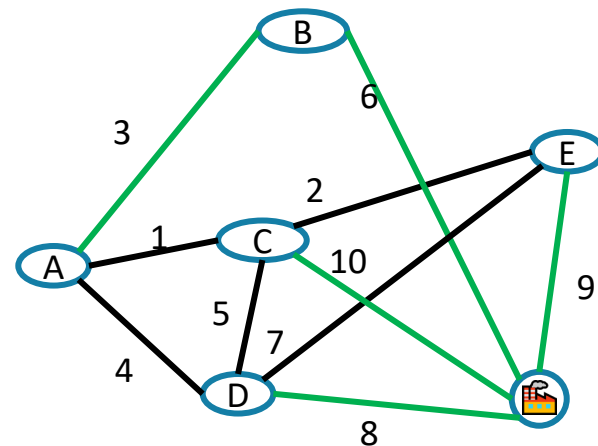


Kürzester Pfad vs Minimaler Spannbaum

Gewichteter kürzester Pfad

Gegeben: ein gewichteter Graph G und Knoten s und t

Finde: den kürzesten Pfad von Knoten s zu Knoten t .

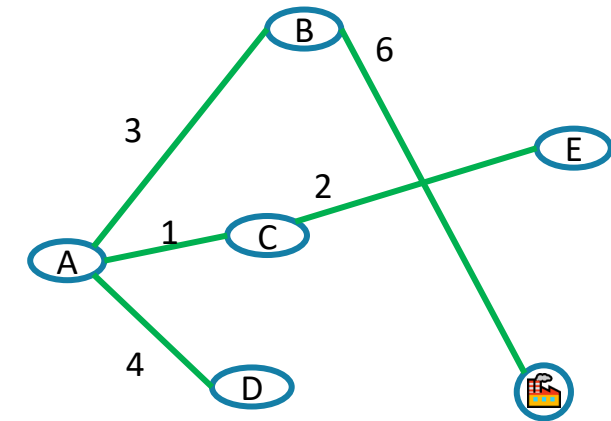


Baum kürzester
Pfade von Kraftwerk

Minimaler-Spannbaum-Problem

Gegeben: ein ungerichteter, gewichteter Graph G

Finde: Eine Menge von Kanten mit minimalem Gewicht, so dass man von jedem Knoten von G zu jedem anderen Knoten nur über diese Kanten gelangen kann.

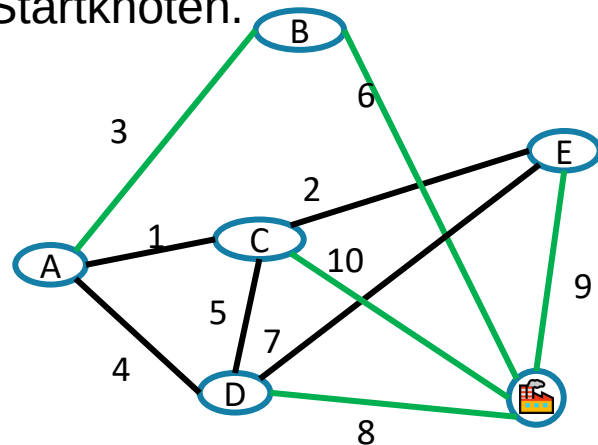


MST des Graphen

Kürzester Pfad vs Minimaler Spannbaum

Baum kürzester Pfade (SPT)

Spezifischer Startknoten (wenn Sie einen anderen Startknoten haben, ändert das den gesamten SPT, so dass es häufig mehrere SPTs für Graphen gibt)
Erfasst das Gesamtgewicht des Pfades zum Startknoten.



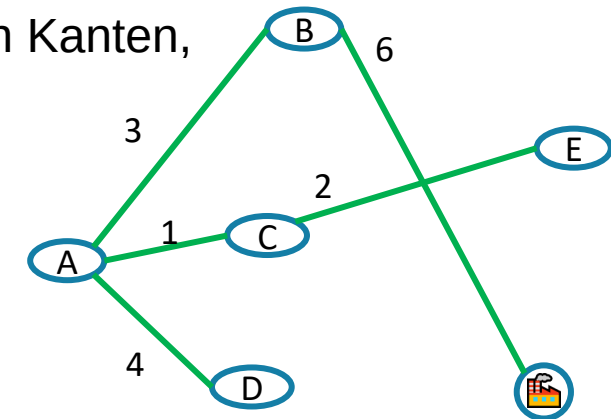
Baum kürzester
Pfade von Kraftwerk

Minimaler Spannbaum

Kein bestimmter Startknoten, da das Ziel nur darin besteht, die Summe der Kantengewichte zu minimieren. Oft nur ein möglicher MST, der die minimale Summe hat.

Alle Knoten sind verbunden

Erfasst die günstigsten Kanten,
die die Konnektivität
aufrechterhalten

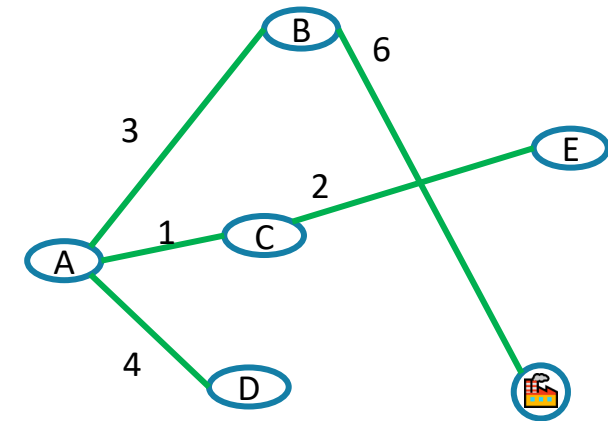
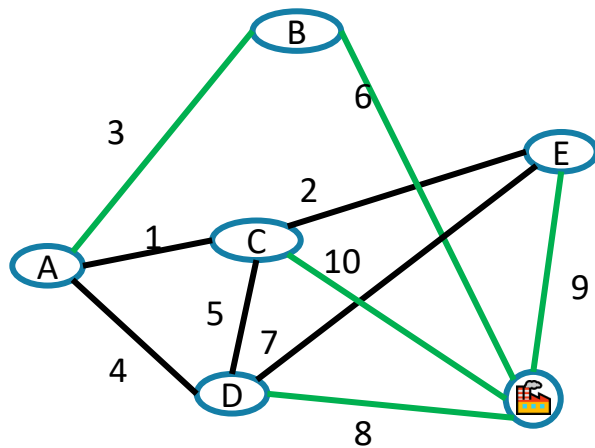


MST des Graphen

Kürzester Pfad vs Minimaler Spannbaum - Anwendung

Minimaler Spannbaum oder Kürzester Pfad?

- Anbindung von Finanzinstitutionen an einen Börsenstandort
 - → Kürzester Pfad, da Signallaufzeit zum Börsenstandort für Hochfrequenzhandel entscheidend ist



Minimale Spannbäume - Anwendungen

- Mieten einer Standleitung von einem Kommunikationsunternehmen, um bspw. mehrere Firmenstandorte direkt miteinander zu verbinden. Unterschiedliche Kosten zwischen Städten.
- Spanning Tree Protocol
 - Zentraler Teil von Switch-Infrastrukturen, mit denen redundante Rechnernetzwerke aufgebaut werden. Ethernet-Technologie muss sicherstellen, dass zwischen zwei Rechnern jeweils nur ein Datenpfad existiert, um Pakete eindeutig weiterleiten zu können. (vgl.: https://de.wikipedia.org/wiki/Spanning_Tree_Protocol)
- Fluglinie
- Bildverarbeitung
 - Gesichtserkennung, Bildsegmentierung (https://en.wikipedia.org/wiki/Minimum_spanning_tree-based_segmentation)
- Erzeugung von Labyrinthen
- https://en.wikipedia.org/wiki/Minimum_spanning_tree#Applications

Minimalen Spannbaum finden

Hier sind zwei Ideen, um ein MST zu finden:

Prim's

Denken Sie Knoten für Knoten

- Erstelle einen Baum über eine Menge von Knoten. Beginne mit einem beliebigen Knoten.
- Prüfe den Abstand aller Nachbarknoten zu der Menge der Knoten.
- Verbinde bei jedem Schritt den Knoten, der am günstigsten verbunden werden kann.

Kruskal's

Kante für Kante denken

- Sortiere die Kanten nach Gewicht. In aufsteigender Reihenfolge.
- Füge Kante zu MST hinzu, wenn sie bisher unverbundene Knoten miteinander verbindet (füge sie nicht hinzu, wenn sie einen Zyklus erzeugen würde)

Beide Ideen funktionieren!!

Interaction Pane Question:

Which of these do you think are more likely to work?

- Thumbs up for vertex by vertex
- Thumbs down for edge by edge
- Clap for both

Prims Algorithmus

Dijkstras Algorithmus

1. Starte an source Knoten
2. Füge nächstliegenden unbearbeiteten Knoten zu SPT hinzu
3. Aktualisiere Distanz von source zu unbearbeiteten Nachbarn
4. Wiederhole ab 2, bis alle Knoten als bearbeitet markiert sind

```
Dijkstra(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed
```

```
while (there are unprocessed vertices):
    let u be the closest unprocessed vertex
    for each edge (u,v) to unprocessed v:
        if (u.dist + weight(u,v) < v.dist):
            v.dist = u.dist + weight(u,v)
            v.predecessor = u
```

```
mark u as processed
```

Idee von Prims Algorithmus:

- Starte an beliebigem Knoten
- Füge nächstliegenden unbearbeiteten Knoten zu MST hinzu
- Aktualisiere Distanz der Nachbarn zum MST, die noch nicht zum MST gehören
- Wiederhole, bis alle Knoten als bearbeitet markiert sind (im MST sind).

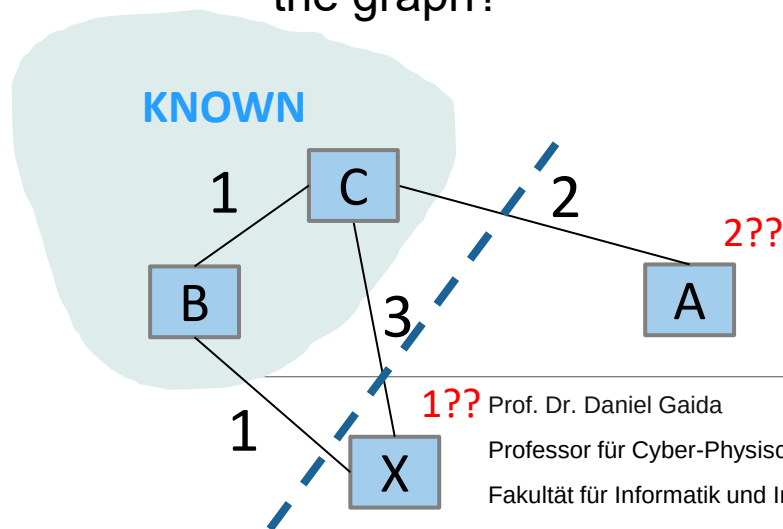
```
Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
```

Adapting Dijkstra's: Prim's Algorithm

- Normally, Dijkstra's checks for a shorter path from the start.
- But MSTs don't care about individual paths, only the overall weight!
- New condition: "would this be a smaller edge to connect the current known set to the rest of the graph?"



```
primShortestPath(G graph, V start)
```

```
Map edgeTo, distTo; Set known;
```

```
initialize distTo with all nodes mapped to  $\infty$ , except start to 0
```

```
PriorityQueue<V> perimeter; perimeter.add(start);
```

```
while (!perimeter.isEmpty()):
```

```
    u = perimeter.removeMin()
```

```
    known.add(u)
```

```
    for each edge (u,v) to unknown v with weight w:
```

```
        oldDist = distTo.get(v)           // previous smallest edge to v
```

```
        newDist = distTo.get(u) + w    // is this a smaller edge to v?
```

```
        if (newDist < oldDist):
```

```
            distTo.put(v, newDist)
```

```
            edgeTo.put(v, u)
```

```
            if (perimeter.contains(v)):
```

```
                perimeter.changePriority(v, newDist)
```

```
            else:
```

```
                perimeter.add(v, newDist)
```

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

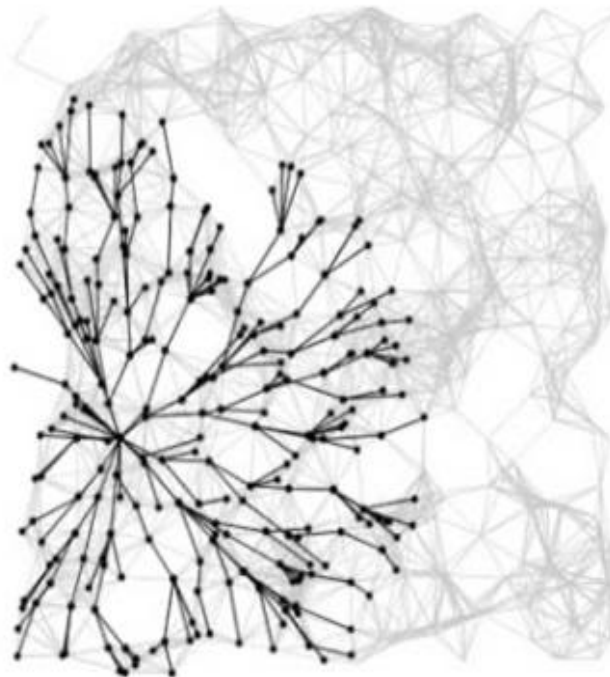
Prims Algorithmus - Demos und Visualisierungen

Visualisierung:

<https://www.youtube.com/watch?v=vmWSnkBVvQ0&t=26s>

Dijkstras Algorithmus

Dijkstra geht radial von der Quelle aus, weil er die Kanten nach den geringsten Kosten des Pfades von der Quelle aus wählt



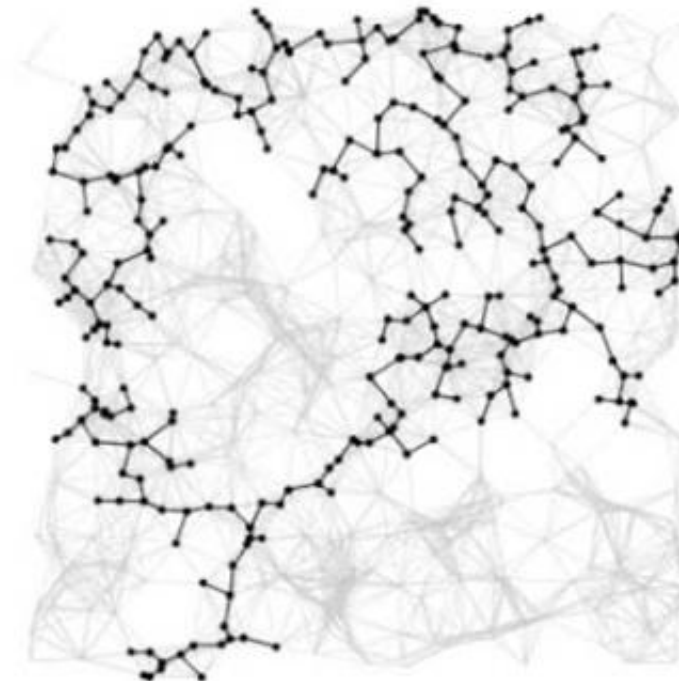
26.08.2026

Seite 23

stitut für Informatik

Prims Algorithmus

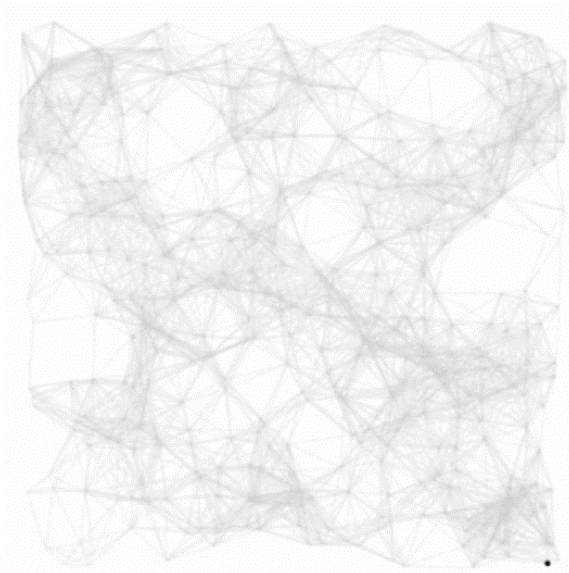
Prim springt im Graphen (im Umfang) herum, weil er die Kanten nach ihrem Gewicht auswählt (es gibt keine Quelle)



Prims Algorithmus - Demos und Visualisierungen

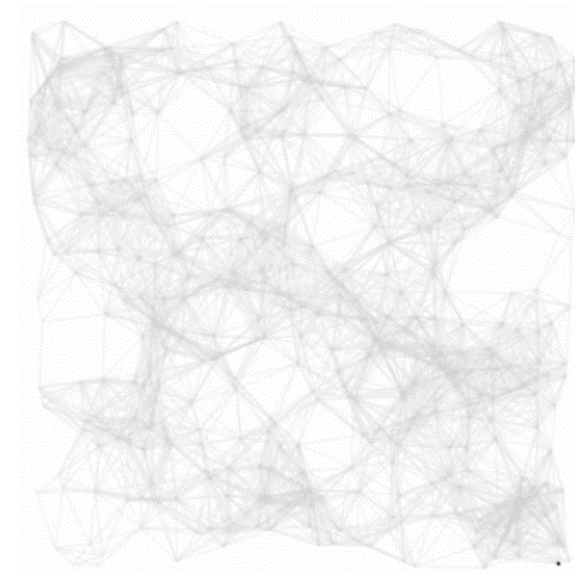
Dijkstras Algorithmus

Dijkstra geht radial von der Quelle aus, weil er die Kanten nach den geringsten Kosten des Pfades von der Quelle aus wählt



Prims Algorithmus

Prim springt im Graphen (im Umfang) herum, weil er die Kanten nach ihrem Gewicht auswählt (es gibt keine Quelle)



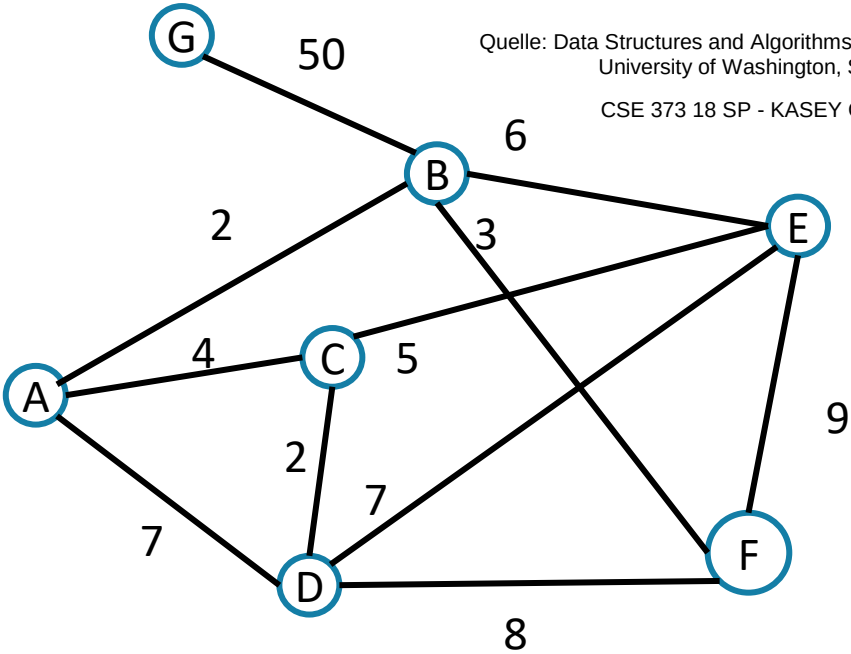
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	
B	∞		
C	∞		
D	∞		
E	∞		
F	∞		
G	∞		

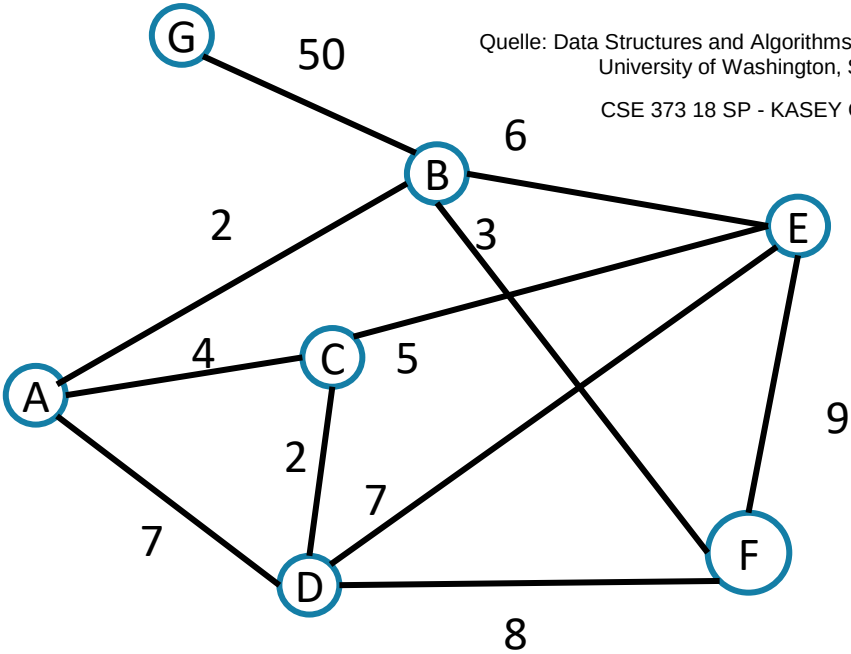
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	✓
B	2	A	
C	4	A	
D	7	A	
E	∞		
F	∞		
G	∞		

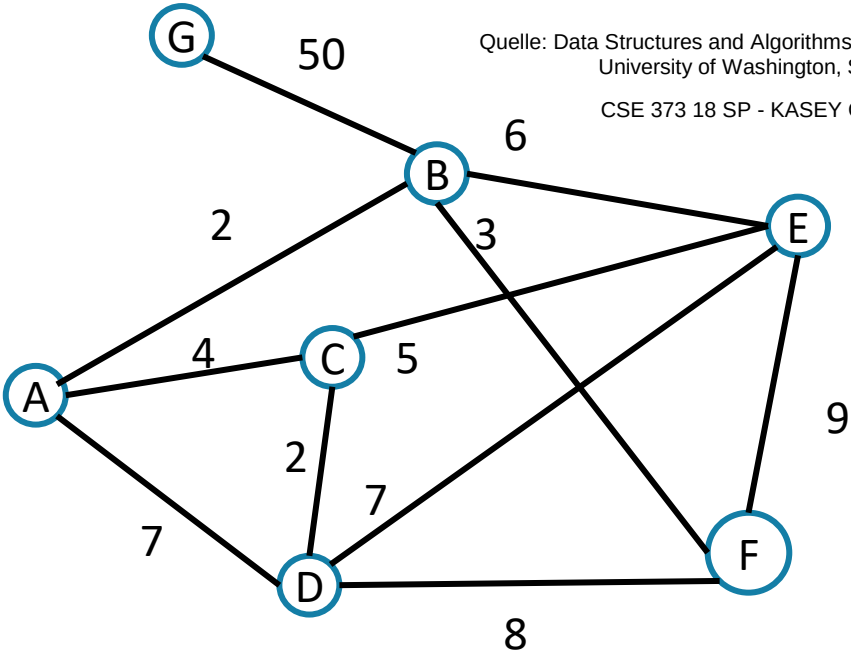
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	✓
B	2	A	✓
C	4	A	
D	7	A	
E	6	B	
F	3	B	
G	50	B	

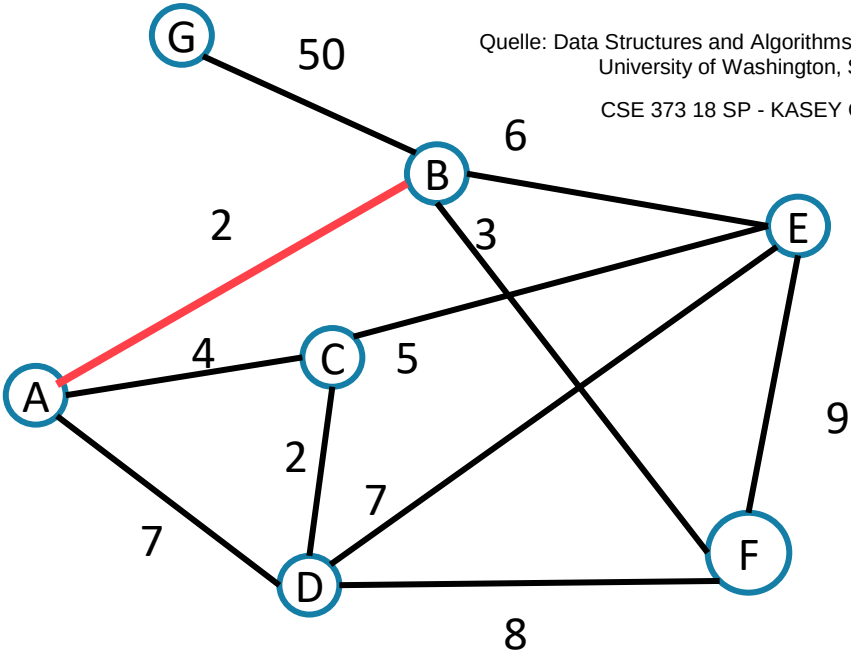
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	✓
B	2	A	✓
C	4	A	
D	7	A	
E	6	B	
F	3	B	✓
G	50	B	

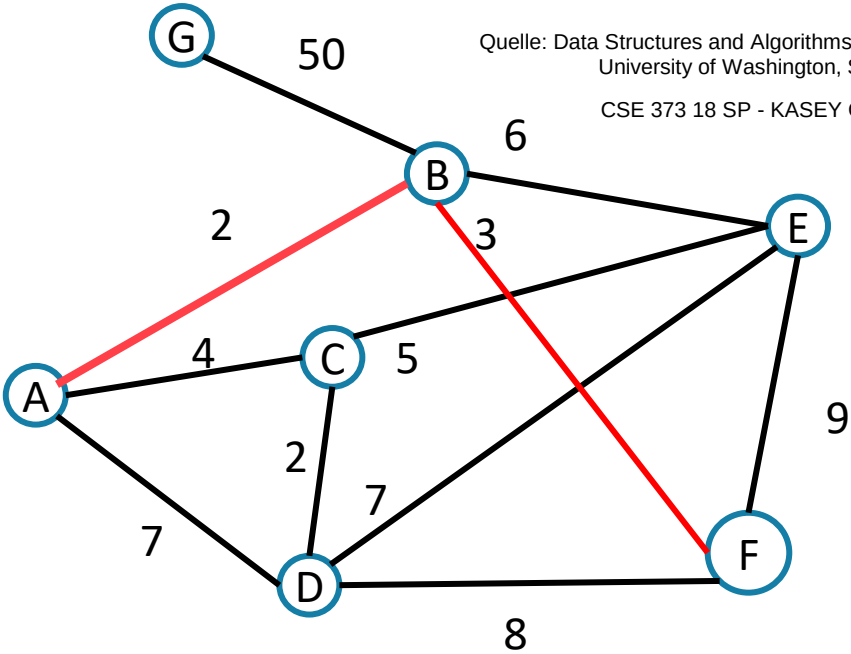
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	✓
B	2	A	✓
C	4	A	✓
D	7 -2	-A- C	
E	6 -5	-B- C	
F	3	B	✓
G	50	B	

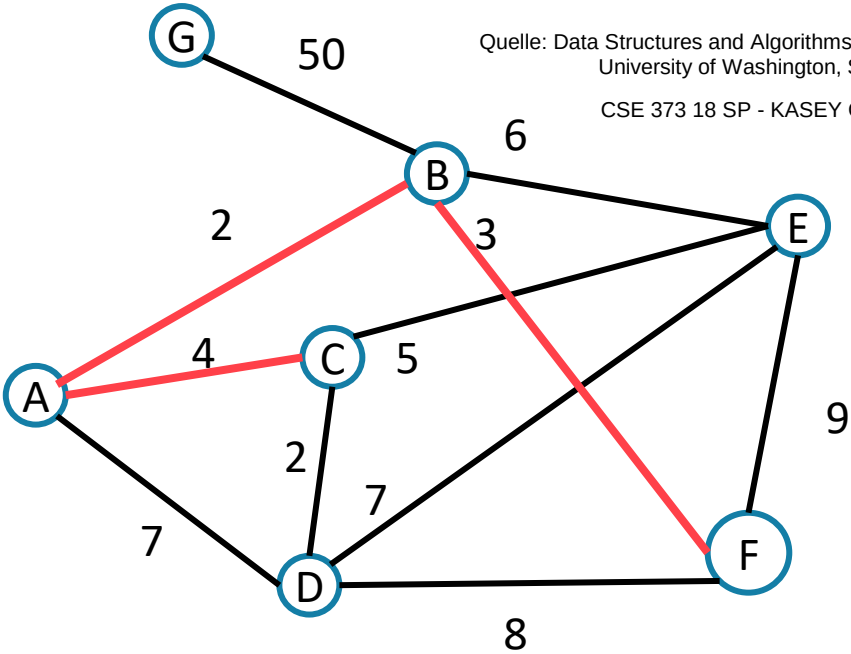
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	✓
B	2	A	✓
C	4	A	✓
D	7 -2	-A- C	✓
E	6 -5	-B- C	
F	3	B	✓
G	50	B	

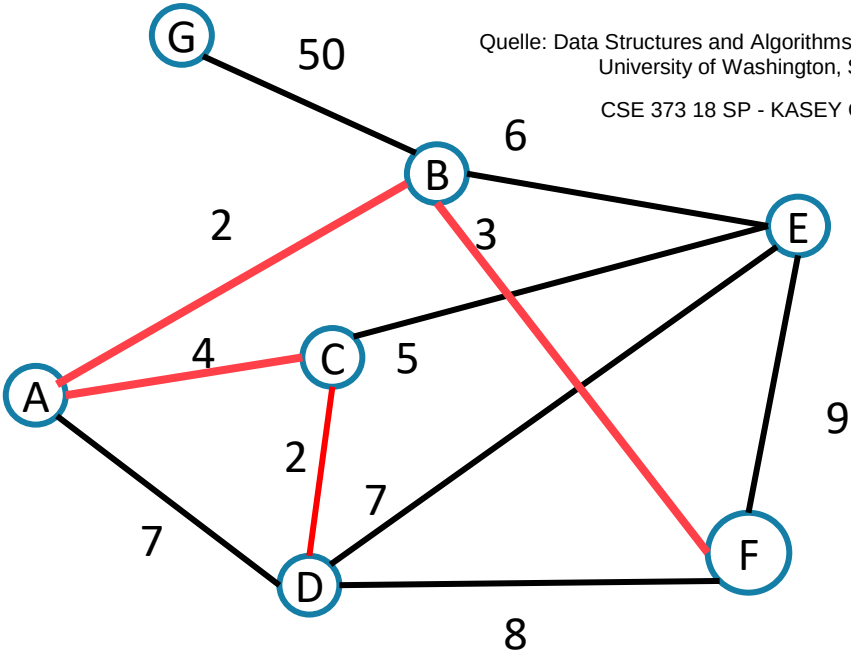
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	✓
B	2	A	✓
C	4	A	✓
D	7-2	-A- C	✓
E	6-5	-B- C	✓
F	3	B	✓
G	50	B	✓

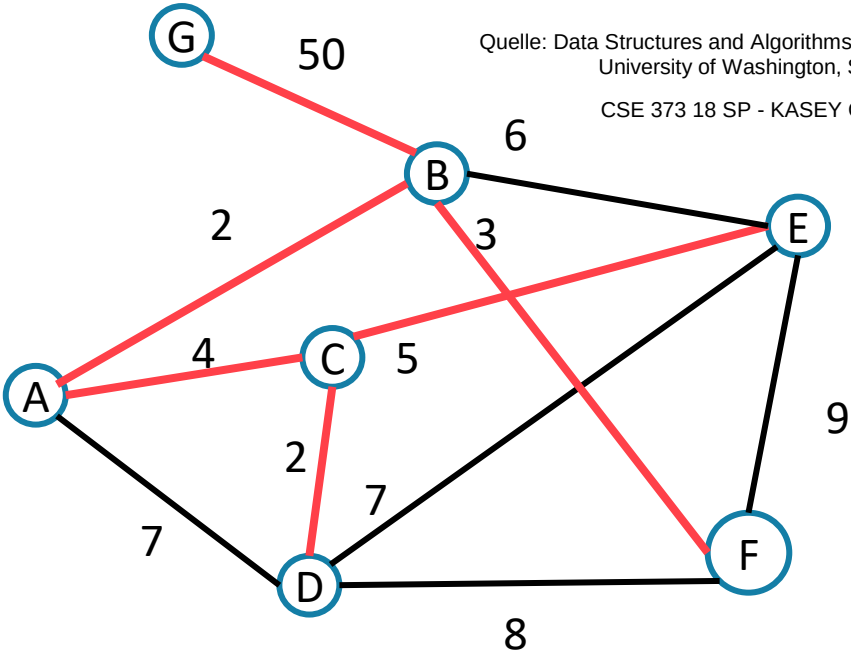
Prims Algorithmus – Übung

```

Prims(Graph G, Vertex source)
    initialize distances to ∞
    mark source as distance 0 (source.dist = 0)
    mark all vertices unprocessed

    while (there are unprocessed vertices):
        let u be the closest unprocessed vertex
        for each edge (u,v) to unprocessed v:
            if (weight(u,v) < v.dist):
                v.dist = weight(u,v)
                v.predecessor = u

        mark u as processed
    
```



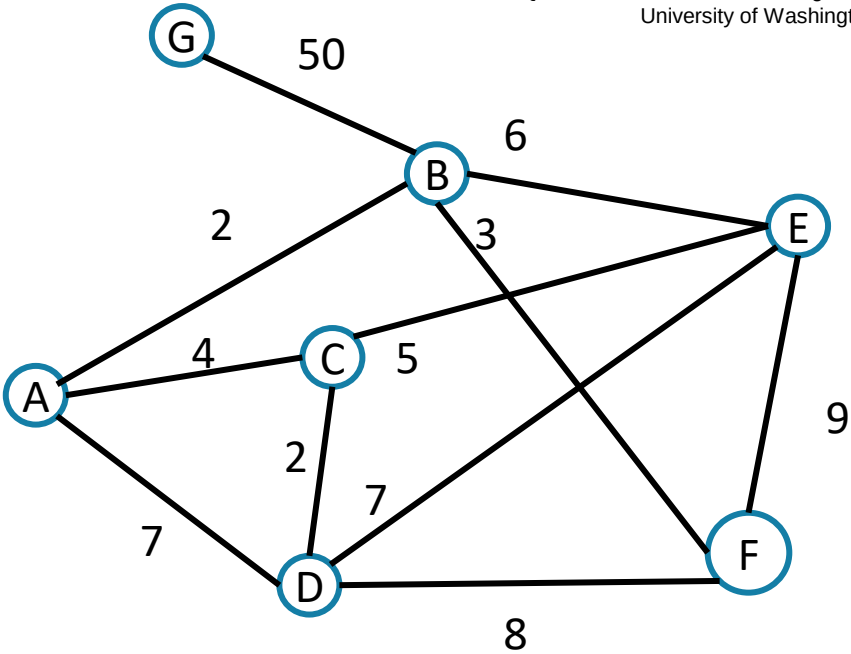
Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 18 SP - KASEY CHAMPION

Knoten	Distanz	Vorgänger	Bearbeitet
A	0	X	✓
B	2	A	✓
C	4	A	✓
D	7 -2	-A- C	✓
E	6 -5	-B- C	✓
F	3	B	✓
G	50	B	✓

Prims Algorithmus – Übung

```
primsShortestPath(G graph, V start)
  Map edgeTo, distTo; Set known;
  initialize distTo with all nodes mapped to ∞, except start to 0
  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()
    known.add(u)
    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous smallest edge to v
      newDist = w                  // is this a smaller edge to v?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```



Vertex	Distance	Best Edge	Processed
A	0	X	✓
B	2	(A, B)	✓
C	4	(A, C)	✓
D	2	(C, D)	✓
E	5	(C, E)	✓
F	3	(B, F)	✓
G	50	(B, G)	✓

Prims Algorithmus – Asymptotische Laufzeit

Damit ein MST existieren kann, darf $|E|$ nicht kleiner als $|V|$ sein, also muss die Anzahl Kanten dominieren:

→ $\Theta(m \log(n))$

$\Theta(|V| \log |V| + |E| \log |V|)$

$\Theta(|V| \log |V|)$

$\Theta(|E| \log |V|)$

```
primsShortestPath(G graph, V start)
    Map edgeTo, distTo; Set known;
    initialize distTo with all nodes mapped to  $\infty$ , except start to 0
    PriorityQueue<V> perimeter; perimeter.add(start);

    while (!perimeter.isEmpty()):
        u = perimeter.removeMin()
        known.add(u)
        for each edge (u,v) to unknown v with weight w:
            oldDist = distTo.get(v)      // previous smallest edge to v
            newDist = w // is this a smaller edge to v?
            if (newDist < oldDist):
                distTo.put(v, newDist)
                edgeTo.put(v, u)
                if (perimeter.contains(v)):
                    perimeter.changePriority(v, newDist)
                else:
                    perimeter.add(v, newDist)
```

Kruskals Algorithmus

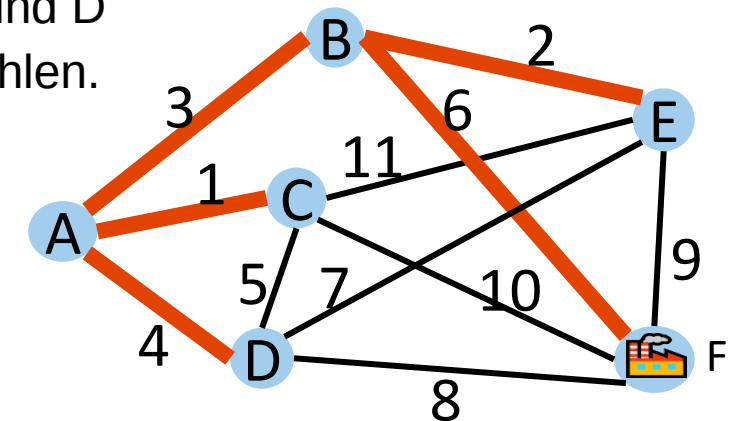
Angenommen, der Minimale Spannbaum (MST) rechts unten wurde mit Prim's Algorithmus erzeugt

Beobachtung: Wir wählen grundsätzlich alle leichtesten Kanten des gesamten Graphen (1, 2, 3, 4, 6).

- Die einzige Ausnahme war Kante (C,D) mit 5. Warum sollten wir Kante (C,D) nicht hinzufügen?
- Weil Hinzufügen von 5 einen Zyklus erzeugen würde, und um A, C und D zu verbinden, würden wir lieber 1 und 4 als 1 und 5 oder 4 und 5 wählen.

Prim denkt „Knoten für Knoten“, aber was ist, wenn wir stattdessen „Kante für Kante“ denken?

- Beginnen Sie mit der leichtesten Kante im gesamten Graphen und arbeiten Sie sich nach oben zu schwereren Kanten
- Fügen Sie Kante zum MST hinzu, solange sie zwei bisher durch den MST unverbundene Knoten verbindet (d.h. fügen Sie keine Kante hinzu, die einen Zyklus erzeugen würde)



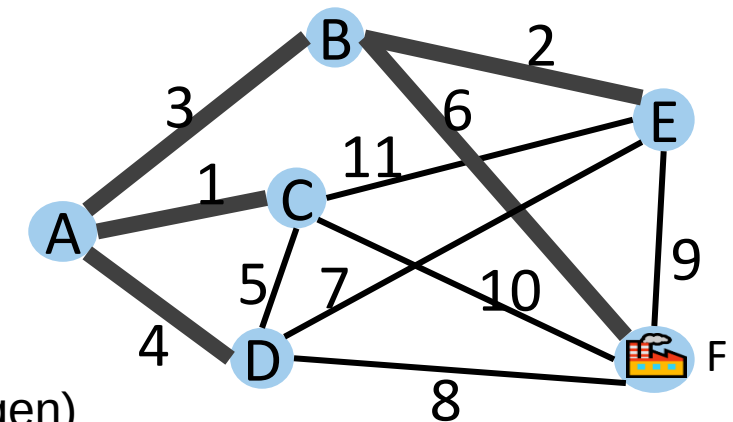
Kruskals Algorithmus

„Kante für Kante“ denken:

- Beginnen Sie mit der leichtesten Kante im gesamten Graphen und arbeiten Sie sich nach oben
- Füge die Kante zum MST hinzu, solange sie zwei durch den MST unverbundene Knoten verbindet

Aufbau eines MST „Kante für Kante“ für diesen Graphen:

- Kante (A,C) mit Gewicht 1 hinzufügen
- Kante (B,E) mit Gewicht 2 hinzufügen
- Kante (A,B) mit Gewicht 3 hinzufügen
- Kante (A,D) mit Gewicht 4 hinzufügen
- Kante (C,D) mit Gewicht 5 überspringen (würde einen Zyklus erzeugen)
- Kante (B,F) mit Gewicht 6 hinzufügen
- Fertig: alle Knoten im MST!

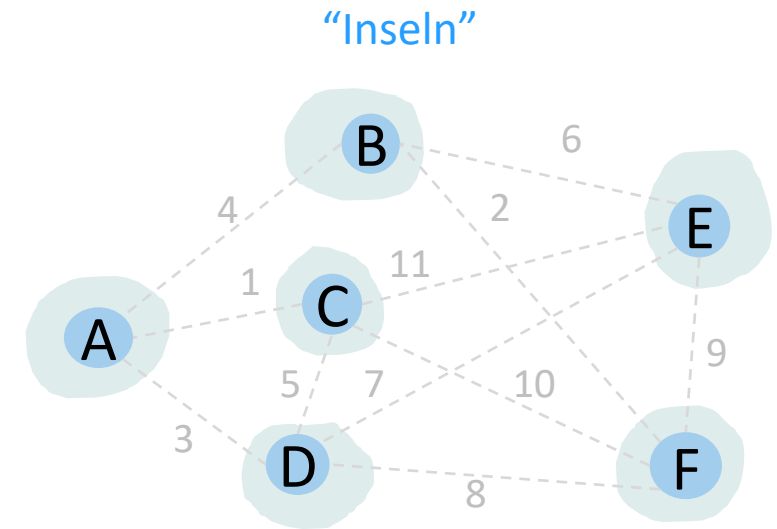


Kruskals Algorithmus

Dieser Ansatz „Kante für Kante“ ist die Funktionsweise von Kruskals Algorithmus!

Wichtige Intuition: Kruskals erfasst „Inseln“ von Knoten

- Beginne mit jedem Knoten als eigene „Insel“.



```
kruskalMST(G graph)
  Set(?) islands; Set finalMST;
  initialize islands with each vertex as tiny island
  sort all edges by weight (smallest to largest)

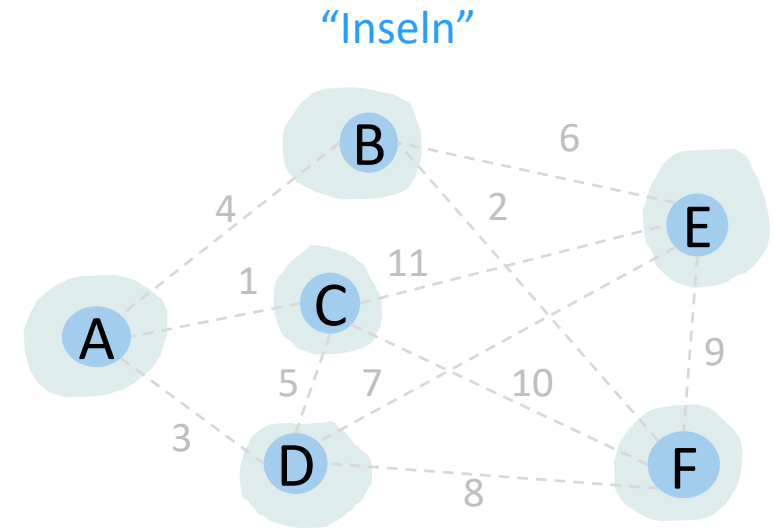
  ...
```

Kruskals Algorithmus

Dieser Ansatz „Kante für Kante“ ist die Funktionsweise von Kruskals Algorithmus!

Wichtige Intuition: Kruskals erfasst „Inseln“ von Knoten

- Beginne mit jedem Knoten als eigene „Insel“.
-
- Wenn Kante zwei Knoten in verschiedenen „Inseln“ verbindet, füge sie zum MST hinzu!



```
kruskalMST(G graph)
    Set(?) islands; Set finalMST;
    initialize islands with each vertex as tiny island
    sort all edges by weight (smallest to largest)

    for each edge (u,v) in ascending order:
        uIsl = islands.find(u)
        vIsl = islands.find(v)
        if (uIsl != vIsl):
            ...
```

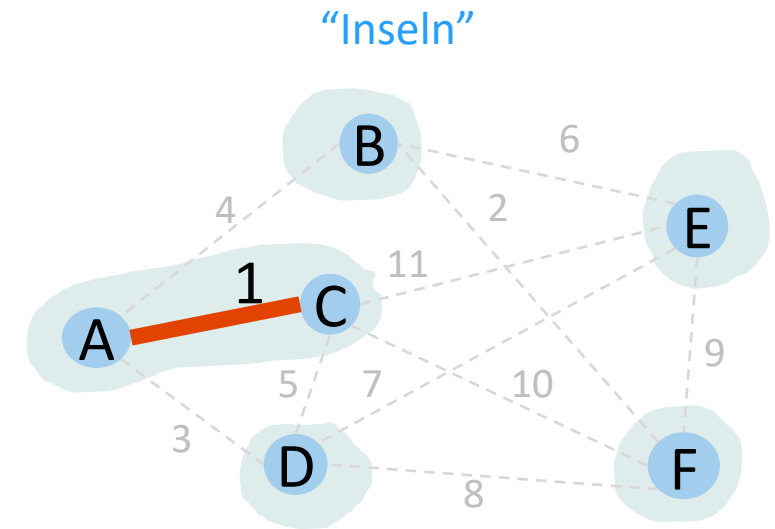
Kruskals Algorithmus

Dieser Ansatz „Kante für Kante“ ist die Funktionsweise von Kruskals Algorithmus!

Wichtige Intuition: Kruskals erfasst „Inseln“ von Knoten

- Beginne mit jedem Knoten als eigene „Insel“.
- Wenn Kante zwei Knoten innerhalb derselben „Insel“ verbindet, bildet sie einen Zyklus! Verwerfe sie.
- Wenn Kante zwei Knoten in verschiedenen „Inseln“ verbindet, füge sie zum MST hinzu! Nun müssen diese „Inseln“ kombiniert werden.

finalMST:



```
kruskalMST(G graph)
```

```
Set(?) islands; Set finalMST;
```

```
initialize islands with each vertex as tiny island
```

```
sort all edges by weight (smallest to largest)
```

```
for each edge (u,v) in ascending order:
```

```
  uIsl = islands.find(u)
```

```
  vIsl = islands.find(v)
```

```
  if (uIsl != vIsl):
```

```
    finalMST.add(edge (u, v))
```

```
    islands.union(u, v)
```

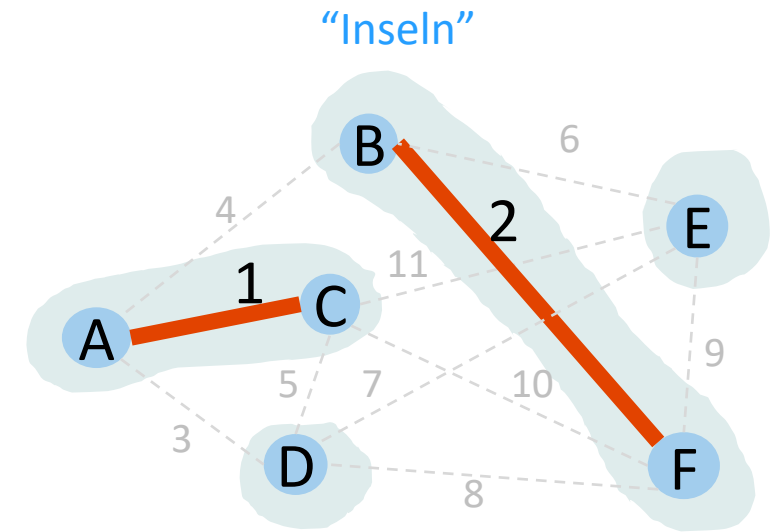
Kruskals Algorithmus

Dieser Ansatz „Kante für Kante“ ist die Funktionsweise von Kruskals Algorithmus!

Wichtige Intuition: Kruskals erfasst „Inseln“ von Knoten

- Beginne mit jedem Knoten als eigene „Insel“.
- Wenn Kante zwei Knoten innerhalb derselben „Insel“ verbindet, bildet sie einen Zyklus! Verwerfe sie.
- Wenn Kante zwei Knoten in verschiedenen „Inseln“ verbindet, füge sie zum MST hinzu! Nun müssen diese „Inseln“ kombiniert werden.

finalMST:



```
kruskalMST(G graph)
    Set(?) islands; Set finalMST;
    initialize islands with each vertex as tiny island
    sort all edges by weight (smallest to largest)

    for each edge (u,v) in ascending order:
        uIsl = islands.find(u)
        vIsl = islands.find(v)
        if (uIsl != vIsl):
            finalMST.add(edge (u, v))
            islands.union(u, v)
```

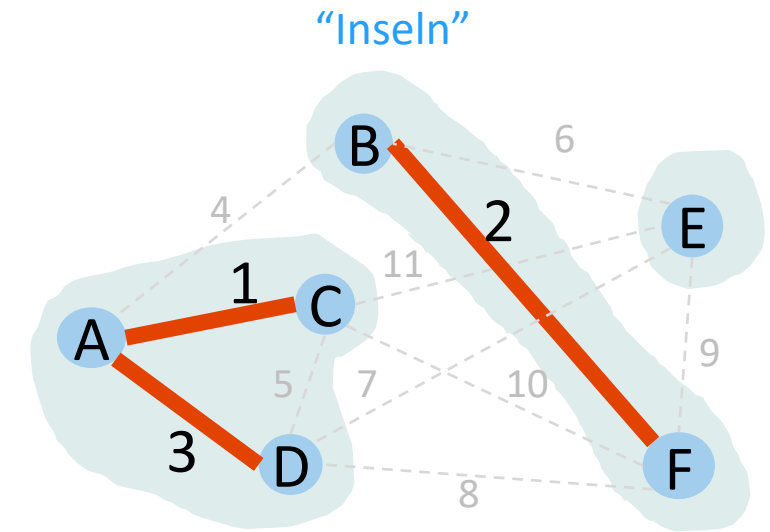

Kruskals Algorithmus

Dieser Ansatz „Kante für Kante“ ist die Funktionsweise von Kruskals Algorithmus!

Wichtige Intuition: Kruskals erfasst „Inseln“ von Knoten

- Beginne mit jedem Knoten als eigene „Insel“.
- Wenn Kante zwei Knoten innerhalb derselben „Insel“ verbindet, bildet sie einen Zyklus! Verwerfe sie.
- Wenn Kante zwei Knoten in verschiedenen „Inseln“ verbindet, füge sie zum MST hinzu! Nun müssen diese „Inseln“ kombiniert werden.

finalMST:



```
kruskalMST(G graph)
    Set(?) islands; Set finalMST;
    initialize islands with each vertex as tiny island
    sort all edges by weight (smallest to largest)

    for each edge (u,v) in ascending order:
        uIsl = islands.find(u)
        vIsl = islands.find(v)
        if (uIsl != vIsl):
            finalMST.add(edge (u, v))
            islands.union(u, v)
```

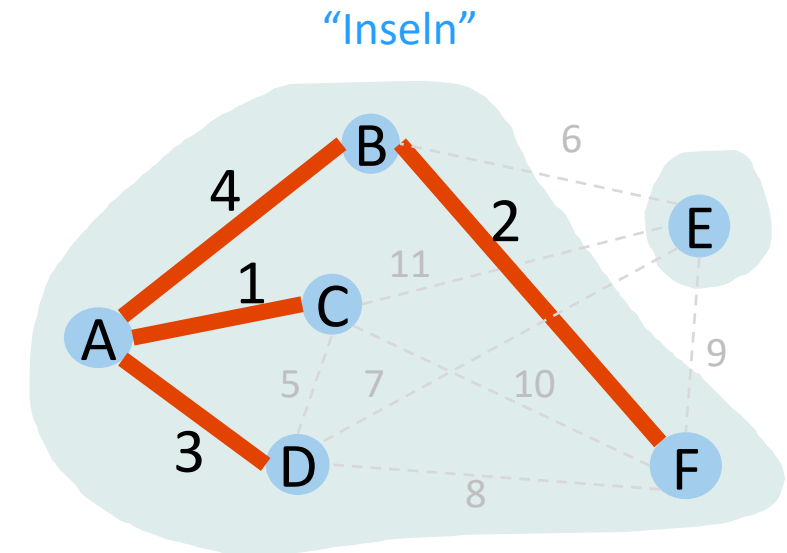
Kruskals Algorithmus

Dieser Ansatz „Kante für Kante“ ist die Funktionsweise von Kruskals Algorithmus!

Wichtige Intuition: Kruskals erfasst „Inseln“ von Knoten

- Beginne mit jedem Knoten als eigene „Insel“.
- Wenn Kante zwei Knoten innerhalb derselben „Insel“ verbindet, bildet sie einen Zyklus! Verwerfe sie.
- Wenn Kante zwei Knoten in verschiedenen „Inseln“ verbindet, füge sie zum MST hinzu! Nun müssen diese „Inseln“ kombiniert werden.

finalMST:



```
kruskalMST(G graph)
    Set(?) islands; Set finalMST;
    initialize islands with each vertex as tiny island
    sort all edges by weight (smallest to largest)

    for each edge (u,v) in ascending order:
        uIsl = islands.find(u)
        vIsl = islands.find(v)
        if (uIsl != vIsl):
            finalMST.add(edge (u, v))
            islands.union(u, v)
```

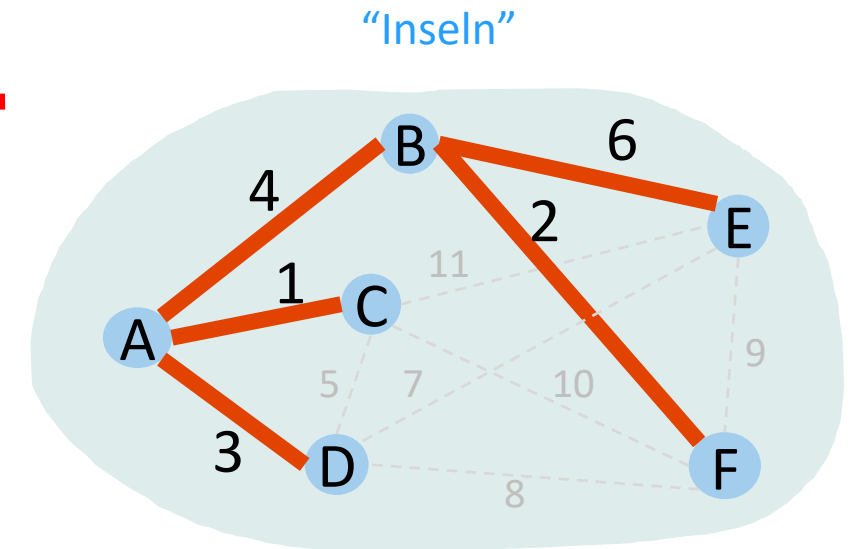
Kruskals Algorithmus

Dieser Ansatz „Kante für Kante“ ist die Funktionsweise von Kruskals Algorithmus!

Wichtige Intuition: Kruskals erfasst „Inseln“ von Knoten

- Beginne mit jedem Knoten als eigene „Insel“.
- Wenn Kante zwei Knoten innerhalb derselben „Insel“ verbindet, bildet sie einen Zyklus! Verwerfe sie.
- Wenn Kante zwei Knoten in verschiedenen „Inseln“ verbindet, füge sie zum MST hinzu! Nun müssen diese „Inseln“ kombiniert werden.

finalMST:



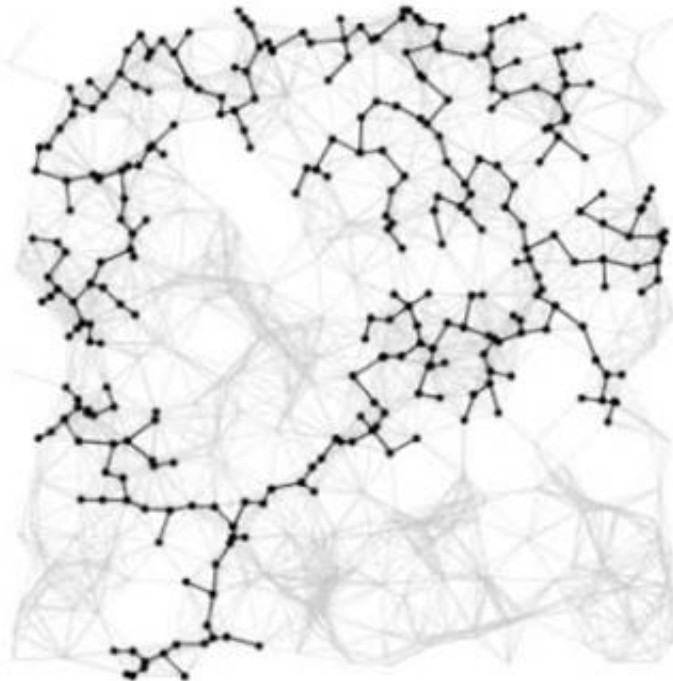
```
kruskalMST(G graph)
    Set(?) islands; Set finalMST;
    initialize islands with each vertex as tiny island
    sort all edges by weight (smallest to largest)

    for each edge (u,v) in ascending order:
        uIsl = islands.find(u)
        vIsl = islands.find(v)
        if (uIsl != vIsl):
            finalMST.add(edge (u, v))
            islands.union(u, v)
```

Kruskals Algorithmus - Demos und Visualisierungen

Prims Algorithmus

Prim springt im Graphen (im Umfang) herum, weil er die Kanten nach ihrem Gewicht auswählt (es gibt keine Quelle)



26.08.2026

Seite 47

Institut für Informatik

Kruskals Algorithmus

Kruskal springt im gesamten Graphen herum, weil er Kanten rein nach dem Kantengewicht auswählt (und dabei Zyklen verhindert)

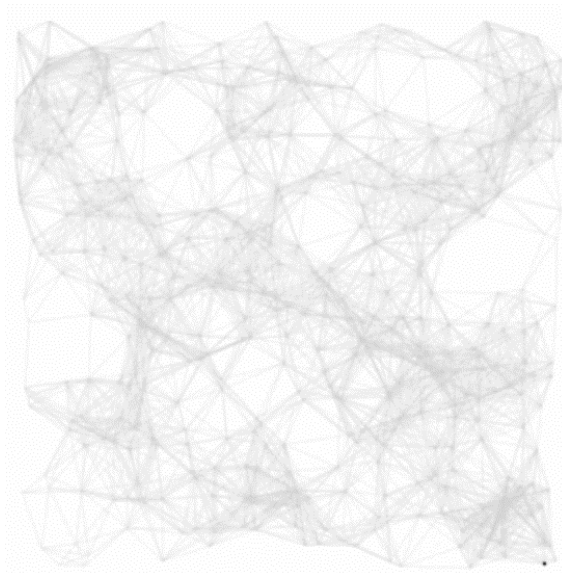


[Kruskal's Visualization Video](#)

Kruskals Algorithmus - Demos und Visualisierungen

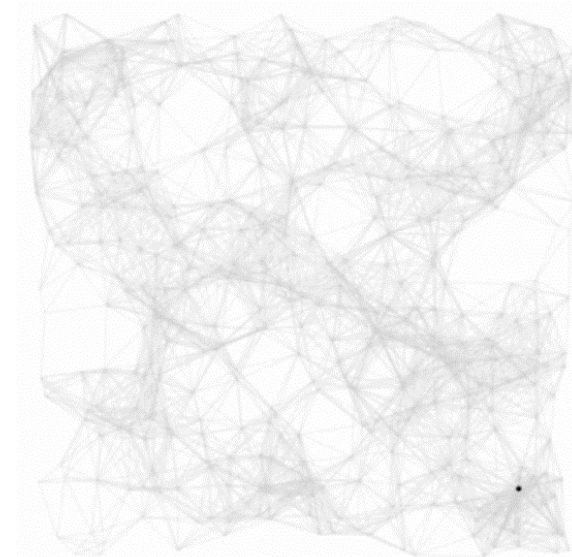
Prims Algorithmus

Prim springt im Graphen (im Umfang) herum, weil er die Kanten nach ihrem Gewicht auswählt (es gibt keine Quelle)



Kruskals Algorithmus

Kruskal springt im gesamten Graphen herum, weil er Kanten rein nach dem Kantengewicht auswählt (und dabei Zyklen verhindert)



Kruskals Algorithmus

KruskalMST(Graph G)

```
    initialize each vertex to be its own island
    sort the edges by weight
    foreach(edge (u, v) in sorted order){
        if(u and v are on different islands){
            add (u,v) to the MST
            Update u and v to be on the same island
        }
    }
```

Kruskals Algorithmus – Übung

KruskalMST(Graph G)

 initialize each vertex to be its own island

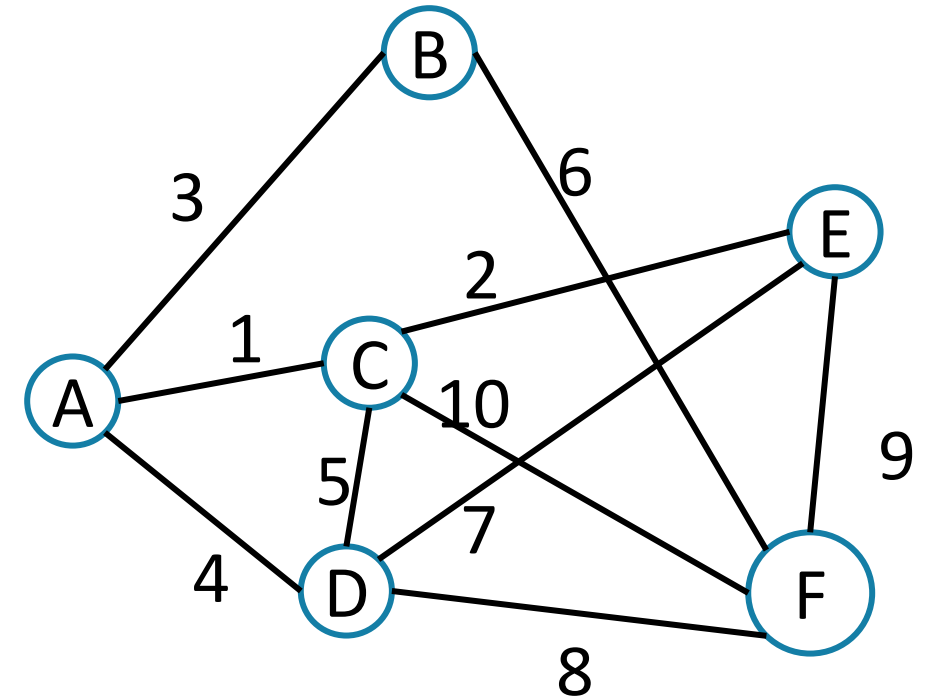
 sort the edges by weight

 foreach(edge (u, v) in sorted order){

 if(u and v are on different islands){

 add (u,v) to the MST

 Update u and v to be on the same island



Edge	Include?	Reason
(A,C)		
(C,E)		
(A,B)		
(A,D)		
(C,D)		

Edge (cont.)	Include?	Reason
(B,F)		
(D,E)		
(D,F)		
(E,F)		
(C,F)		

Kruskals Algorithmus – Übung

KruskalMST(Graph G)

initialize each vertex to be its own island

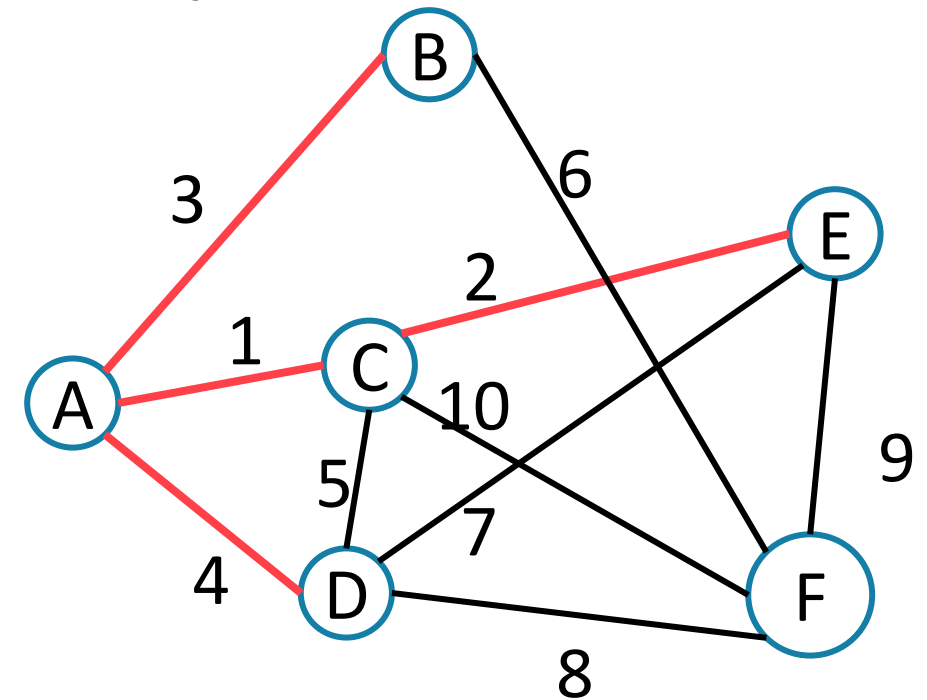
sort the edges by weight

foreach(edge (u, v) in sorted order){

if(u and v are on different islands){

add (u,v) to the MST

Update u and v to be on the same island



Edge	Include?	Reason
(A,C)	Yes	
(C,E)	Yes	
(A,B)	Yes	
(A,D)	Yes	
(C,D)	No	Cycle A,C,D,A

Edge (cont.)	Include?	Reason
(B,F)		
(D,E)		
(D,F)		
(E,F)		
(C,F)		

Kruskals Algorithmus – Übung

KruskalMST(Graph G)

 initialize each vertex to be its own island

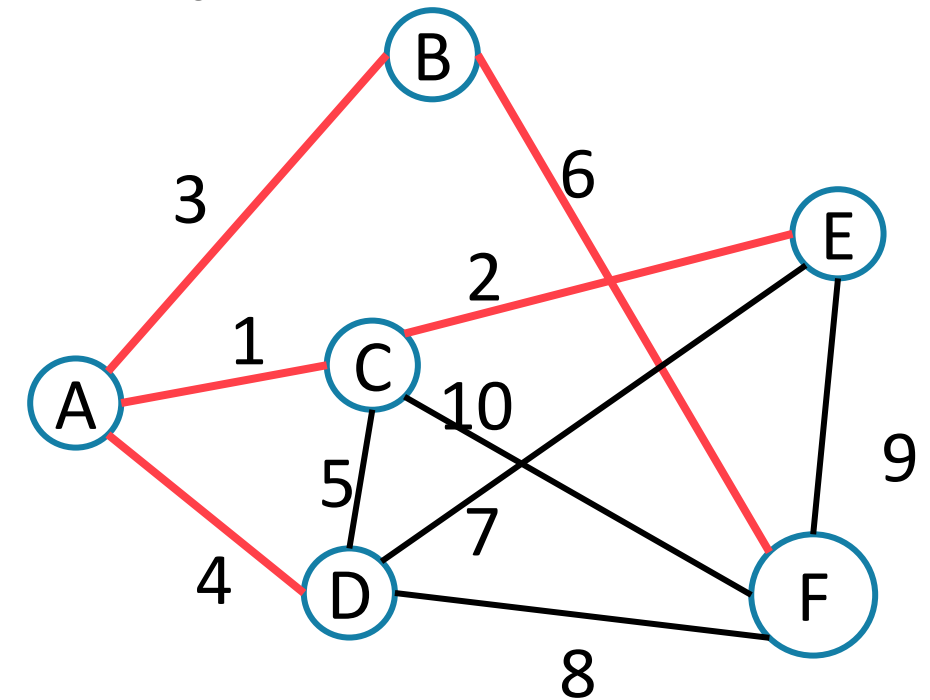
 sort the edges by weight

 foreach(edge (u, v) in sorted order){

 if(u and v are on different islands){

 add (u,v) to the MST

 Update u and v to be on the same island



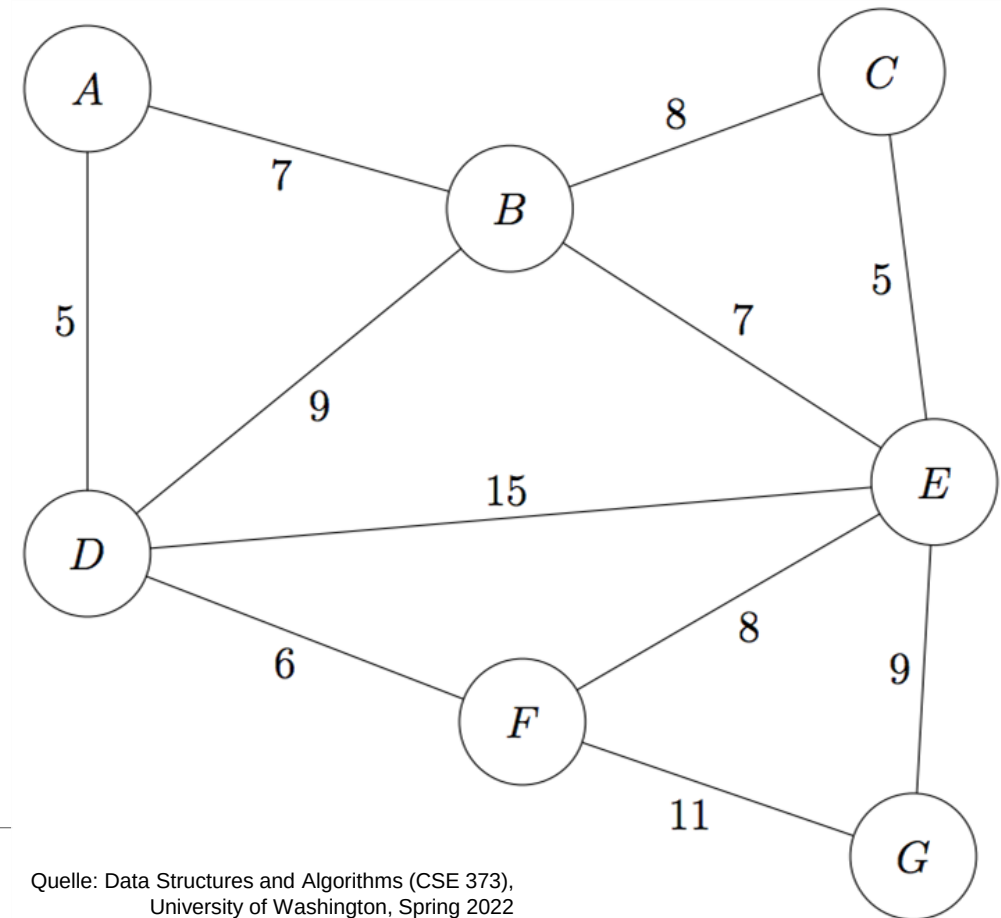
Edge	Include?	Reason
(A,C)	Yes	
(C,E)	Yes	
(A,B)	Yes	
(A,D)	Yes	
(C,D)	No	Cycle A,C,D,A

Edge (cont.)	Include?	Reason
(B,F)	Yes	
(D,E)	No	Cycle A,C,E,D,A
(D,F)	No	Cycle A,D,F,B,A
(E,F)	No	Cycle A,C,E,F,D,A
(C,F)	No	Cycle C,A,B,F,C

Kruskals Algorithmus – Übung 2

Führen Sie Kruskals Algorithmus auf dem folgenden Graphen aus, um den Minimalen Spannbaum des Graphen zu finden.

```
KruskalMST(Graph G)
  initialize each vertex to be an independent island
  sort the edges by weight
  foreach(edge (u, v) in sorted order){
    if(u and v are on different islands){
      add (u,v) to the MST
      update u and v to be on the same island
    }
  }
```



Algorithmus von Kruskal

KruskalMST(Graph G)

```
initialize each vertex to be its own island  
sort the edges by weight  
foreach(edge (u, v) in sorted order){  
    if(u and v are on different islands){  
        add (u,v) to the MST  
        Update u and v to be on the same island  
    }  
}
```

Die fett hervorgehobenen Codezeilen sind ein wenig skizzenhaft.

Können wir eine unserer ADTs verwenden?

Übersicht für heute

- Minimale Spannbäume
 - Algorithmus von Prim
 - Algorithmus von Kruskal

- **ADT**
 - **Menge**
 - **Disjunkte Mengen**

- Implementierung des ADT Disjunkter Mengen
 - Wald disjunkter Mengen
 - Quick-Union, Weighted Quick-Union
 - Pfadverkürzung

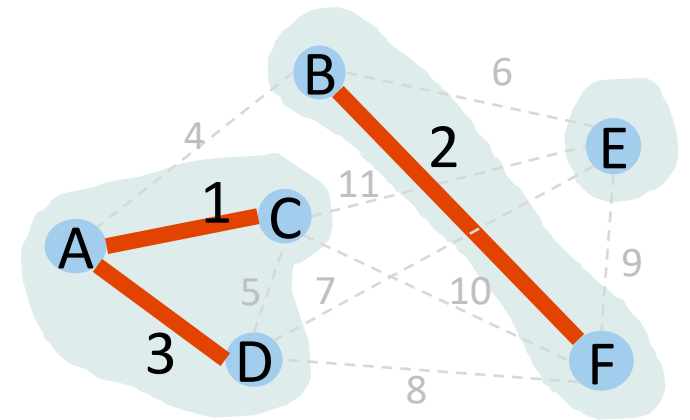
- Laufzeit des Algorithmus von Kruskal mit optimaler Implementierung des ADT Disjunkter Mengen

Kruskals Algorithmus

Wahl eines ADTs

Kruskal muss herausfinden, zu welcher Insel ein Knoten gehört, und muss diese Inseln zusammenführen

- Unsere bestehenden ADTs eignen sich nicht gut für die Vereinigung von zwei Mengen (Menge = Insel) ...
- Definieren wir einen neuen ADT!



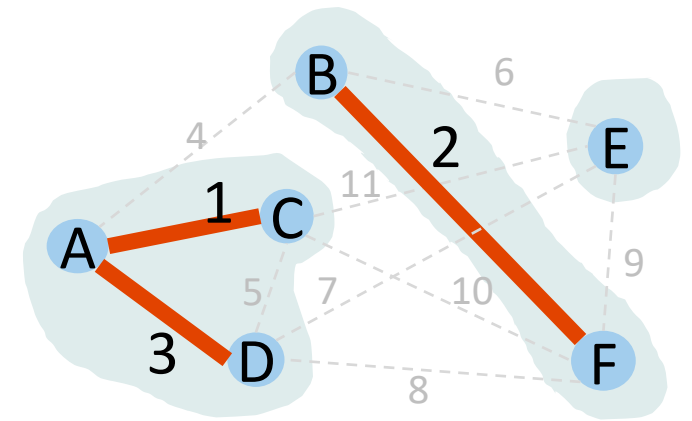
```
kruskalMST(G graph)
  Set(?) islands; Set finalMST;
  initialize islands with each vertex as tiny island
  sort all edges by weight (smallest to largest)

  for each edge (u,v) in ascending order:
    uIsl = islands.find(u)
    vIsl = islands.find(v)
    if (uIsl != vIsl):
      finalMST.add(edge (u, v))
      islands.union(u, v)
```

ADT Disjunkter Mengen (aka „Union-Find“)

Kruskal verwendet den ADT Disjunkter Mengen

- Eine einzelne Instanz dieses ADT enthält eine „Familie“ von Mengen, die disjunkt sind (kein Element gehört zu mehreren Mengen)



```
kruskalMST(G graph)
    DisjointSets<V> islands; Set finalMST;
    initialize islands with each vertex as tiny island
    sort all edges by weight (smallest to largest)

    for each edge (u,v) in ascending order:
        uIsl = islands.find(u)
        vIsl = islands.find(v)
        if (uIsl != vIsl):
            finalMST.add(edge (u, v))
            islands.union(u, v)
```

ADT DISJUNKTER MENGEN

State

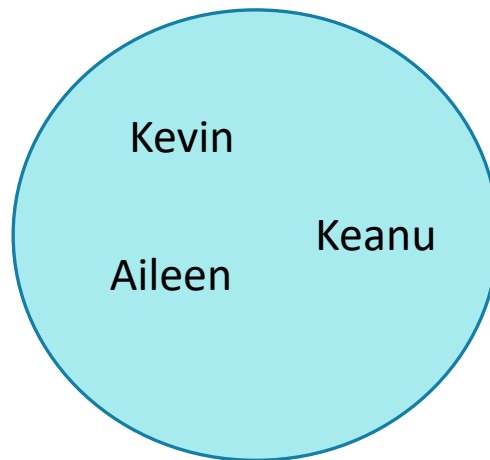
- Familie von Mengen
- disjunkt: keine gemeinsamen Elemente
- Jede Menge hat einen Repräsentanten (either a member or a unique ID)

Behavior

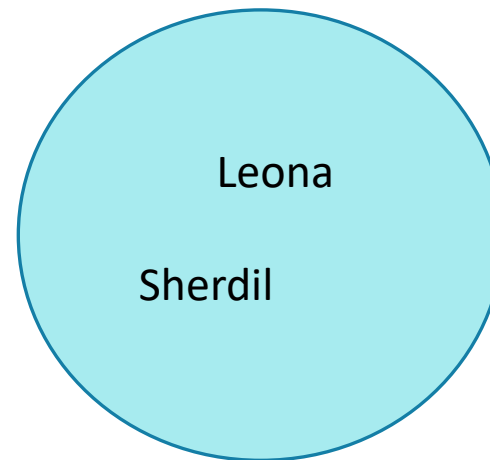
- makeSet(value) – new set with value as only member (and representative)
- find(value) – return representative of the set containing value
- union(x, y) – combine sets containing x and y into one set with all elements, choose single new representative

Disjunkte Mengen in der Mathematik

- „In der Mengenlehre heißen zwei Mengen disjunkt, wenn sie kein gemeinsames Element haben.“ - Wikipedia
- disjunkt = nicht überlappend

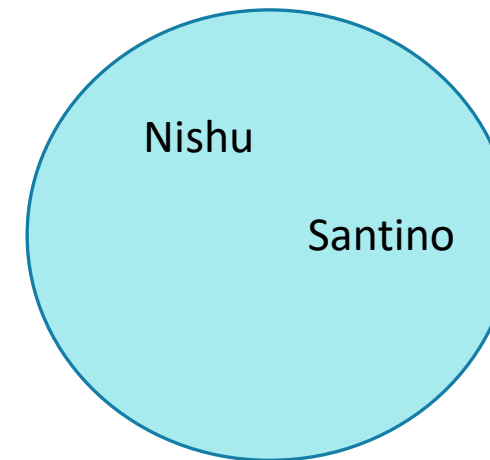


Menge #1

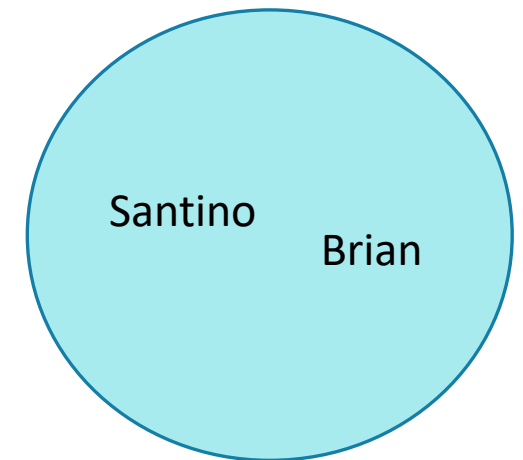


Menge #2

Diese beiden Mengen sind disjunkte Mengen



Menge #3



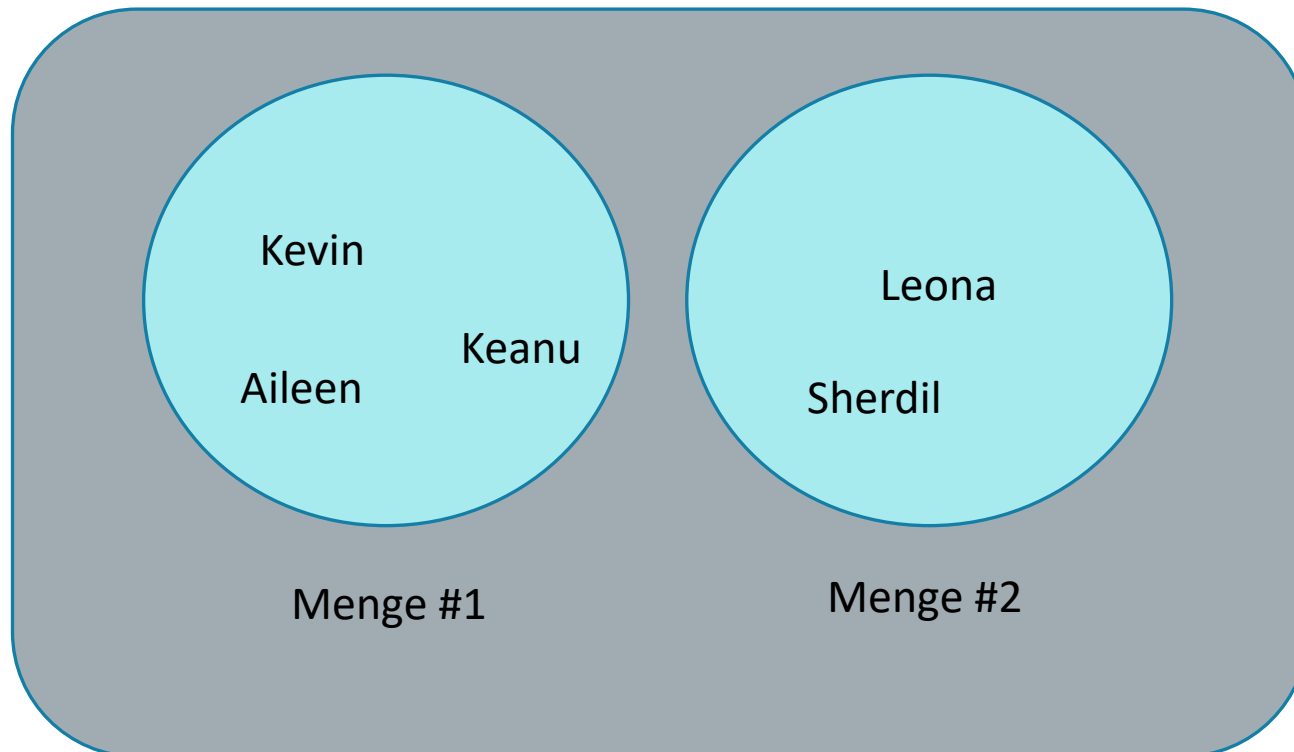
Menge #4

Diese beiden Mengen sind keine disjunkten Mengen

Disjunkte Mengen in der Informatik



In der Informatik ist der ADT disjunkter Mengen etwas, das mehrere disjunkte Mengen enthält



Der graue Block ist der ADT disjunkter Mengen, der eine beliebige Anzahl von Mengen enthält, die alle disjunkt sind.

Hinweis: Dies könnte sich anders anfühlen als die ADTs, die wir bisher kennengelernt haben. Die ADTs, die wir bisher gesehen haben (Tabellen, Listen, usw.) speichern Werte direkt.

Der ADT disjunkter Mengen speichert die ADTs Menge.

Beim ADT disjunkter Mengen geht es darum, dass Sie Werte in Mengen gruppieren und prüfen können, in welcher bestimmten Menge sich Ihre Werte befinden.

ADT Disjunkter Mengen – Methoden

Nur drei Methoden (und makeSet ist recht einfach!)

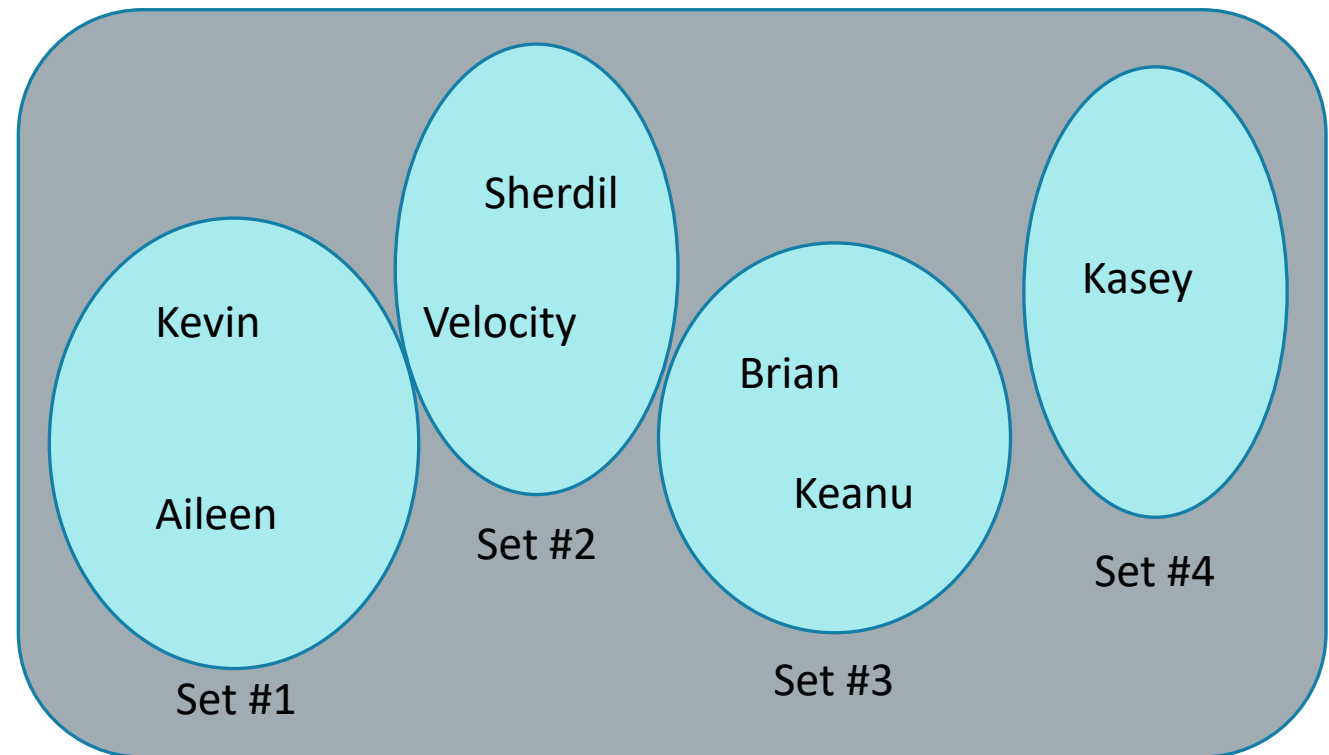
- find(value)
- union(valueA, valueB)
- makeSet(value)

ADT Disjunkter Mengen - find(value)

find(value) gibt eine ID zurück, die angibt, in welcher Menge sich `value` befindet. Die ID wird als Repräsentant bezeichnet (da es sich um einen Wert handelt, der die gesamte Menge repräsentiert).

Beispiele:

- `find(Brian)` → 3
- `find(Sherdil)` → 2
- `find(Velocity)` → 2
- `find(Kevin) == find(Aileen)`
 - True



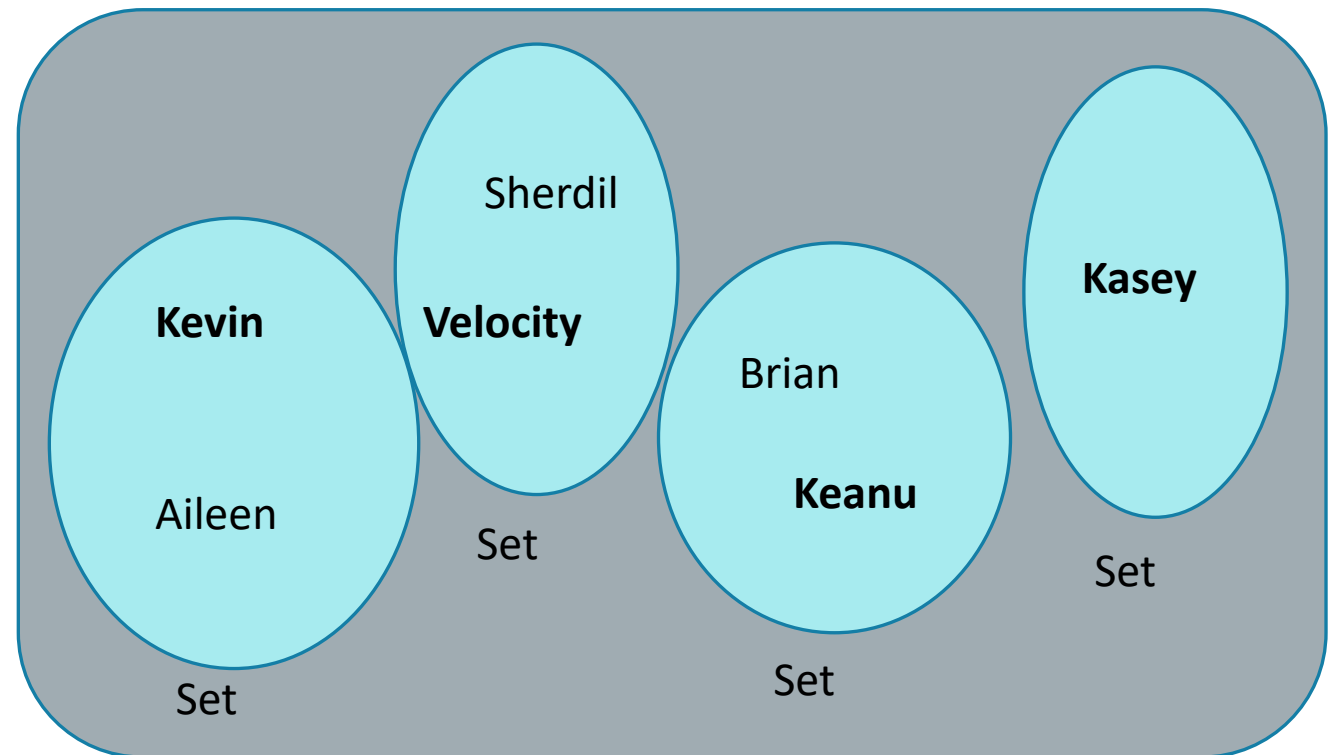
ADT Disjunkter Mengen - find(value)

Alternative

find(value) gibt den Repräsentanten der Menge zurück, in welcher Menge sich `value` befindet. Der Repräsentant ist ein bestimmtes Element der Menge, die diese repräsentiert.

Beispiele:

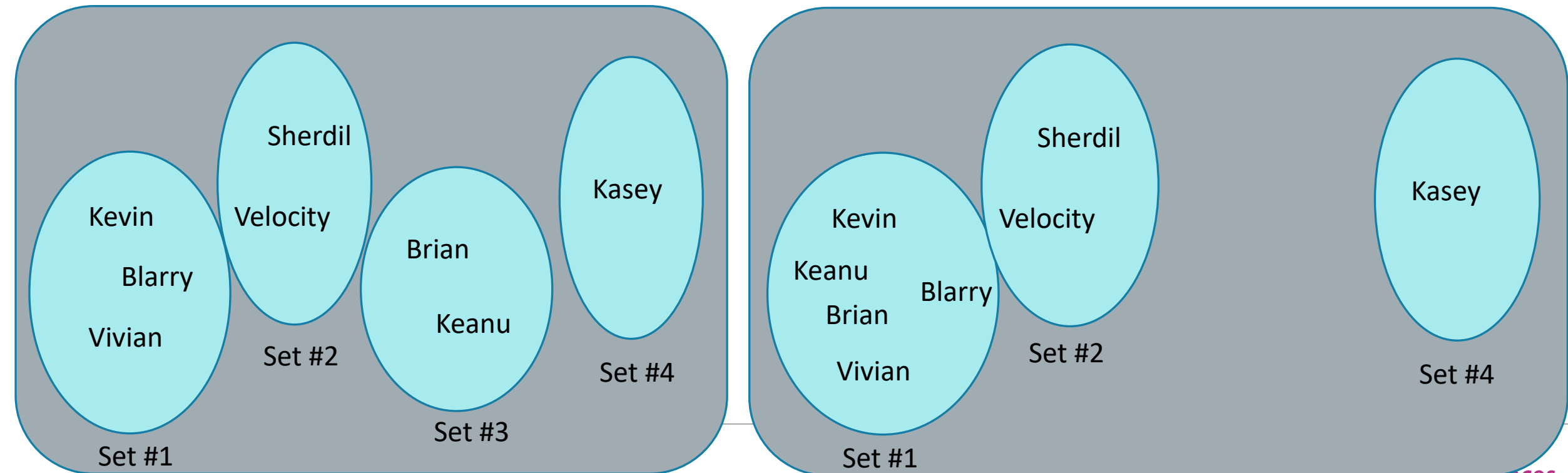
- `find(Brian)` → Keanu
- `find(Sherdil)` → Velocity
- `find(Velocity)` → Velocity
- `find(Kevin) == find(Aileen)`
 - True



ADT Disjunkter Mengen - `union(valueA, valueB)`

`union(valueA, valueB)` vereint die Menge, in der sich `valueA` befindet, mit der Menge, in der sich `valueB` befindet. Im Grunde werden die beiden Mengen zu einer Menge zusammengefasst.

Beispiel: `union(Blarry, Brian)`



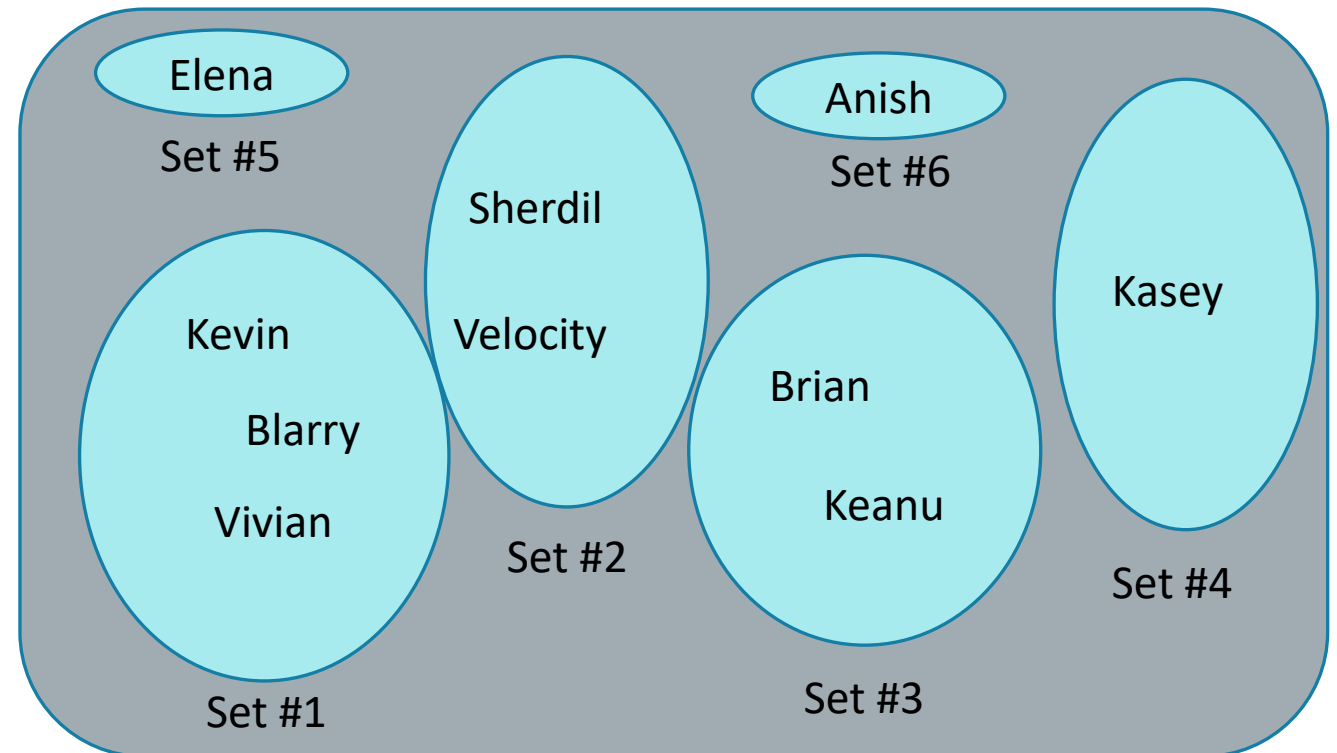
ADT Disjunkter Mengen - makeSet(value)

makeSet(value) macht eine neue Menge, die nur den übergebenen `value` in sich hat.

Beispiele:

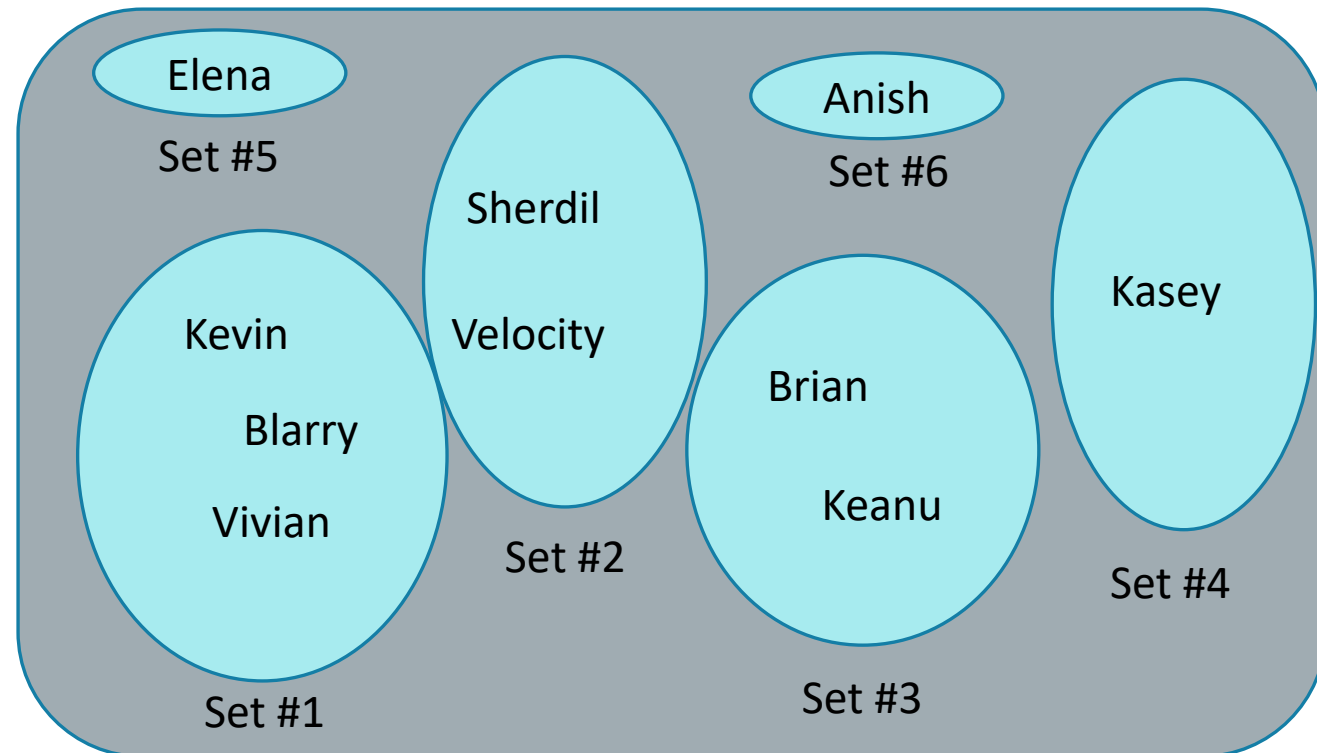
`makeSet(Elena)`

`makeSet(Anish)`



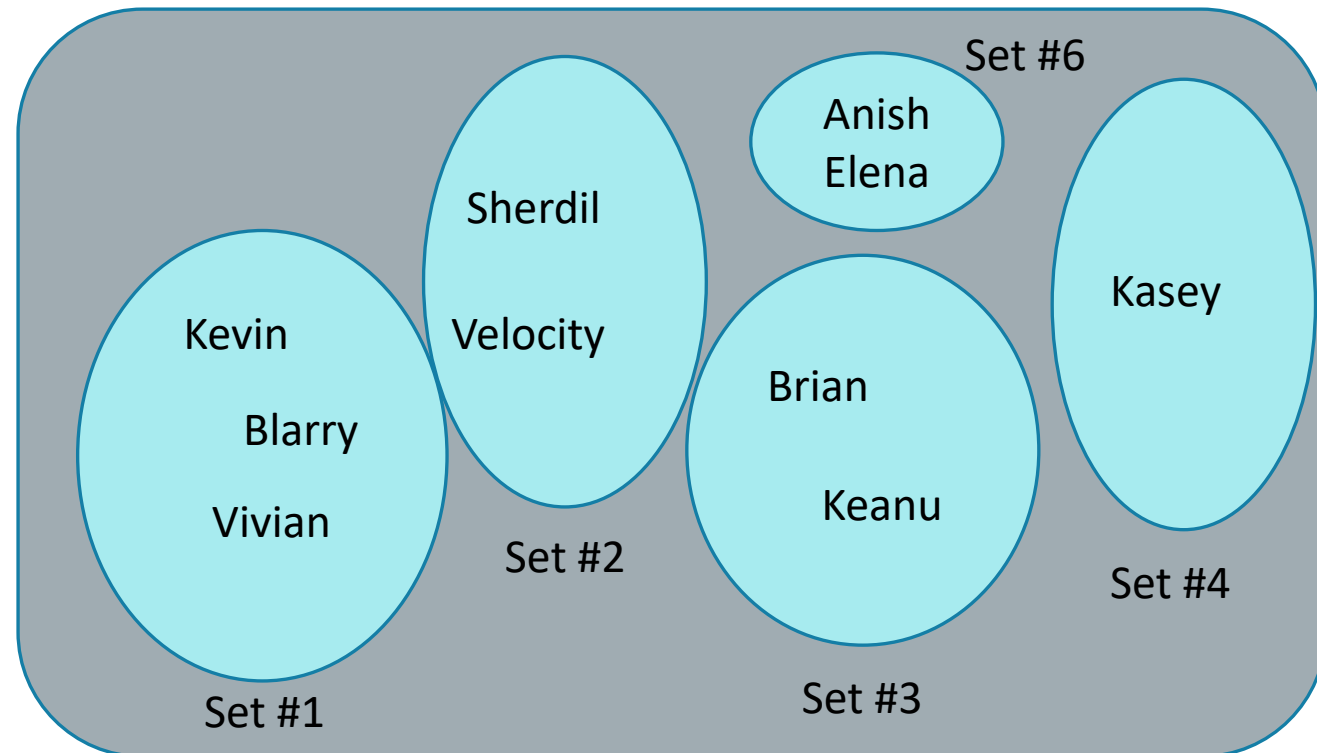
ADT Disjunkter Mengen – Nutzung im Praktikum 3

- Praktikumsgruppe = disjunkte Menge
- Alle Praktikumsgruppen des Semesters = Menge disjunkter Mengen
- Zusammenlegen von Praktikumsgruppen, die zu klein sind:
 - `union(Elena, Anish)`



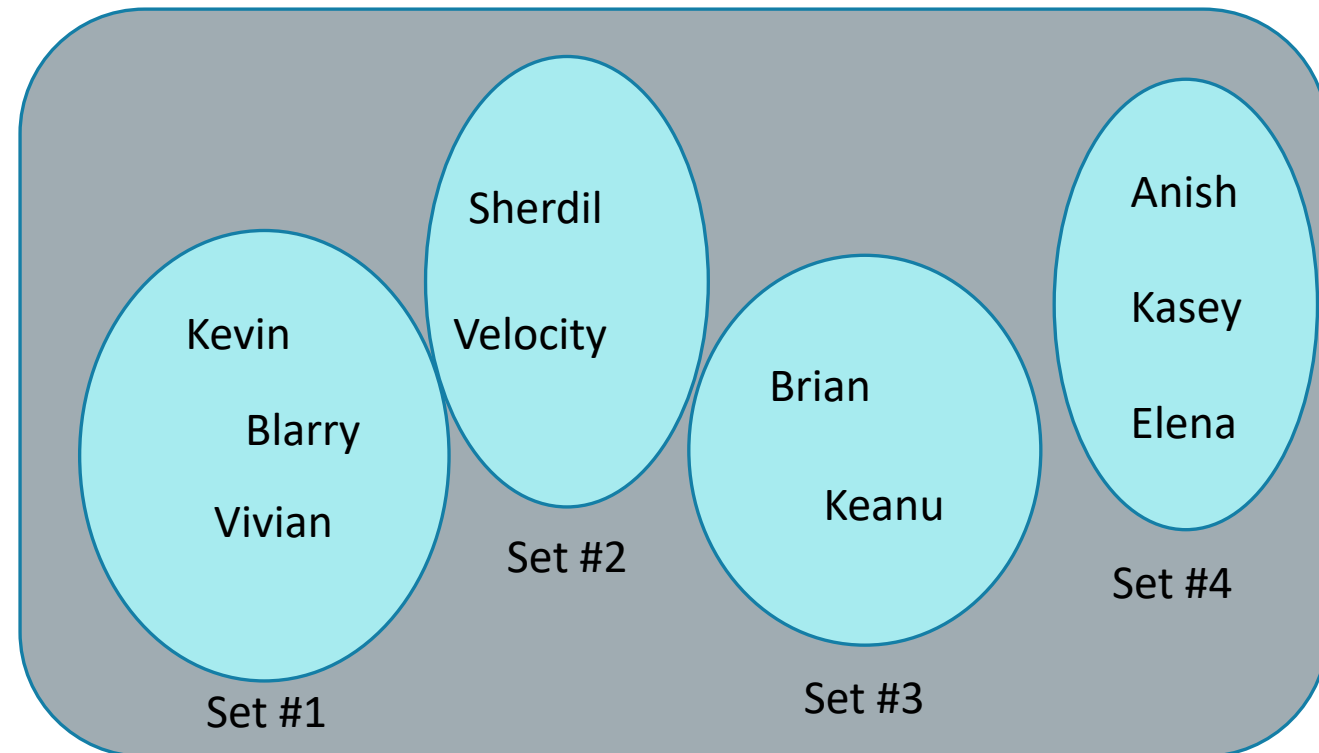
ADT Disjunkter Mengen – Nutzung im Praktikum 3

- Praktikumsgruppe = disjunkte Menge
- Alle Praktikumsgruppen des Semesters = Menge disjunkter Mengen
- Zusammenlegen von Praktikumsgruppen, die zu klein sind:
 - `union(Elena, Anish)`
 - `union(Anish, Kasey)`



ADT Disjunkter Mengen – Nutzung im Praktikum 3

- Praktikumsgruppe = disjunkte Menge
- Alle Praktikumsgruppen des Semesters = Menge disjunkter Mengen
- Zusammenlegen von Praktikumsgruppen, die zu klein sind:
 - `union(Elena, Anish)`
 - `union(Anish, Kasey)`



ADT Menge und ADT disjunkter Mengen

ADT Menge

state

Set of elements

- Elements must be unique!
- No required order

Count of Elements

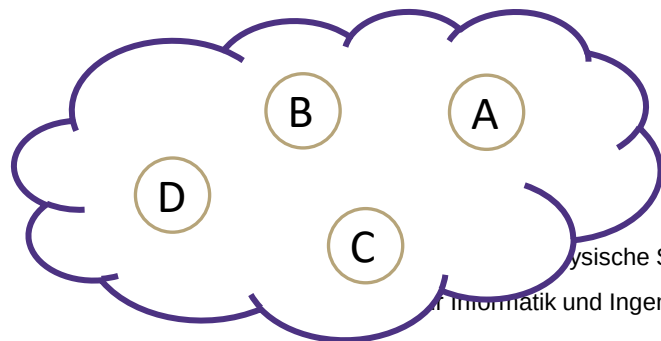
behavior

create(value) - creates a new set with a single member, value

add(value) - adds value into set if it is unique, otherwise add is ignored

remove(value) – removes value from set

size() – returns current number of elements in set



ADT Disjunkter Mengen

state

Set of Sets

- **Disjoint:** Elements must be unique across sets
- No required order
- Each set has id/representative

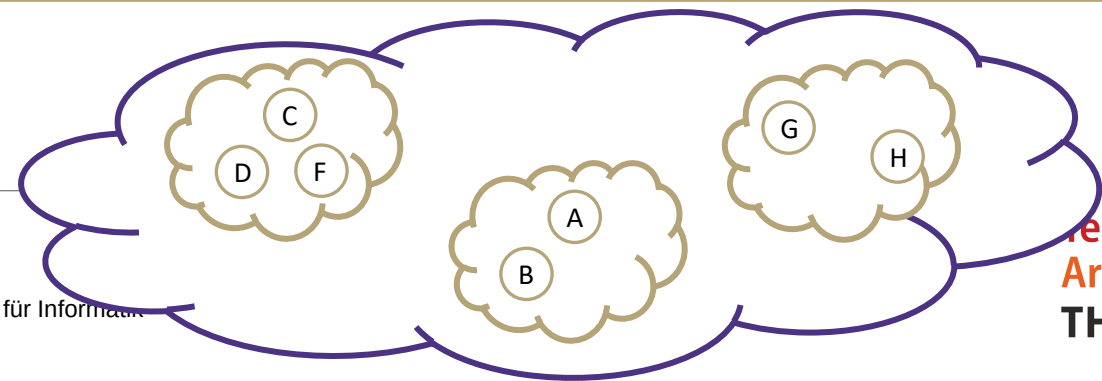
Count of Sets

behavior

makeSet(value) – creates a new set within the disjoint set where the only member is value. Picks id/representative for set

find(value) – looks up the set containing element value, returns id/representative of that set

union(x, y) – looks up set containing x and set containing y, combines two sets into one. All of the values of one set are added to the other, and the now empty set goes away. Picks new id/representative for resulting set.



Übersicht für heute

- Minimale Spannbäume
 - Algorithmus von Prim
 - Algorithmus von Kruskal
- ADT
 - Menge, Disjunkte Mengen
- **Implementierung des ADT Disjunkter Mengen**
 - **Quick-Find**
 - **Wald disjunkter Mengen**
 - **Quick-Union, Weighted Quick-Union**
 - Pfadverkürzung
- Laufzeit des Algorithmus von Kruskal mit optimaler Implementierung des ADT Disjunkter Mengen

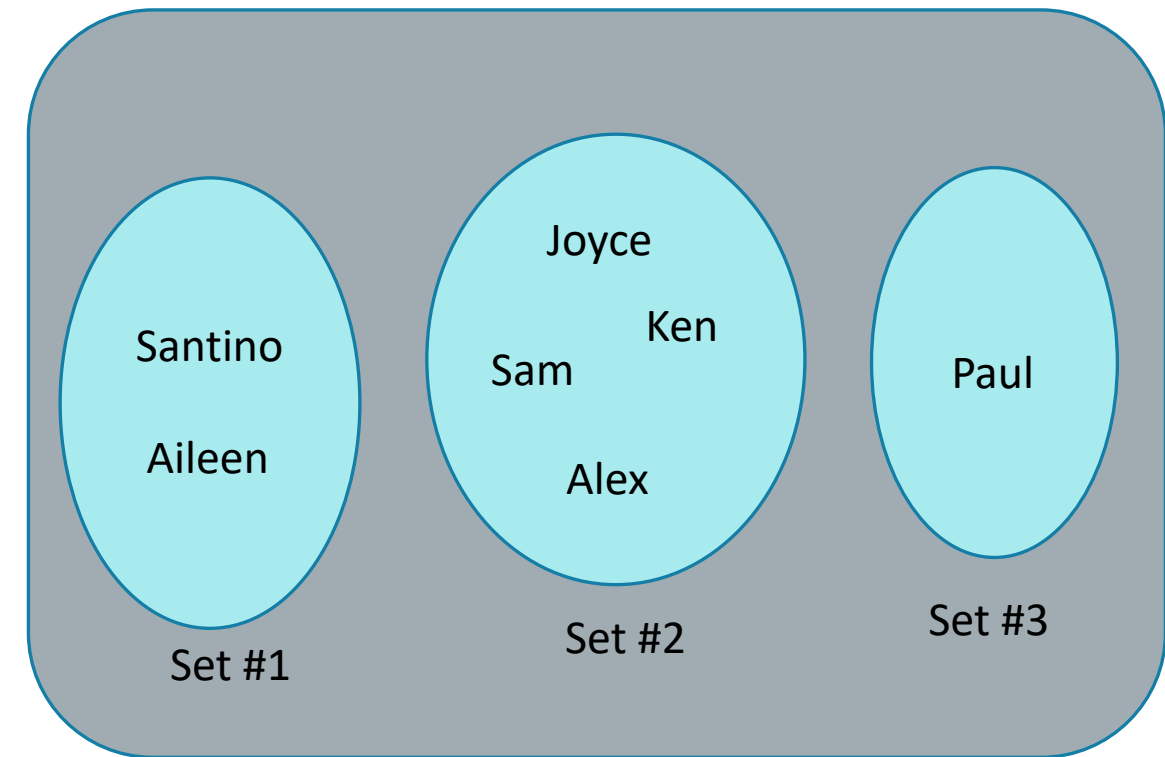
Implementierung des ADT Disjunkter Mengen: Baseline

Baseline

Element mit ID als Schlüssel-Wert-Paar in Tabelle (implementiert als Array) speichern.
Schlüssel: Element; Wert: ID der Menge.

Schlüssel	Wert
Aileen	1
Joyce	2
Santino	1
Sam	2
Ken	2
Alex	2
Paul	3

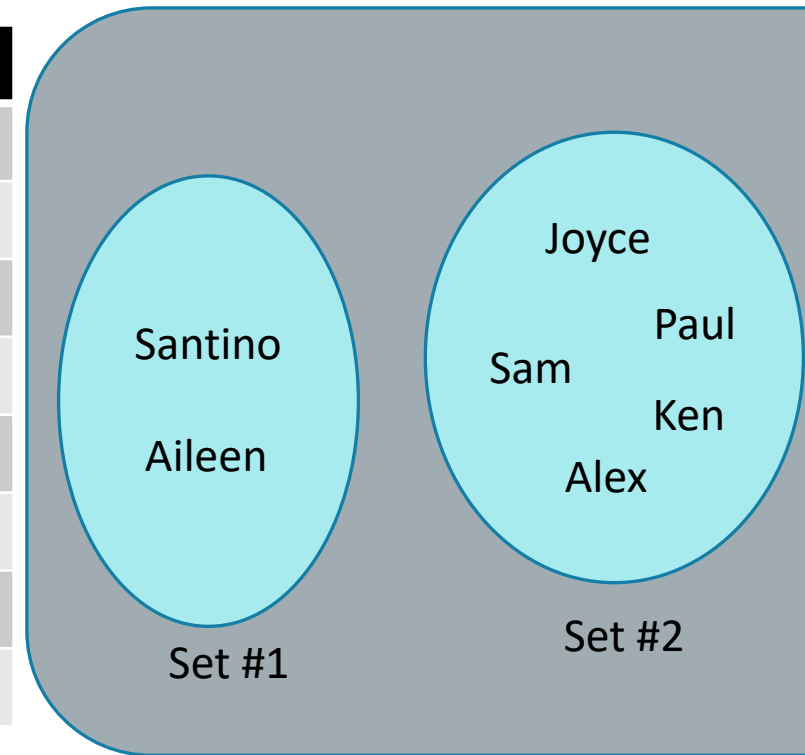
ADT Disjunkter Mengen



Implementierung des ADT Disjunkter Mengen: Baseline

- Tabelle als Array mit Schlüssel-Wert-Paaren in zufälliger Reihenfolge
- find(„Sam“)
 - WC: Alle Indizes der Tabelle durchlaufen
- union(„Ken“, „Paul“)
 - find(„Ken“) → 2
 - find(„Paul“) → 3
 - Alle Indizes der Tabelle durchgehen und 3 durch 2 ersetzen.

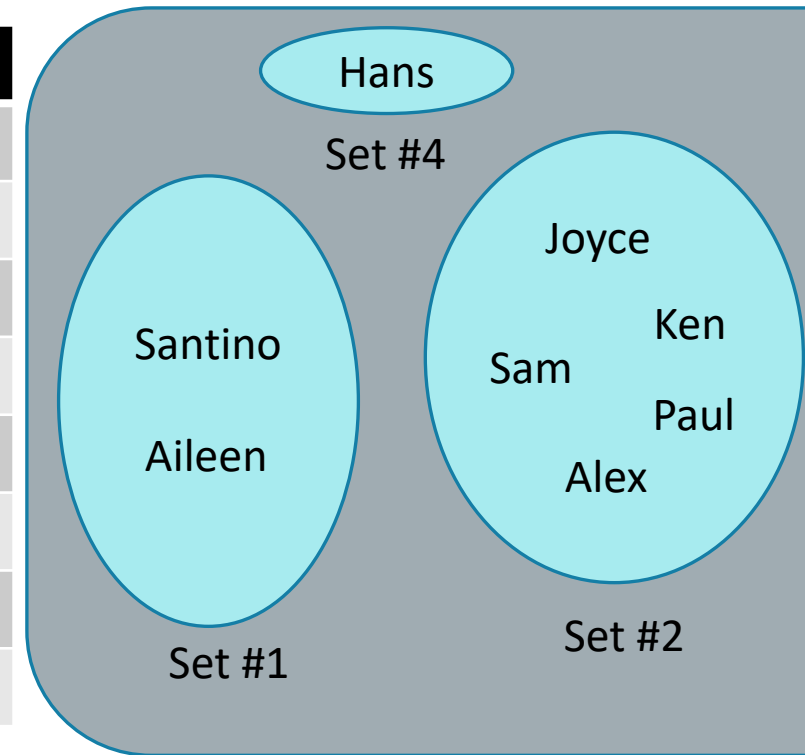
Schlüssel	Wert
Aileen	1
Joyce	2
Santino	1
Sam	2
Ken	2
Alex	2
Paul	2



Implementierung des ADT Disjunkter Mengen: Baseline

- Tabelle als Array mit Schlüssel-Wert-Paaren in zufälliger Reihenfolge
- find(„Sam“)
 - WC: Alle Indizes der Tabelle durchlaufen
- union(„Ken“, „Paul“)
 - find(„Ken“) → 2
 - find(„Paul“) → 3
 - Alle Indizes der Tabelle durchgehen und 3 durch 2 ersetzen.
- makeSet(„Hans“)
 - Fügt („Hans“, 4) in leere Zeile der Tabelle ein.

Schlüssel	Wert
Aileen	1
Joyce	2
Santino	1
Sam	2
Ken	2
Alex	2
Paul	2
Hans	4



Implementierung des ADT Disjunkter Mengen: Baseline

- Tabelle als Array mit Schlüssel-Wert-Paaren in zufälliger Reihenfolge
- find(„Sam“)
 - WC: Alle Indizes der Tabelle durchlaufen
- union(„Ken“, „Paul“)
 - find(„Ken“) $\rightarrow$ 2
 - find(„Paul“) $\rightarrow$ 3
 - Alle Indizes der Tabelle durchgehen und 3 durch 2 ersetzen.
- makeSet(„Hans“)
 - Fügt („Hans“, 4) in leere Zeile der Tabelle ein.

Schlüssel	Wert
Aileen	1
Joyce	2
Santino	1
Sam	2
Ken	2
Alex	2
Paul	2
Hans	4

Annahme: Wir müssen nicht prüfen ob value bereits als Schlüssel existiert

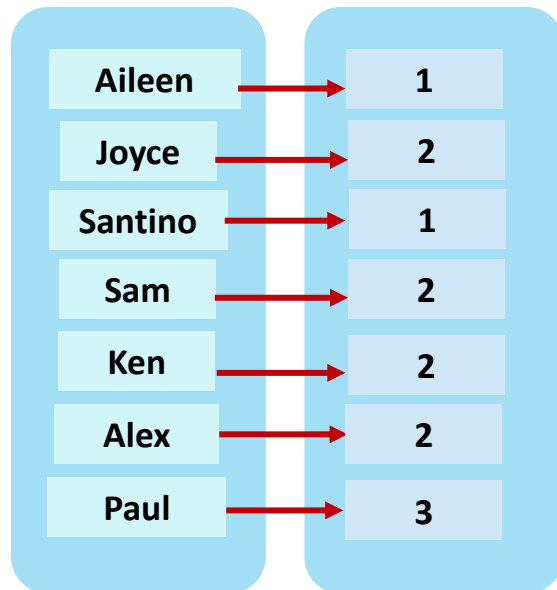
	Baseline
makeSet(value)	$\Theta(1)$
find(value)	$\Theta(n)$
union(x, y)	$\Theta(n)$

Implementierung des ADT Disjunkter Mengen: Quick-Find

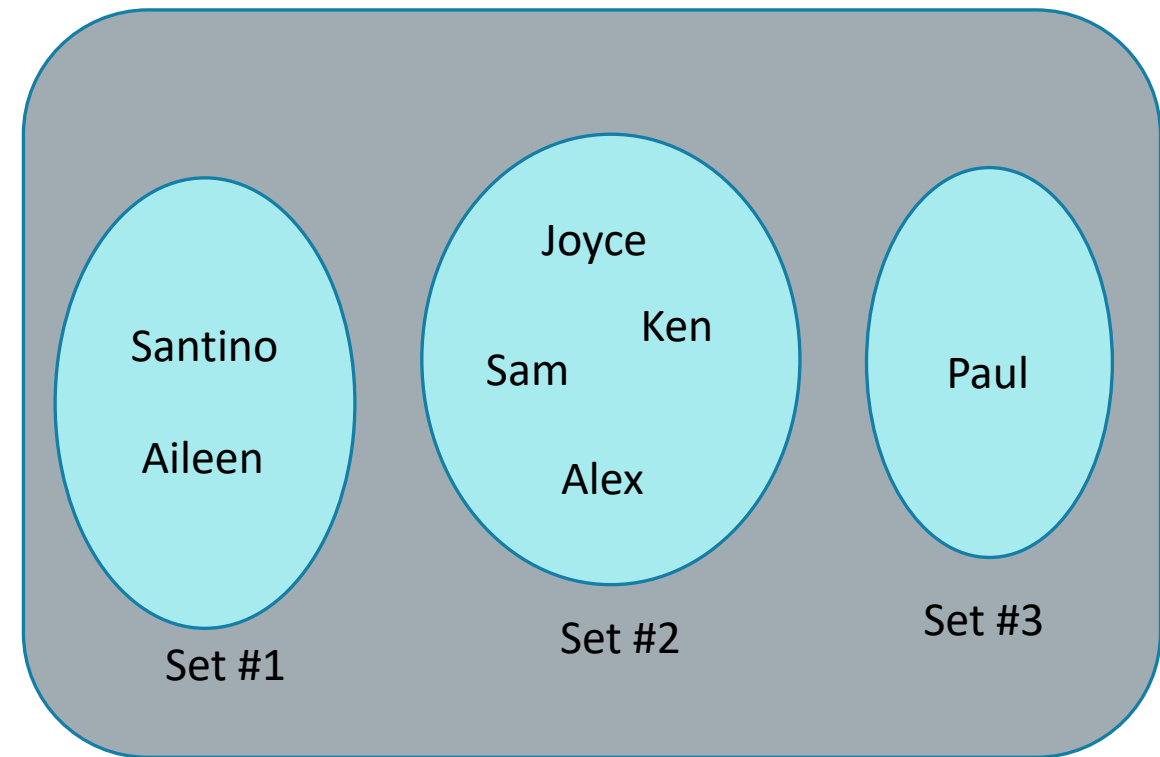
Quick-Find

Element mit ID als Schlüssel-Wert-Paar in **Hashtabelle** speichern.

Schlüssel: Element; Wert: ID der Menge.



ADT Disjunkter Mengen



Implementierung des ADT Disjunkter Mengen: Quick-Find

find(„Aileen“)

- Hashwert für „Aileen“ berechnen, an Position in Hashtabelle springen und repräsentative ID der Menge auslesen $\rightarrow 1$
- Laufzeit: $\Theta(1+\lambda)$

union(„Aileen“, „Paul“)

- find(„Aileen“) $\rightarrow 1$; in $\Theta(1+\lambda)$
- find(„Paul“) $\rightarrow 3$; in $\Theta(1+\lambda)$
- Vereinige Mengen 1 und 3, d.h.:
 - Gehe alle Schlüssel-Wert-Paare in Hashtabelle durch und ändere ID mit Wert 3 zu Wert 1. $\rightarrow \Theta(n)$
- Laufzeit insgesamt: $\Theta(n)$

	Baseline	Quick-Find
makeSet(value)	$\Theta(1)$	$\Theta(1+\lambda)$
find(value)	$\Theta(n)$	$\Theta(1+\lambda)$
union(x, y)	$\Theta(n)$	$\Theta(n)$



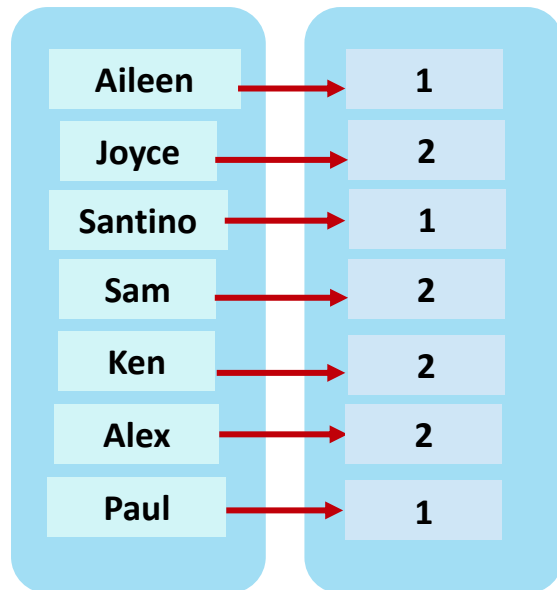
Implementierung des ADT Disjunkter Mengen: Quick-Find

Situation nach Aufruf: union(„Aileen“, „Paul“)

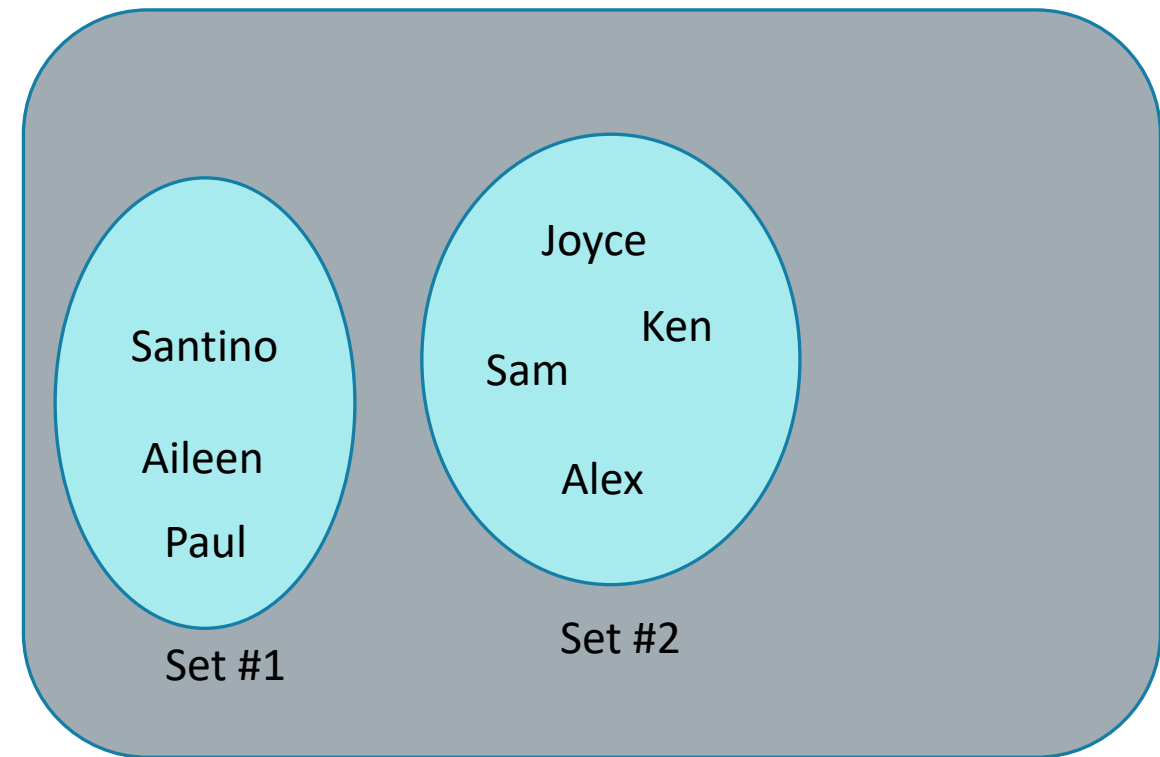
Quick-Find

Element mit ID als Schlüssel-Wert-Paar in Hashtabelle speichern.

Schlüssel: Element; Wert: ID der Menge.

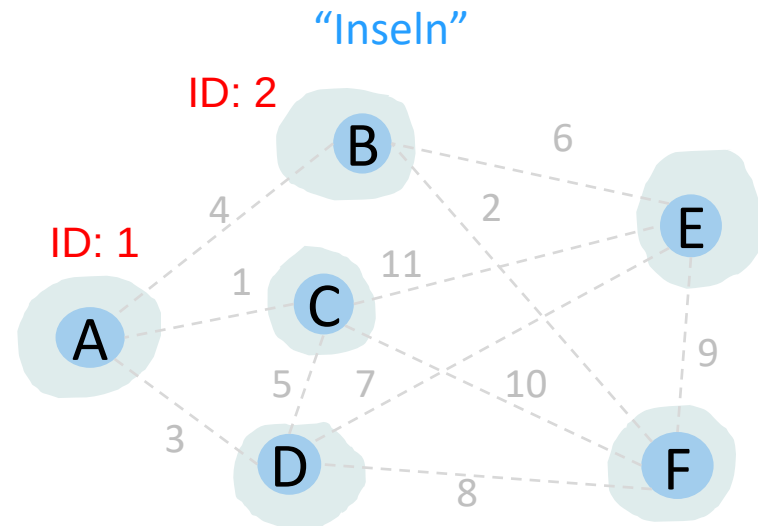


ADT Disjunkter Mengen



Kruskals Algorithmus - Übung

Zeichnen Sie die Hashtabelle nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung Quick-Find für folgenden Graphen. Nutzen Sie die Hashfunktion: $h \rightarrow \text{ord}(\text{Knoten}) - 64$.

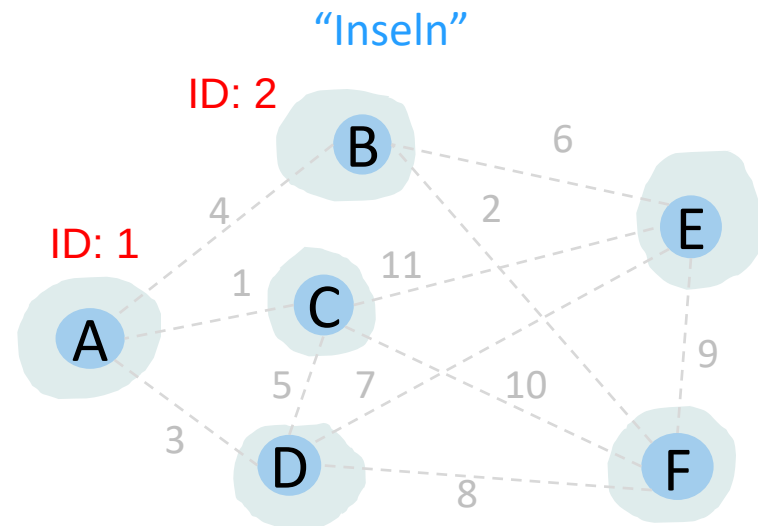


makeSet(A) $\rightarrow h(A) = \text{ord}(A) - 64 = 1 \rightarrow$ key-value-pair (A, 1) an Index 1
 makeSet(B) $\rightarrow h(B) = \text{ord}(B) - 64 = 2 \rightarrow$ key-value-pair (B, 2) an Index 2
 makeSet(C) $\rightarrow h(C) = \text{ord}(C) - 64 = 3 \rightarrow$ key-value-pair (C, 3) an Index 3

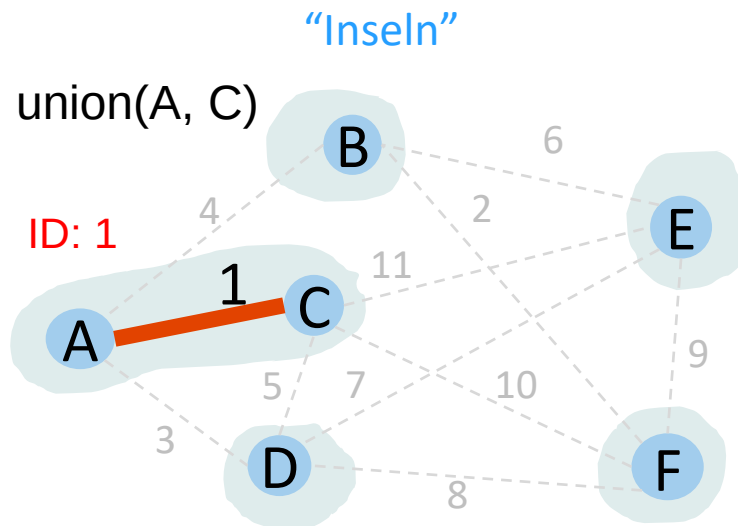
0	1	2	3	4	5

Kruskals Algorithmus - Übung

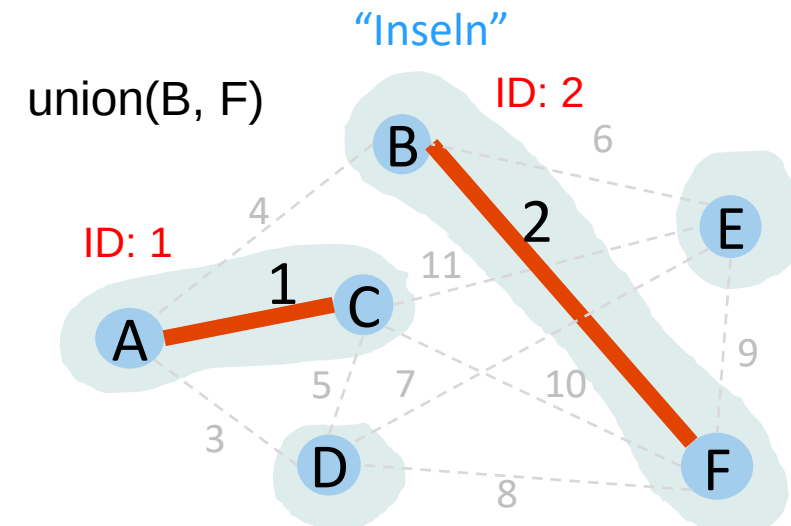
Zeichnen Sie die Hashtabelle nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung Quick-Find für folgenden Graphen. Nutzen Sie die Hashfunktion: $h \rightarrow \text{ord}(\text{Knoten}) - 64$.



0	1	2	3	4	5
(F, 6)	(A, 1)	(B, 2)	(C, 3)	(D, 4)	(E, 5)



0	1	2	3	4	5
(F, 6)	(A, 1)	(B, 2)	(C, 1)	(D, 4)	(E, 5)

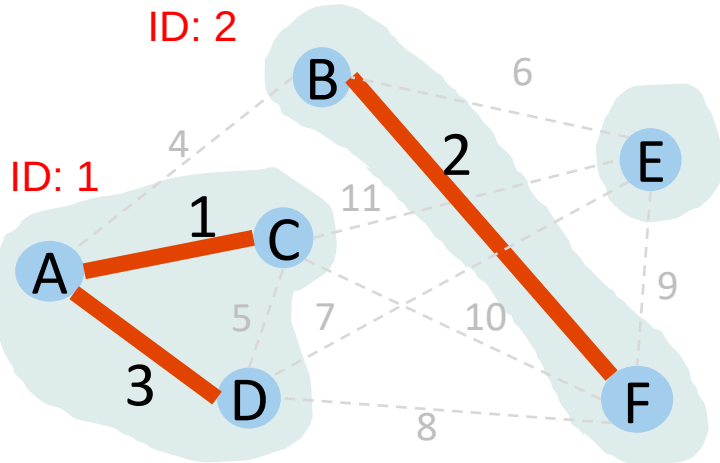


0	1	2	3	4	5
(F, 2)	(A, 1)	(B, 2)	(C, 1)	(D, 4)	(E, 5)

Kruskals Algorithmus - Übung

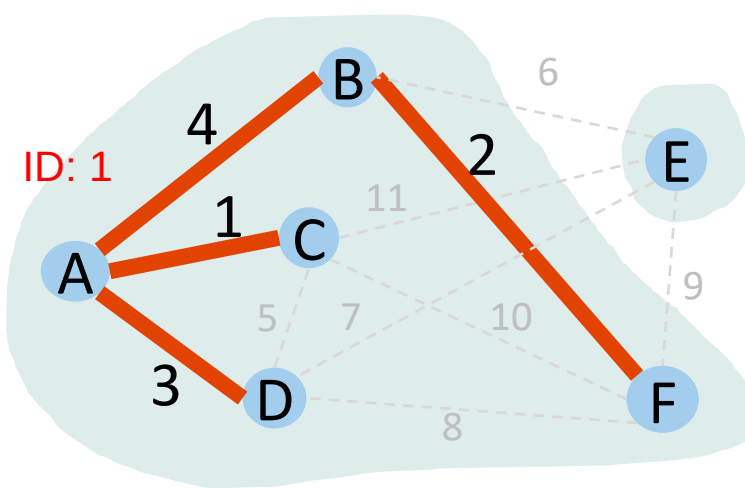
Zeichnen Sie die Hashtabelle nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung Quick-Find für folgenden Graphen. Nutzen Sie die Hashfunktion: $h \rightarrow \text{ord}(\text{Knoten}) - 64$.

“Inseln”



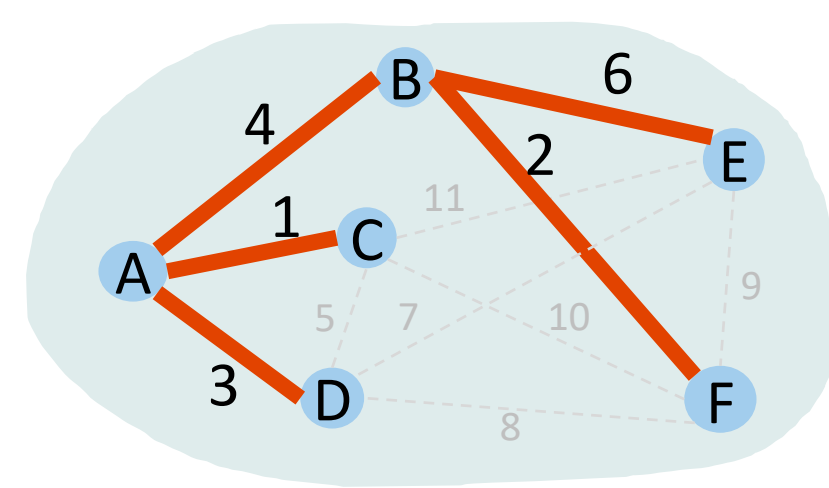
0	1	2	3	4	5
(F, 2)	(A, 1)	(B, 2)	(C, 1)	(D, 1)	(E, 5)

“Inseln”



0	1	2	3	4	5
(F, 1)	(A, 1)	(B, 1)	(C, 1)	(D, 1)	(E, 5)

“Inseln”



0	1	2	3	4	5
(F, 1)	(A, 1)	(B, 1)	(C, 1)	(D, 1)	(E, 1)

Implementierung des ADT Disjunkter Mengen: Quick-Union

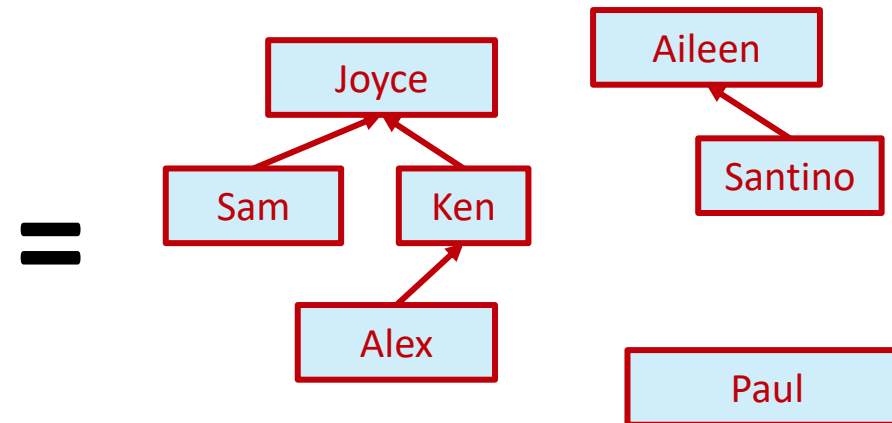
Wald disjunkter Mengen

Grundlegende Idee:

- Quick-Union speichert eine Menge als Baum. Die Wurzel des Baums ist der Repräsentant der Menge.
- Jede Menge wird baumähnlich, aber etwas anders, ein sogenannter Up-Tree: speichert Zeiger von Kindern zu Eltern!



ADT Disjunkter Mengen



Implementierung mit Quick-Union

Implementierung des ADT Disjunkter Mengen: Quick-Union

Wald disjunkter Mengen

ADT Disjunkter Mengen

state

Set of Sets

- **Disjoint:** Elements must be unique across sets
- No required order
- Each set has representative

Count of Sets

behavior

makeSet(value) – creates a new set within the disjoint set where the only member is value. Picks representative for set

find(value) – looks up the set containing element value, returns representative of that set

union(x, y) – looks up set containing x and set containing y, combines two sets into one. Picks new representative for resulting set

ForrestDisjointSet<E>

state

```
Map<E, SetNode<E>>
nodeInventory
```

behavior

makeSet(value) – create a new setNode and save it in nodeInventory

find(value) – locates node with value and moves up tree to find root

union(x, y) – append tree with y as a child of tree with x

Aileen

Santino

SetNode<E>

state

E value

SetNode<E> parent

behavior

SetNode(value)

setParent(root) – sets parent to root

getParent() – return parent

getValue() – return value

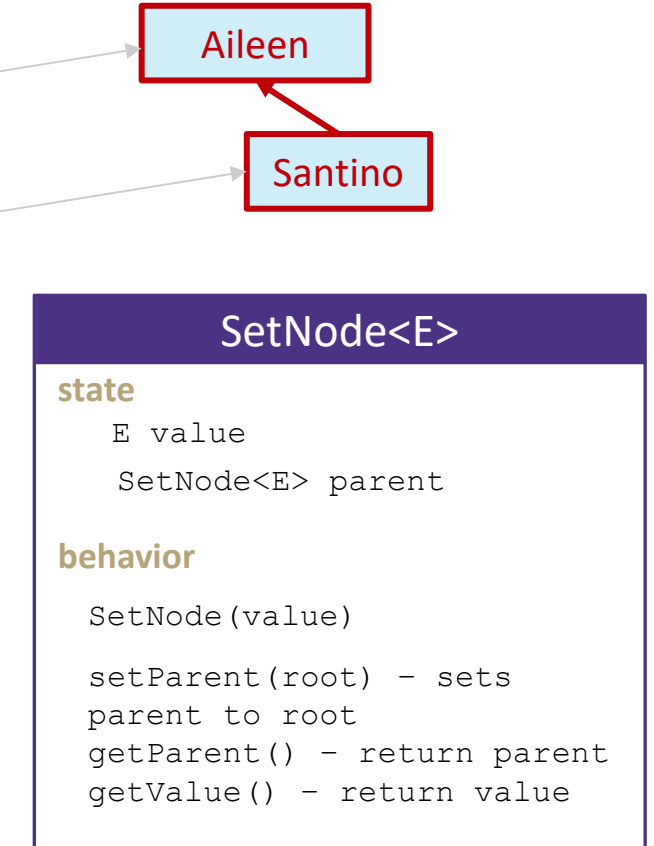
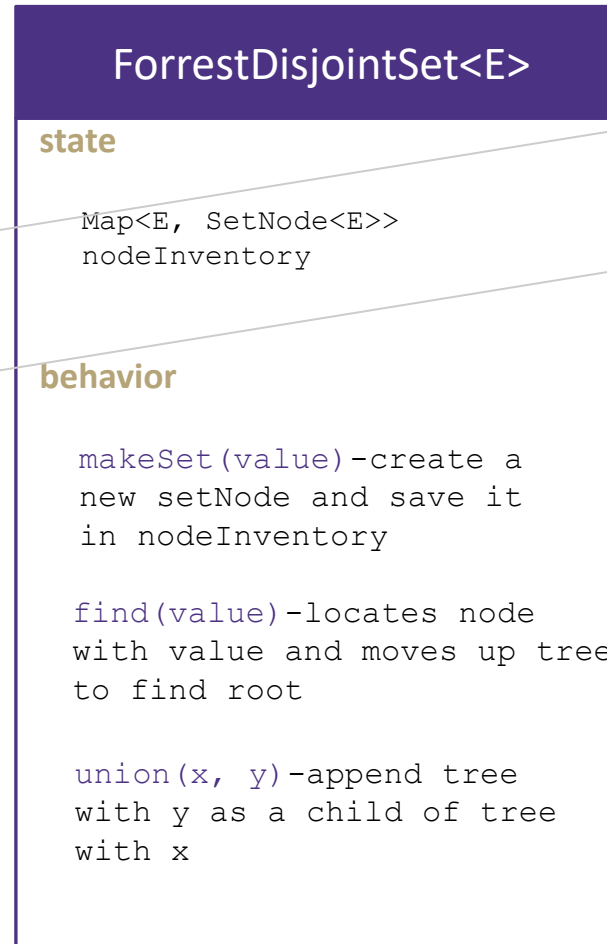
Implementierung des ADT Disjunkter Mengen: Quick-Union

Wald disjunkter Mengen

nodeInventory:

Schlüssel:

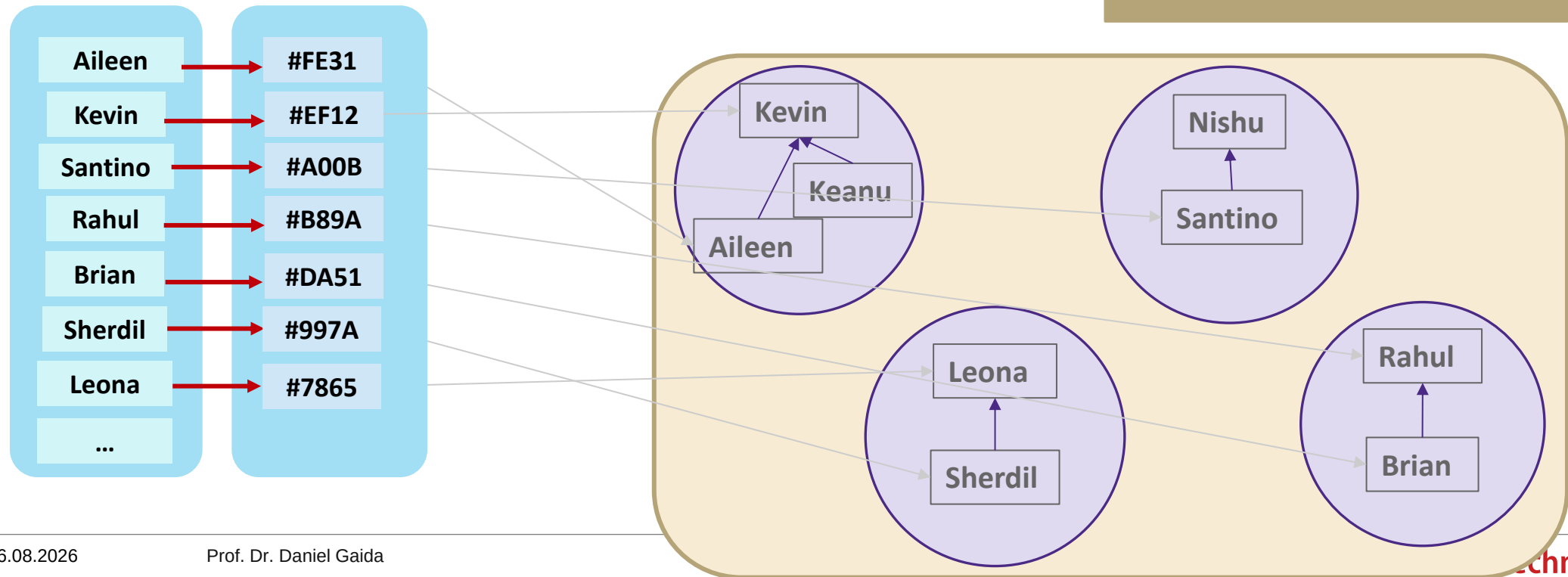
Wert:



Wald disjunkter Mengen: Implementierung

```
Map<E, SetNode<E>> nodeInventory
```

ADT Disjunkter Mengen wird repräsentiert durch:
ForrestDisjointSet<E> mit
- Hashtabelle mit Knotenreferenzen (nodeInventory)



Wald disjunkter Mengen: Implementierung

ForrestDisjointSet<E>

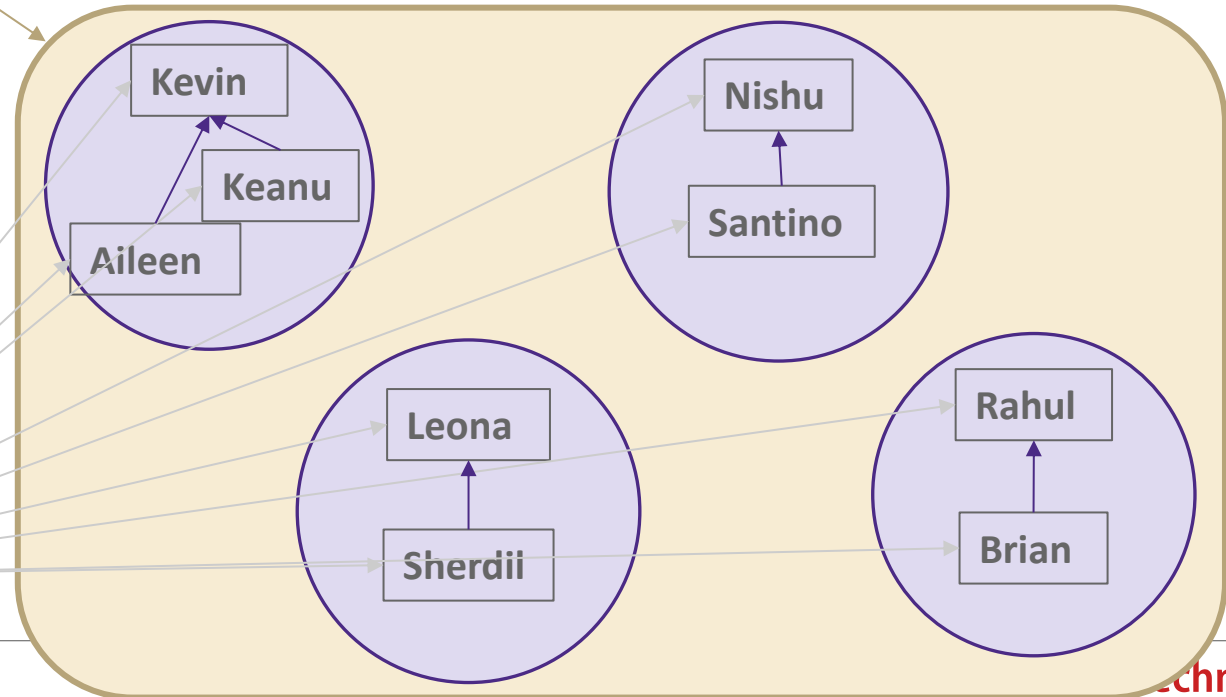
1

Jeder SetNode<E> kann eine unbegrenzte Anzahl von Kindern haben, aber nur einen Elternteil. SetNode speichert Elter anstelle von Kindern.

myNode = nodeInventory.get(„Sherdil“)
myNode.getParent()
→ SetNode mit Element „Leona“

Map<E, SetNode<E>>
nodeInventory

9



Wald disjunkter Mengen: Implementierung

ForrestDisjointSet<E>

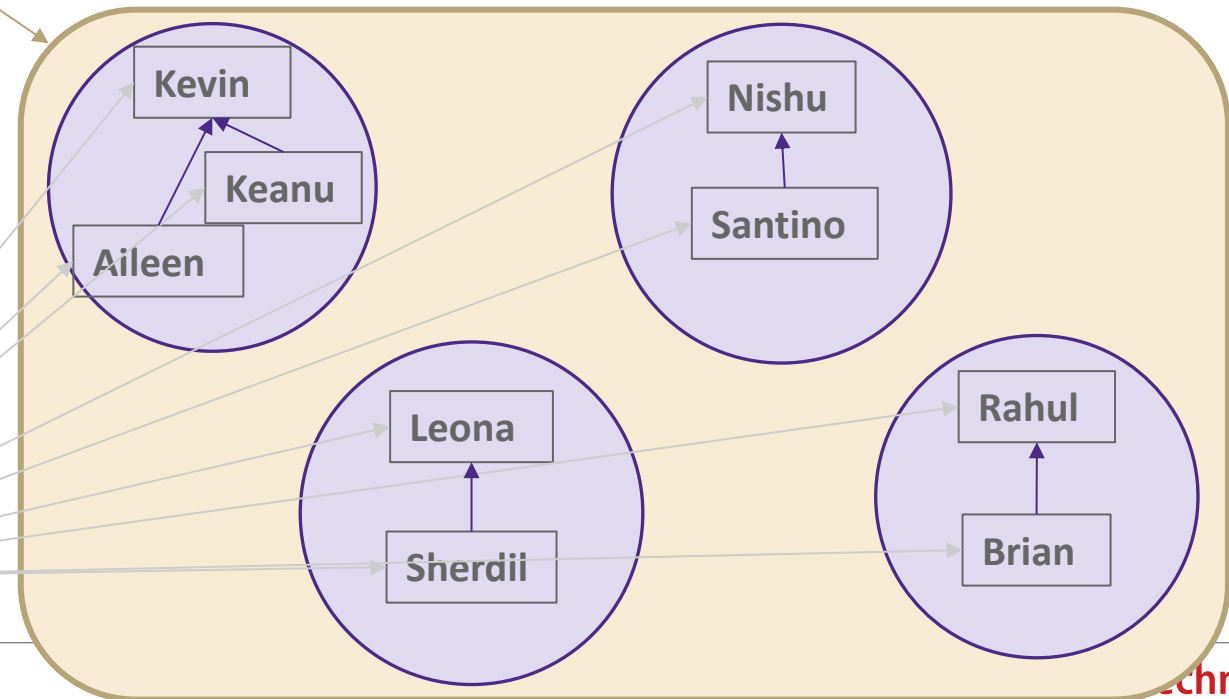
1

Implementierung des ADTs disjunkter Mengen ist eine Kombination aus:

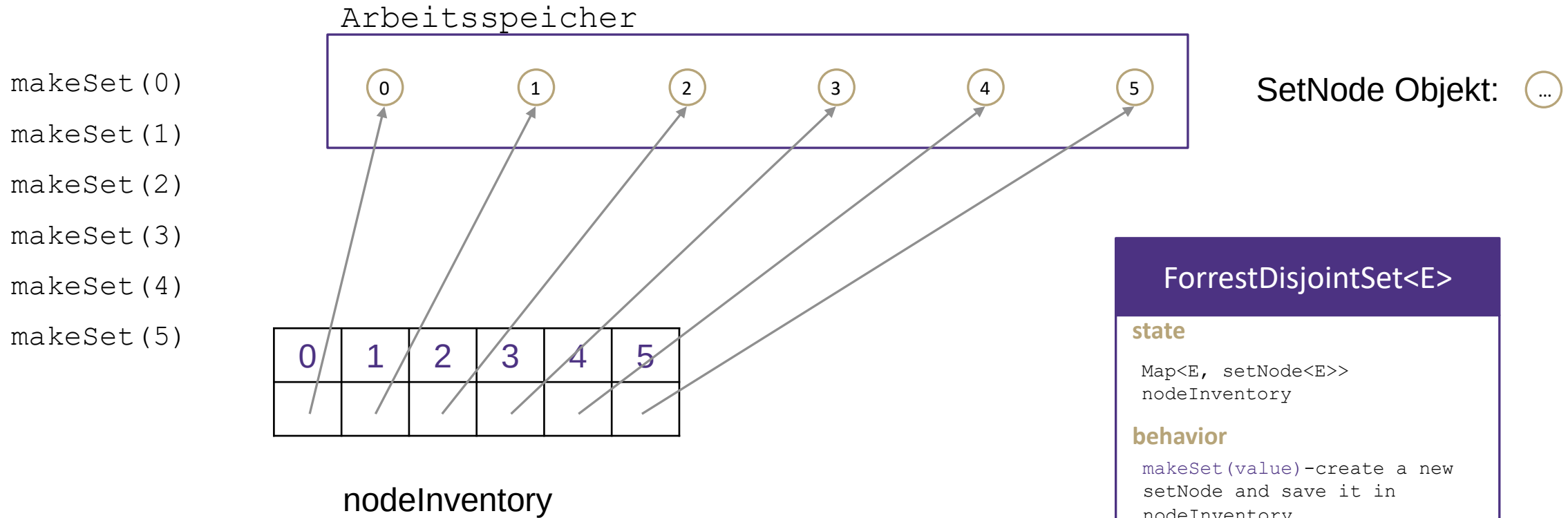
- Datenstruktur: Hashtabelle
- Datenstruktur: setNode, das sich zu Up-Tree verketteten lässt

Map<E, SetNode<E>>
nodeInventory

9



Wald disjunkter Mengen - Implementierung von makeSet(x)



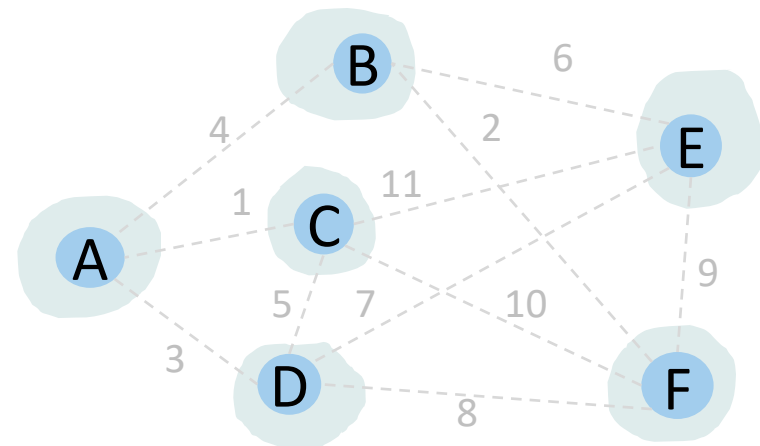
Worst case Laufzeit?

■ $O(1+\lambda)$

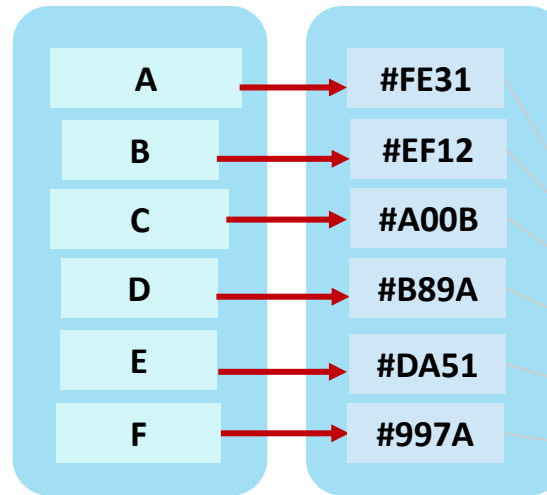
Kruskals Algorithmus - Übung

Zeichnen Sie den **Wald disjunkter Bäume** nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung **Quick-Union** für folgenden Graphen. → hier nur Iteration 0 (makeSet(...))

“Inseln”



NodeInventory:



makeSet (A)
makeSet (B)
makeSet (C)
...

setNodes:



Quick-Union: Implementierung von find()

```
find("Alex") :  
  jump to "Alex" node  
  travel upward until root  
  return root node "Joyce"
```

```
find(Santino) -> Knoten Aileen  
find(Ken) -> Knoten Joyce  
find(Santino) != find(Ken)  
find(Santino) == find(Aileen)
```

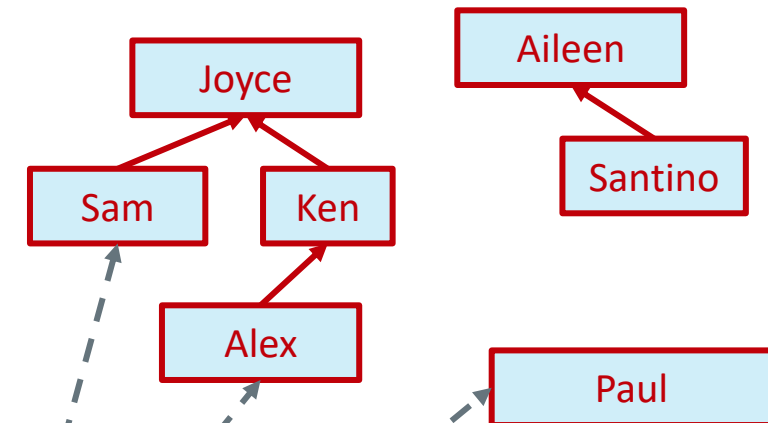
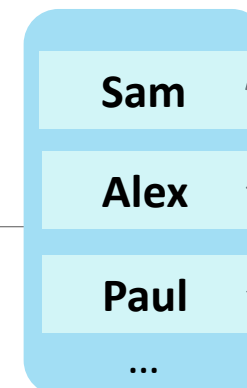
Schlüsselidee:

- Man kann von jedem Knoten aus nach oben reisen, bis parent == NULL, dann hat man Wurzelknoten gefunden

Wie können wir schnell zu einem Knoten springen?

- Durch nodeInventory (wird in zukünftigen Folien weggelassen)

nodeInventory:



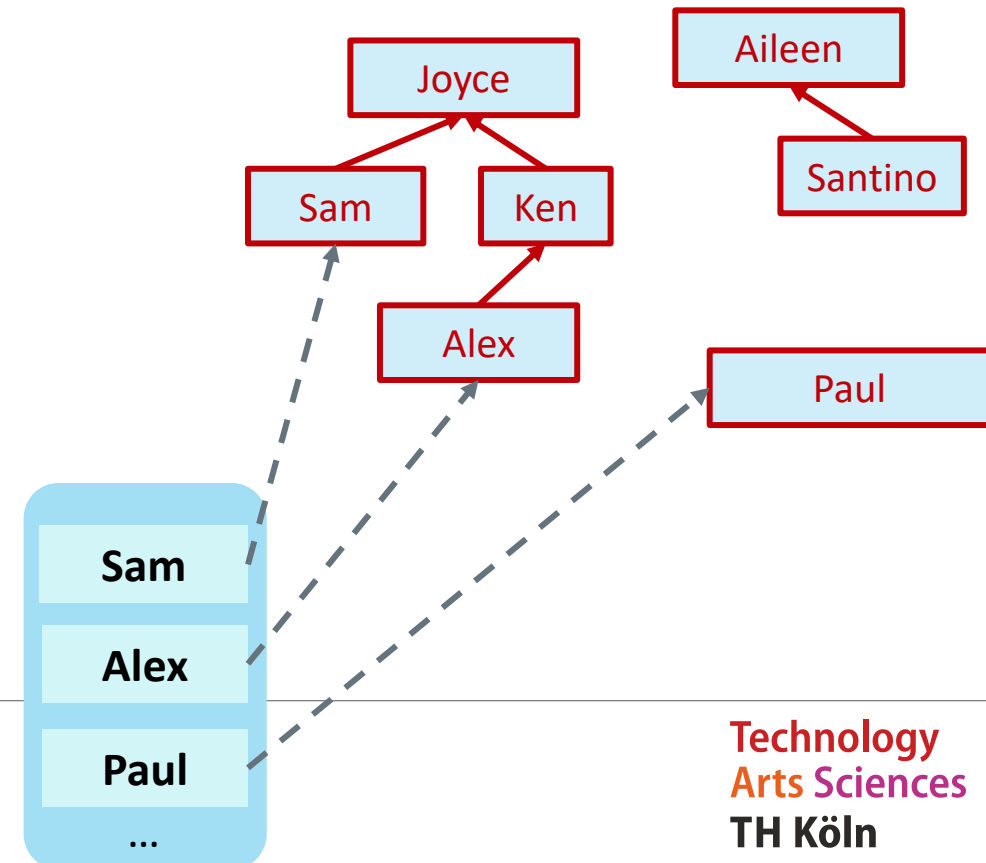
Quick-Union: Implementierung von find()

```
find("Alex") :  
  jump to "Alex" node  
  travel upward until root  
  return root node "Joyce"
```

Laufzeit:

- Zum Knoten springen: $O(1+\lambda)$
- Zum Wurzelknoten gehen:
 - Hängt von Höhe des Baumes ab
 - Worst Case: $O(n)$, falls Baum ein degenerierter Baum ist

nodeInventory:



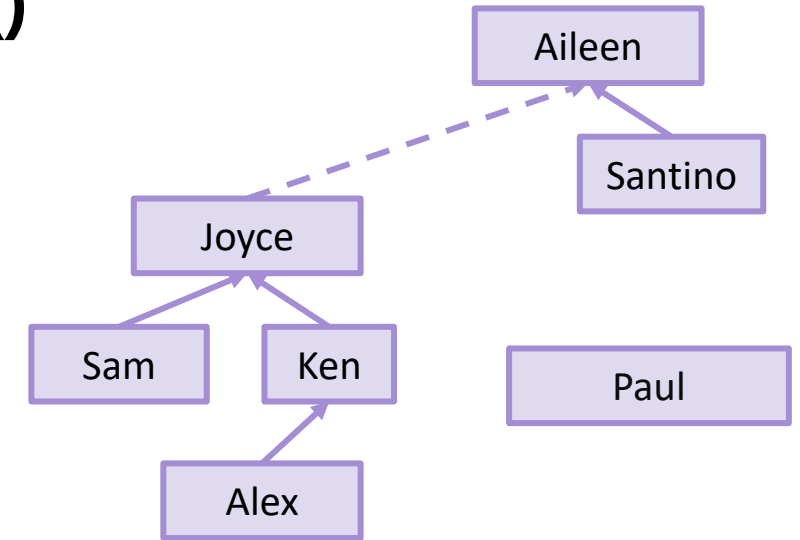
Quick-Union: Implementierung von union()

Schlüsselidee: Einfaches Umpointern von Zeigern zur Vereinigung ganzer Bäume!

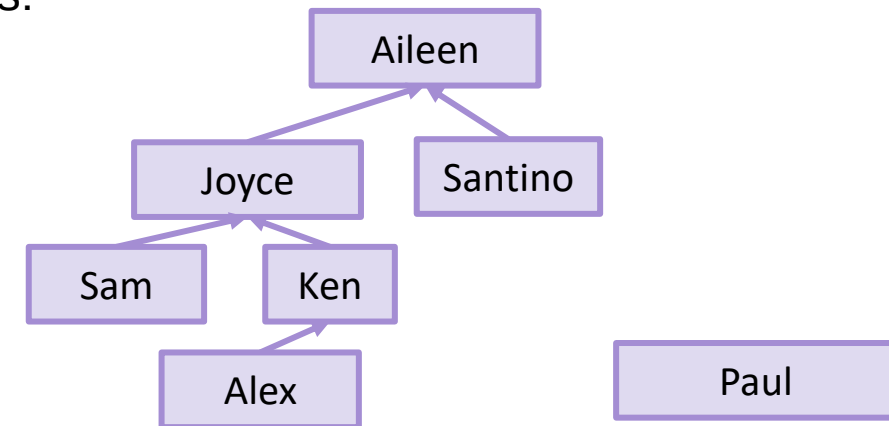
```
union("Ken", "Santino"):
    rootK = find("Ken") //node Joyce
    rootS = find("Santino") //node Aileen
    set rootK to point to rootS
```

Laufzeit:

- $O(1)$ für das Umpointern
- 2mal Laufzeit von find(), also WC-Fall: $O(n)$

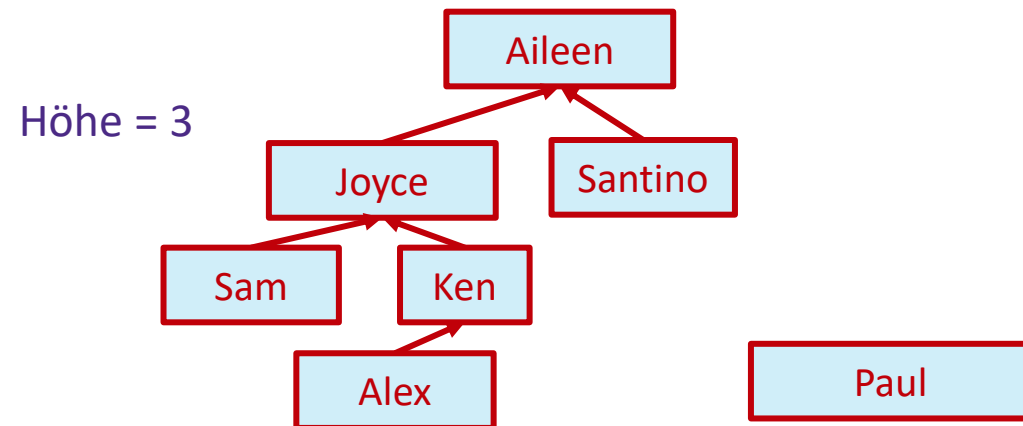


Ergebnis:



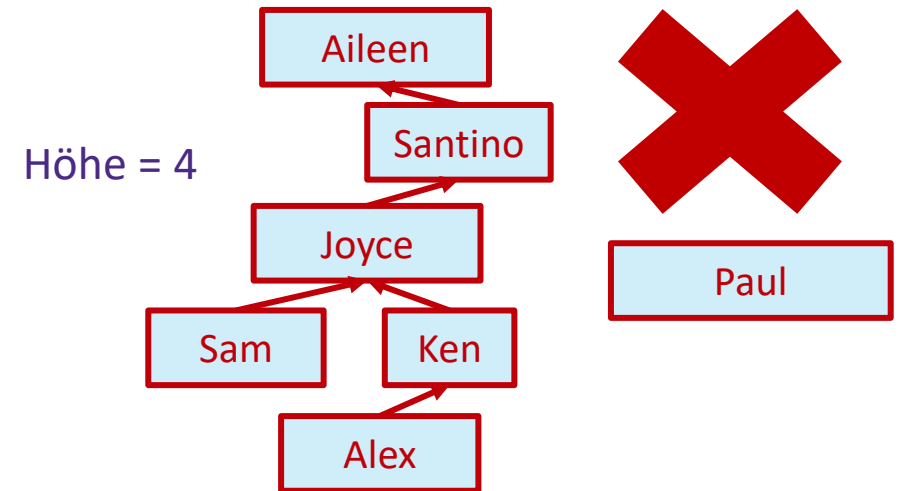
Quick-Union: Warum bestimmen wir den zweiten Wurzelknoten?

```
union("Ken", "Santino"):
    rootK = find("Ken")
    roots = find("Santino")
    set rootK to point to roots
```

Warum verwenden wir nicht einfach:

```
union("Ken", "Santino"):
    rootK = find("Ken")
    set rootK to point to Santino
```

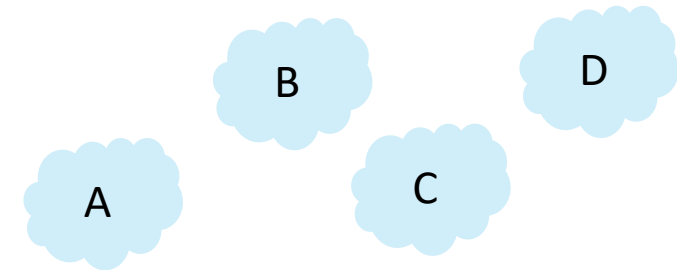


Grundgedanke: Die Laufzeit zukünftiger `find()`-Aufrufe kann minimiert werden, wenn wir die Höhe des Baums kurz halten!

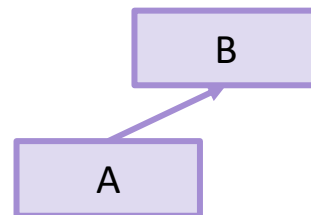
- Ein direkter Verweis auf das zweite Element (`Santino`) würde den Baum höher machen

Quick-Union: Worst Case Fall für find()

Versuche, eine Reihe von Aufrufen von union() zu finden, die eine Worst-Case-Laufzeit von find(A) für den finalen Up-Tree erzeugt:



union(A, B)



find(A):

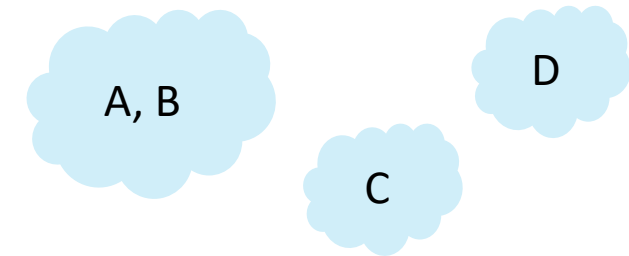
```
jump to node A
travel upward until root
return root
```

union(A, B):

```
rootA = find(A)
rootB = find(B)
set rootA to point to rootB
```

Quick-Union: Worst Case Fall für find()

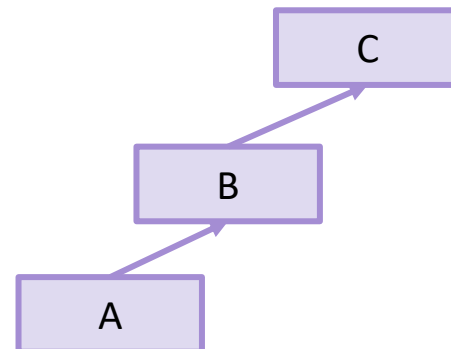
Versuche, eine Reihe von Aufrufen von union() zu finden, die eine Worst-Case-Laufzeit von find(A) für den finalen Up-Tree erzeugt:



find(A):
 jump to node A
 travel upward until root
 return root

union(B, C):
 rootB = find(B)
 rootC = find(C)
 set rootB to point to rootC

union(A, B)
 union(B, C)



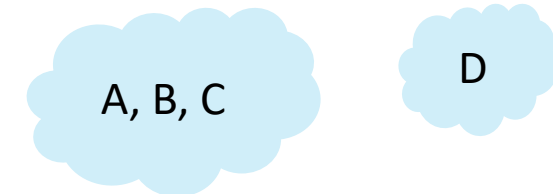
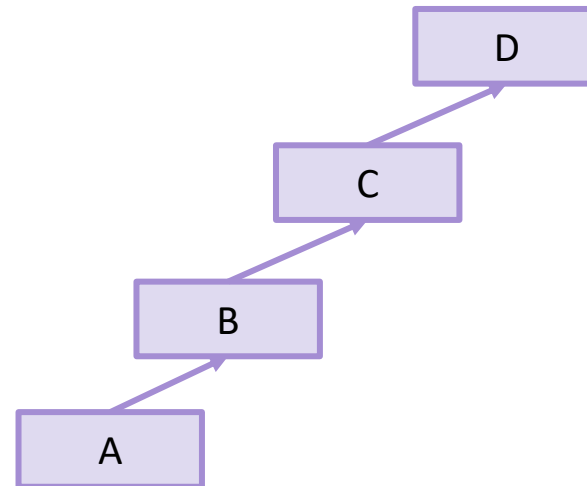
Quick-Union: Worst Case Fall für find()

Versuche, eine Reihe von Aufrufen von union() zu finden, die eine Worst-Case-Laufzeit von find(A) für den finalen Up-Tree erzeugt:

union(A, B)

union(B, C)

union(C, D)



find(A):

jump to node A
travel upward until root
return root

union(C, D):

rootC = find(C)
rootD = find(D)
set rootC to point to rootD

Quick-Union: Worst Case Fall für find()

Versuche, eine Reihe von Aufrufen von union() zu finden, die eine Worst-Case-Laufzeit von find(A) für den finalen Up-Tree erzeugt:

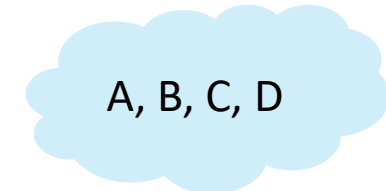
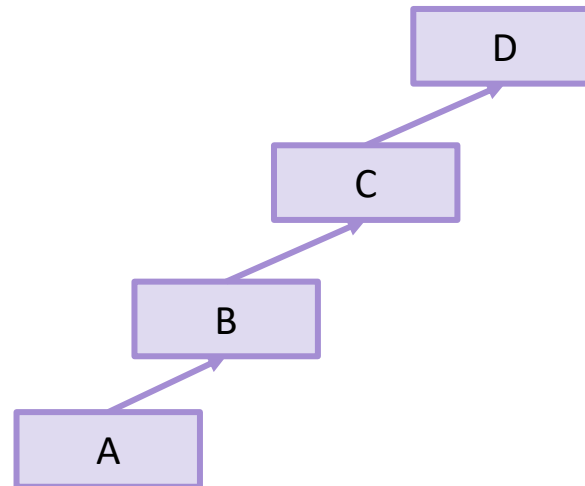
union(A, B)

union(B, C)

union(C, D)

find(A)

$O(n)$ Laufzeit :(



find(A):
 jump to node A
 travel upward until root
 return root

union(A, B):
 rootA = find(A)
 rootB = find(B)
 set rootA to point to rootB

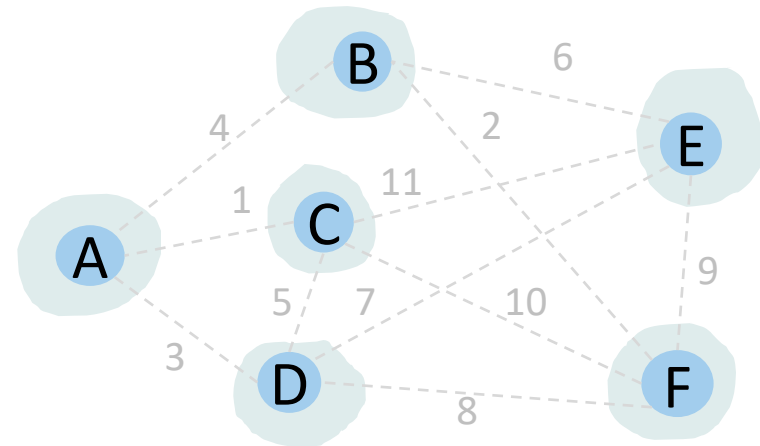
Übersicht für heute

- Minimale Spannbäume
 - Algorithmus von Prim
 - Algorithmus von Kruskal
- ADT
 - Menge, Disjunkte Mengen
- **Implementierung des ADT Disjunkter Mengen**
 - **Quick-Find**
 - **Wald disjunkter Mengen**
 - **Quick-Union, Weighted Quick-Union**
 - Pfadverkürzung
- Laufzeit des Algorithmus von Kruskal mit optimaler Implementierung des ADT Disjunkter Mengen

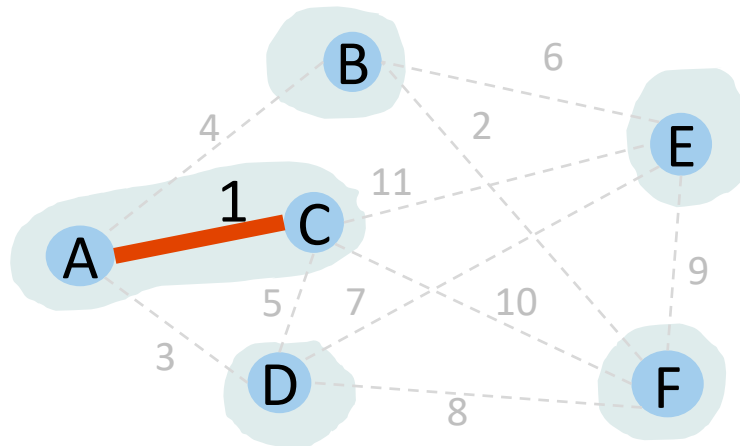
ADT Disjunkter Mengen – Wofür benötigen wir ihn?

Bspw. in Kruskal's Algorithmus

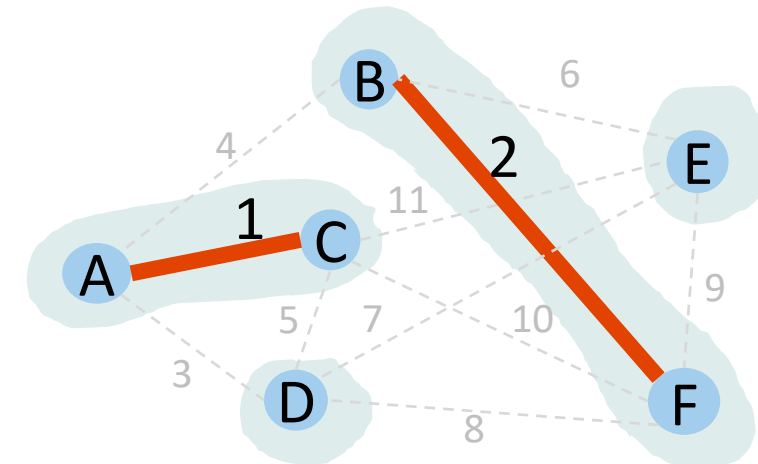
“Inseln”



“Inseln”

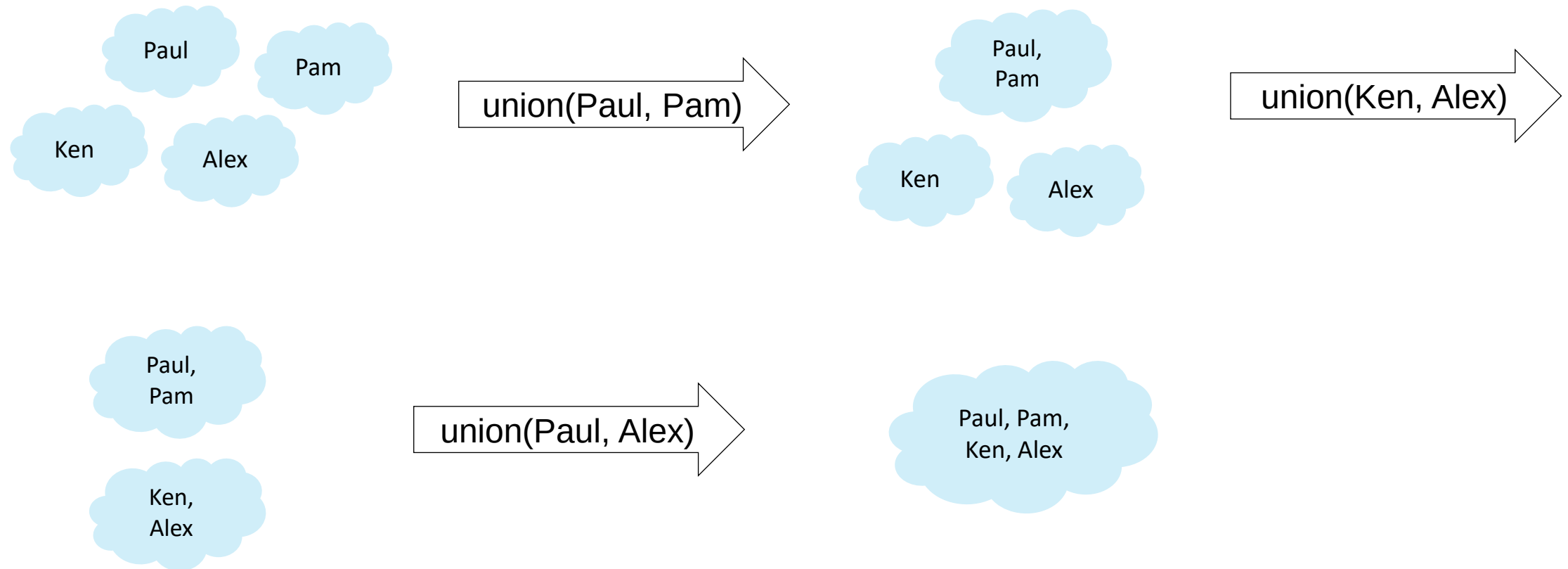


“Inseln”



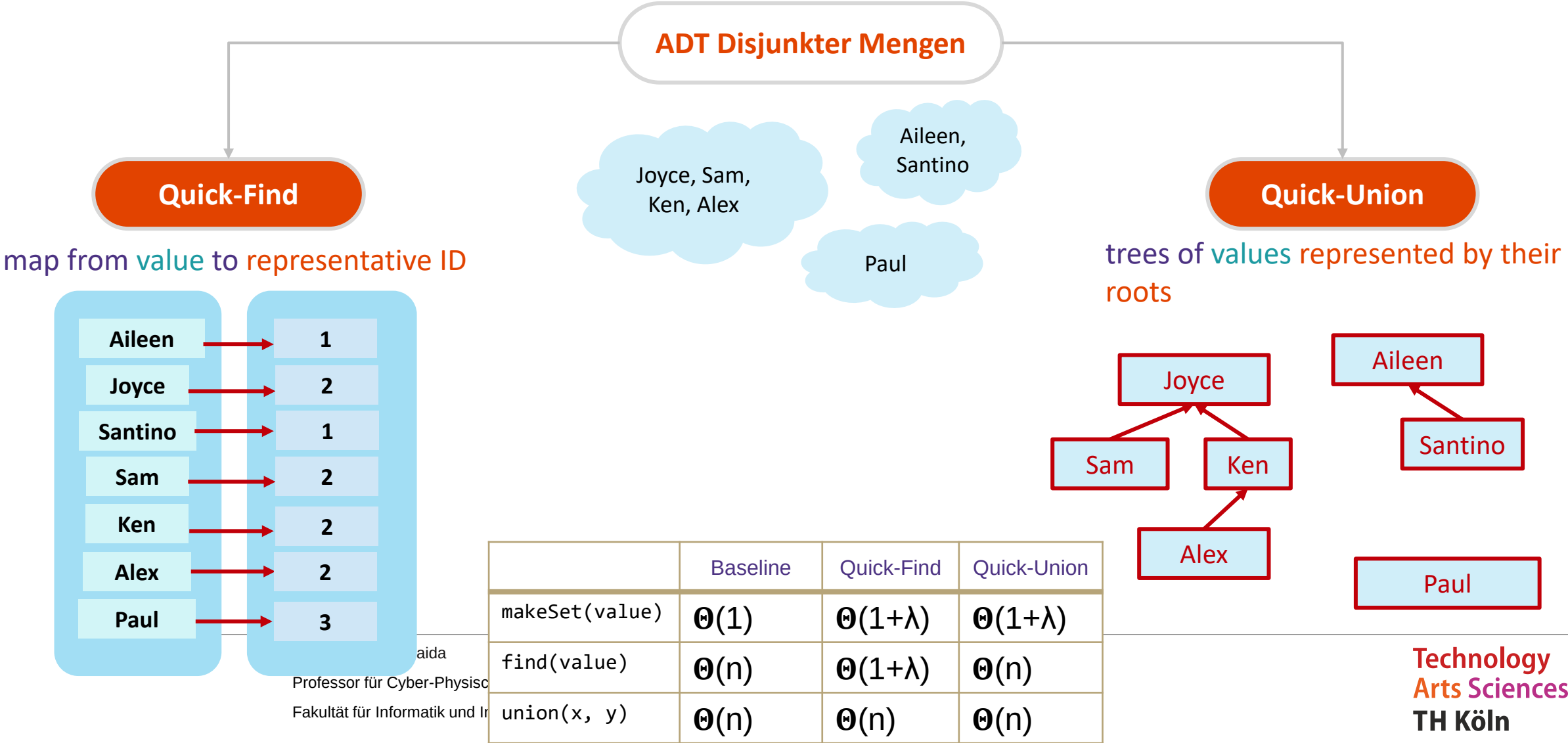
ADT Disjunkter Mengen – Wofür benötigen wir ihn?

Bspw. in Kruskal's Algorithmus



ADT Disjunkter Mengen: Implementierung

Quick-Find vs. Quick-Union



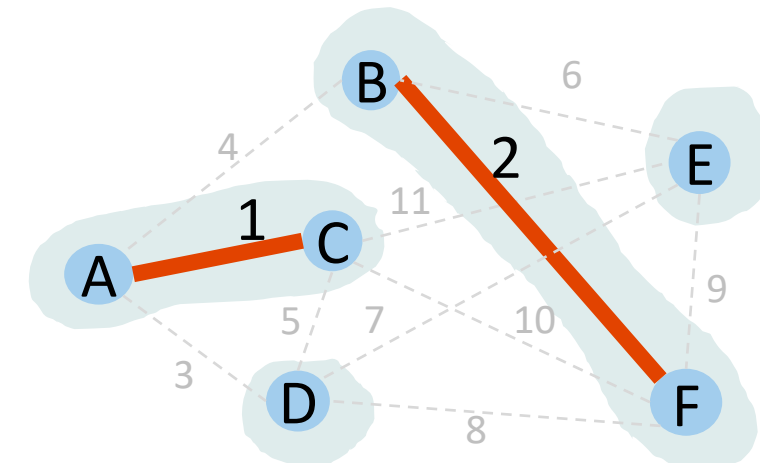
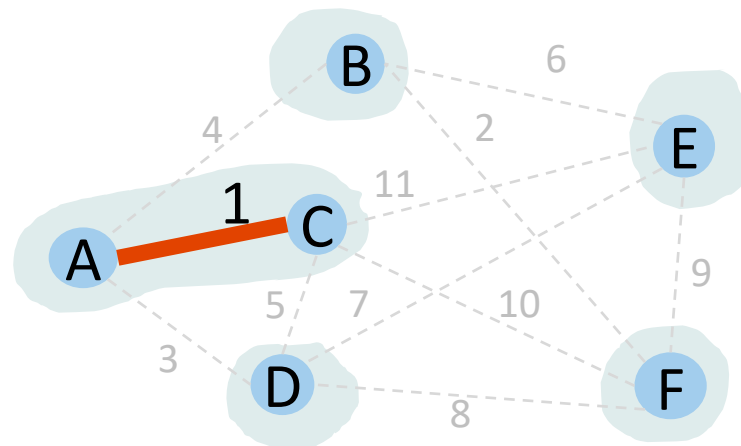
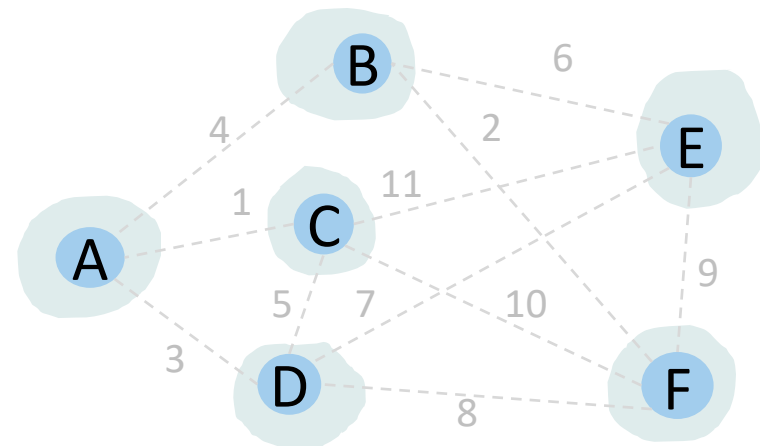
Kruskals Algorithmus - Übung

Zeichnen Sie den **Wald disjunkter Bäume** nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung **Quick-Union** für folgenden Graphen.

“Inseln”

“Inseln”

“Inseln”



setNodes:

A B C D E F

union(A, C)

C B D E F
A → C

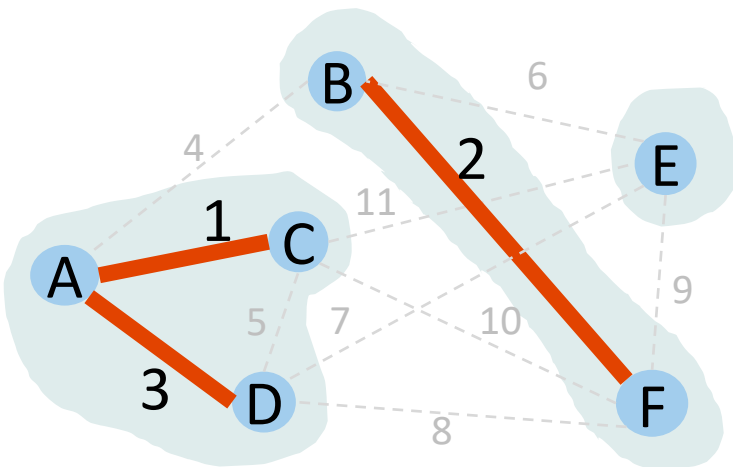
union(B, F)

C F D E
A → C, B → F

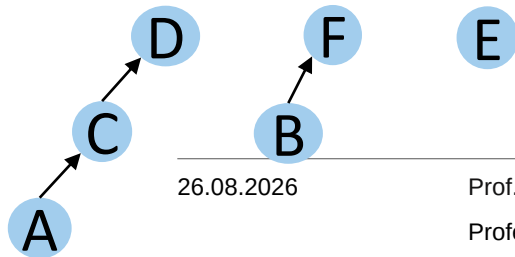
Kruskals Algorithmus - Übung

Zeichnen Sie den **Wald disjunkter Bäume** nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung **Quick-Union** für folgenden Graphen.

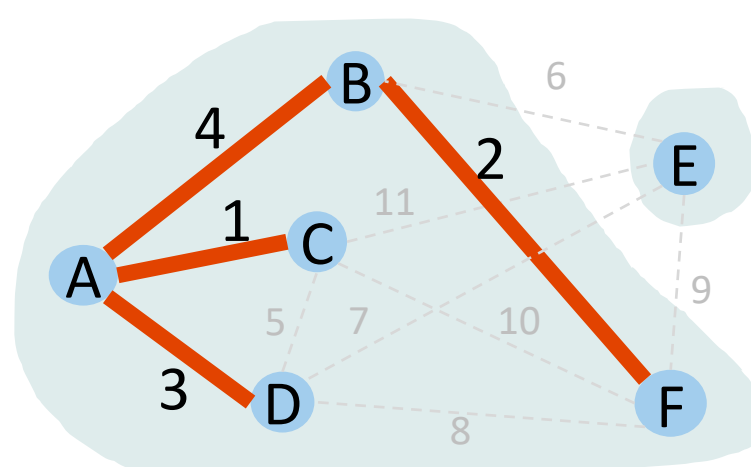
“Inseln”



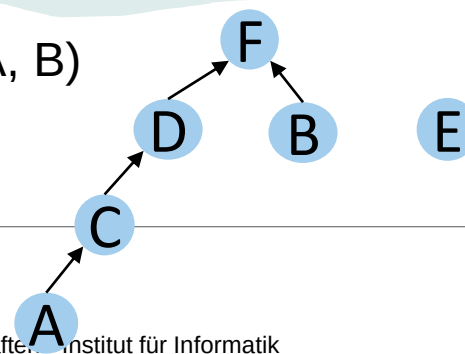
union(A, D)



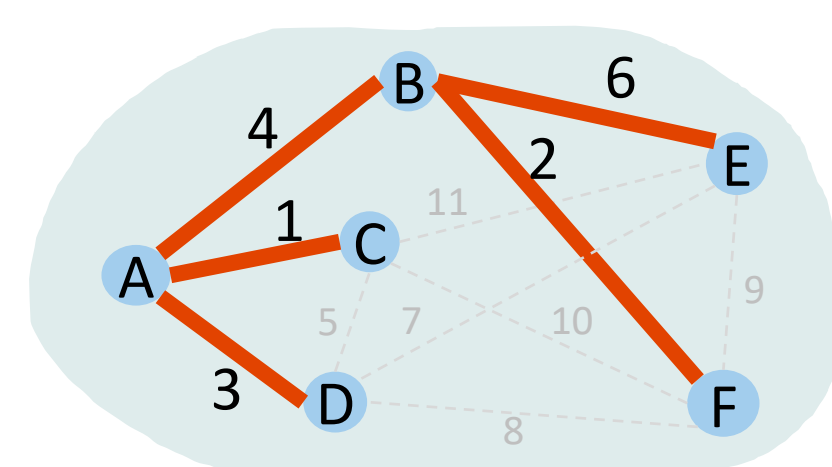
“Inseln”



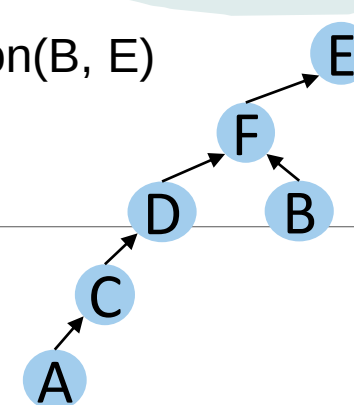
union(A, B)



“Inseln”



union(B, E)

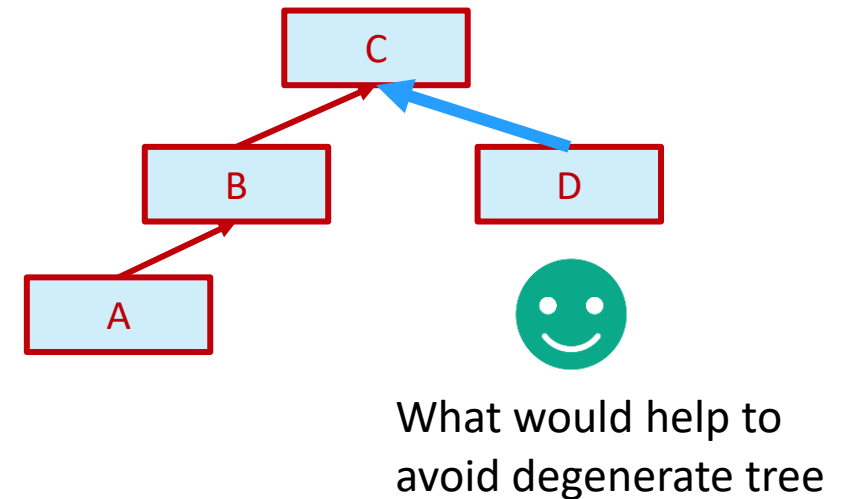
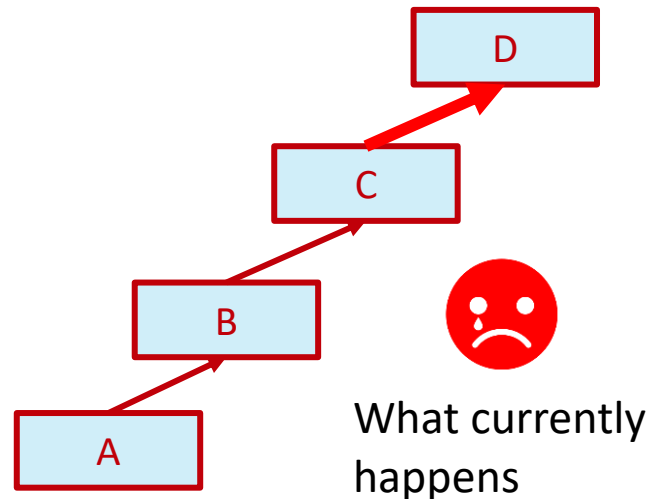


Analyse des Quick-Union Worst Case Falls

Wie kommt es zu dem degenerierten Baum?

- Wir erhalten einen degenerierten Baum, wenn wir Wurzel eines großen Baumes unter Wurzel eines kleinen Baumes setzen.
- In Quick-Union kommt rootX (Wurzel des Arguments x) immer unter rootY ($\text{union}(x, y)$)
- Aber was wäre, wenn wir sicherstellen, dass kleinere Baum unter größeren Baum gehangen wird?

$\text{union}(C, D)$



Weighted Quick-Union (Union by size)

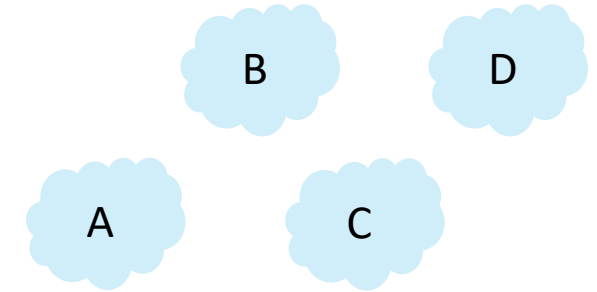
Ziel: Immer den kleineren Baum unter den größeren Baum hängen

Umsetzung: Speicherung der Anzahl der Knoten (oder des „Gewichts“) jedes Baums in der Wurzel

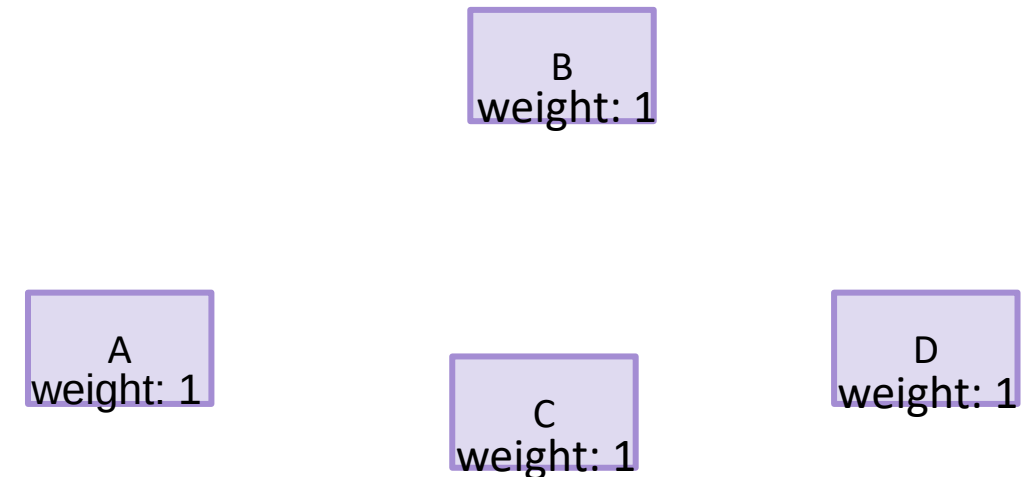
- Nachschlagen in konstanter Zeit, anstatt den gesamten Baum zum Zählen durchlaufen zu müssen

```
union(x, y):  
    rootX = find(x)  
    rootY = find(y)  
    put lighter root under heavier root
```

union(A, B)



setNode Objekte:



Weighted Quick-Union (Union by size)

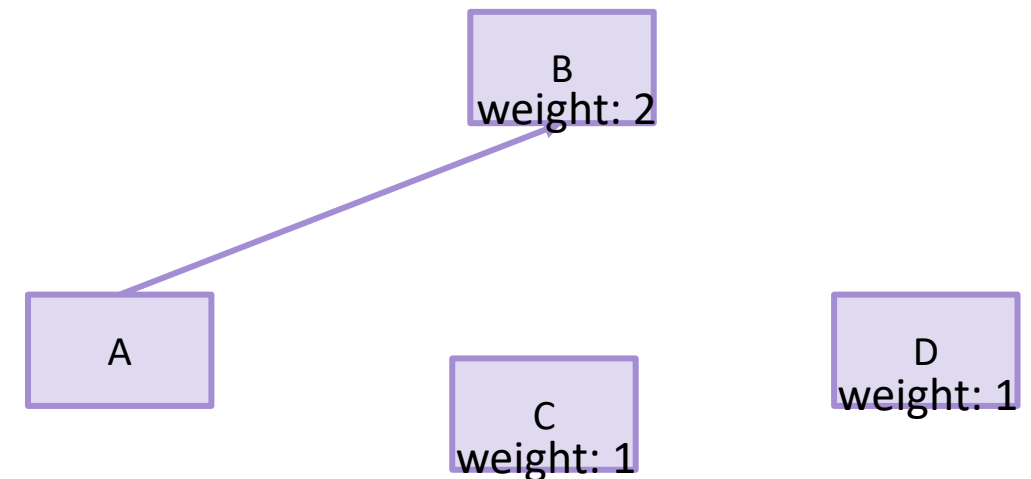
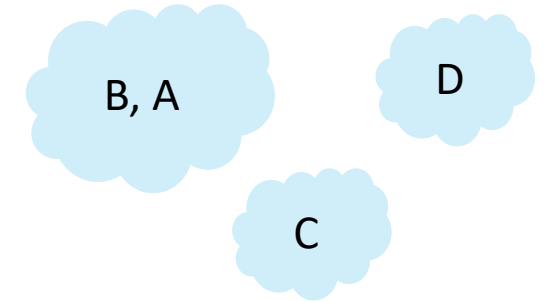
Ziel: Immer den kleineren Baum unter den größeren Baum hängen

Umsetzung: Speicherung der Anzahl der Knoten (oder des „Gewichts“) jedes Baums in der Wurzel

- Nachschlagen in konstanter Zeit, anstatt den gesamten Baum zum Zählen durchlaufen zu müssen

```
union(x, y):  
    rootX = find(x)  
    rootY = find(y)  
    put lighter root under heavier root
```

union(A, B)
union(B, C)



Weighted Quick-Union (Union by size)

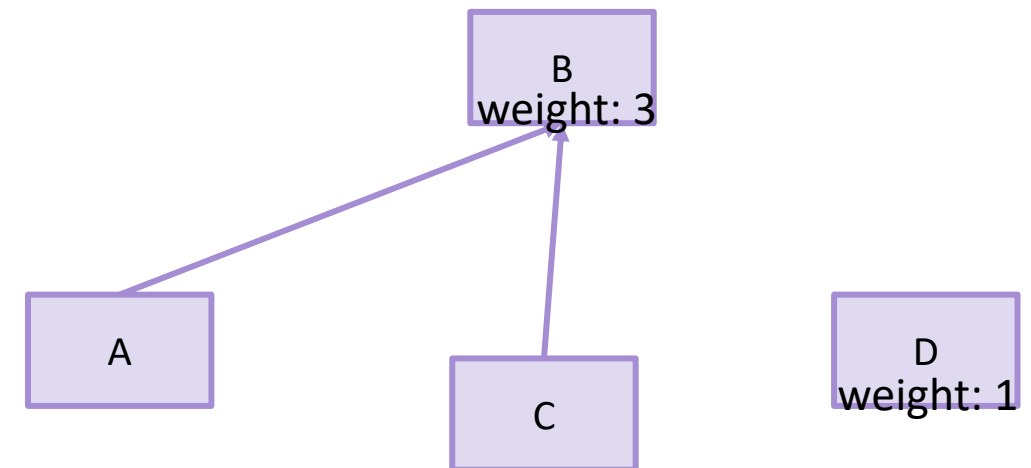
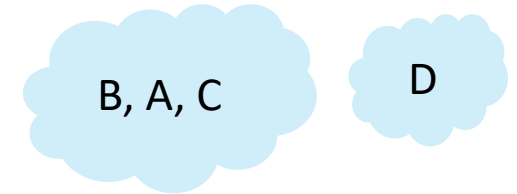
Ziel: Immer den kleineren Baum unter den größeren Baum hängen

Umsetzung: Speicherung der Anzahl der Knoten (oder des „Gewichts“) jedes Baums in der Wurzel

- Nachschlagen in konstanter Zeit, anstatt den gesamten Baum zum Zählen durchlaufen zu müssen

```
union(x, y):  
    rootX = find(x)  
    rootY = find(y)  
    put lighter root under heavier root
```

union(A, B)
union(B, C)
union(C, D)



Weighted Quick-Union (Union by size)

Ziel: Immer den kleineren Baum unter den größeren Baum hängen

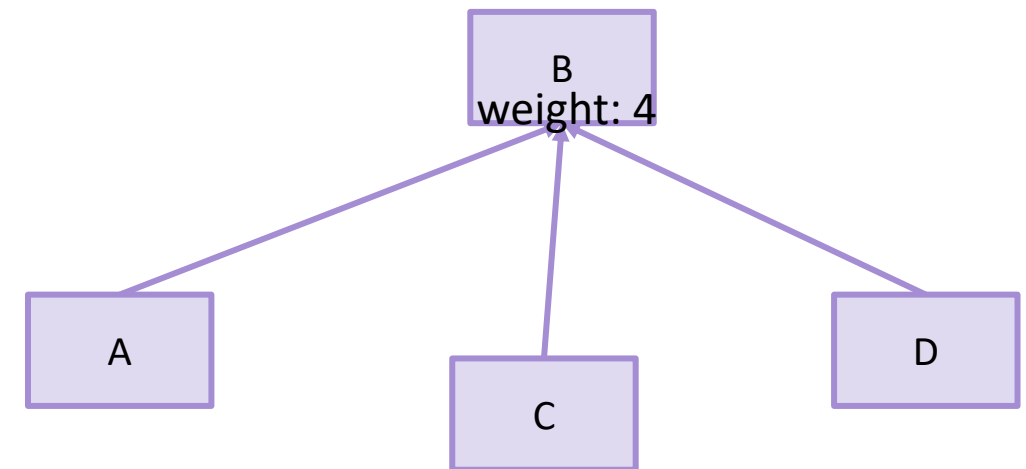
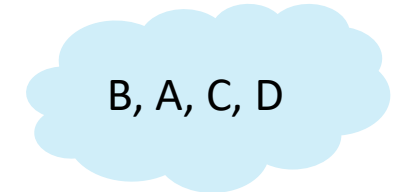
Umsetzung: Speicherung der Anzahl der Knoten (oder des „Gewichts“) jedes Baums in der Wurzel

- Nachschlagen in konstanter Zeit, anstatt den gesamten Baum zum Zählen durchlaufen zu müssen

```
union(x, y):  
    rootX = find(x)  
    rootY = find(y)  
    put lighter root under heavier root
```

```
union(A, B)  
union(B, C)  
union(C, D)  
find(A)
```

$O(1)$ runtime :)



Weighted Quick-Union: Performance

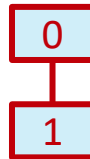
Betrachten wir den Worst Case, bei dem die Baumhöhe so schnell wie möglich wächst

n	H
1	0

0

Weighted Quick-Union: Performance

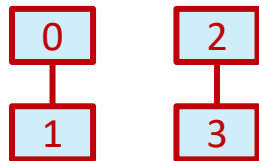
Betrachten wir den Worst Case, bei dem die Baumhöhe so schnell wie möglich wächst



n	H
1	0
2	1

Weighted Quick-Union: Performance

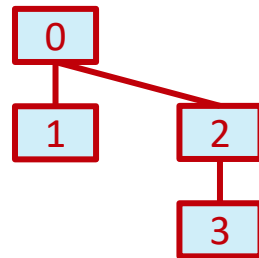
Betrachten wir den Worst Case, bei dem die Baumhöhe so schnell wie möglich wächst



n	H
1	0
2	1
4	?

Weighted Quick-Union: Performance

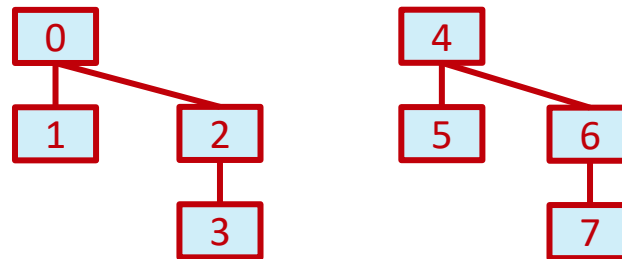
Betrachten wir den Worst Case, bei dem die Baumhöhe so schnell wie möglich wächst



n	H
1	0
2	1
4	2

Weighted Quick-Union: Performance

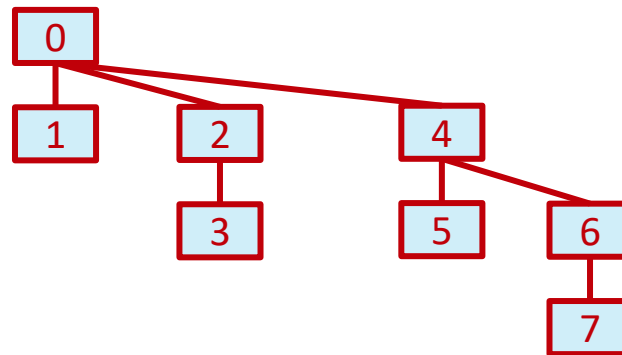
Betrachten wir den Worst Case, bei dem die Baumhöhe so schnell wie möglich wächst



n	H
1	0
2	1
4	2
8	?

Weighted Quick-Union: Performance

Betrachten wir den Worst Case, bei dem die Baumhöhe so schnell wie möglich wächst



n	H
1	0
2	1
4	2
8	3

Weighted Quick-Union: Performance

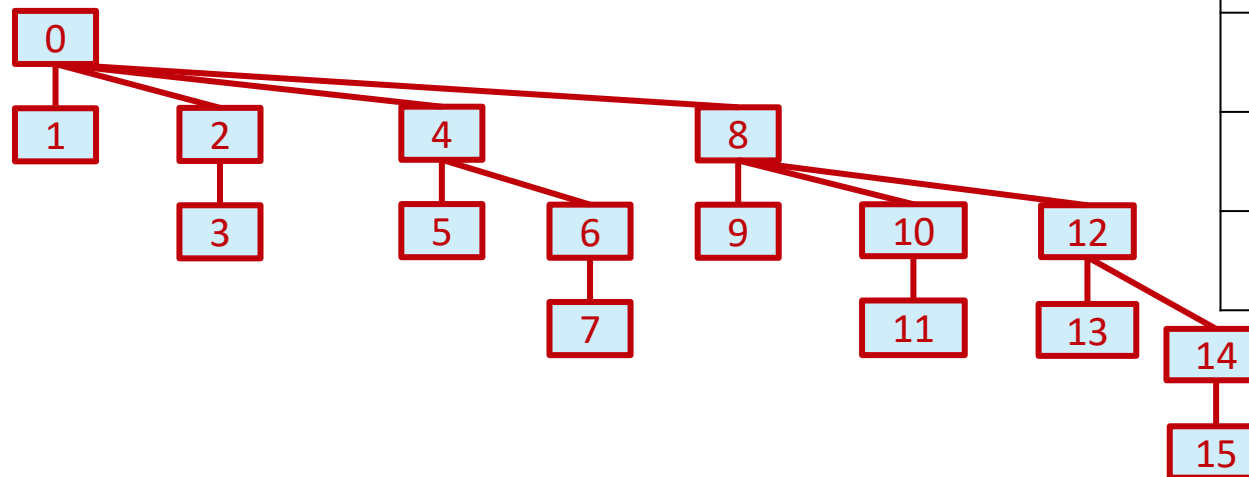
Betrachten wir den Worst Case, bei dem die Baumhöhe so schnell wie möglich wächst

Worst Case Baumhöhe ist $\Theta(\log n)$

- $\text{find}(15)$ hätte also Laufzeit von $\Theta(\log n)$

Laufzeiten:

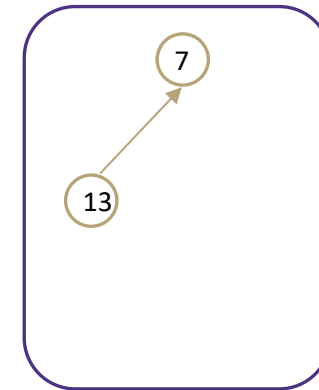
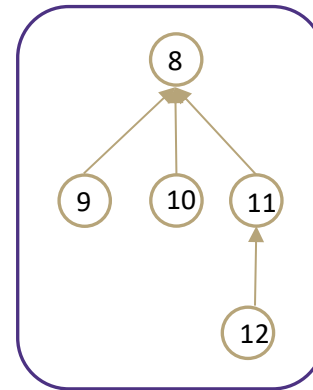
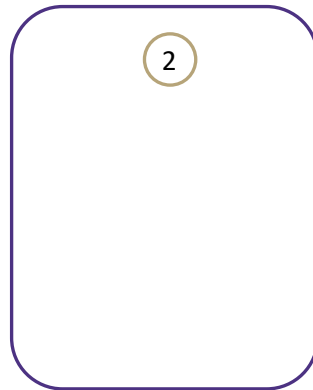
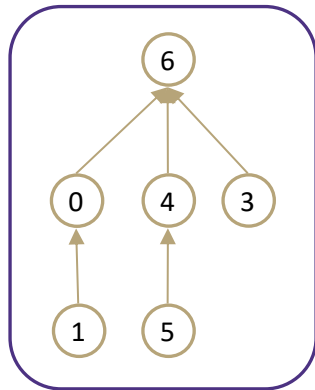
- $\text{find}()$
 - $\Theta(\log n)$
- $\text{union}()$
 - $\Theta(\log n)$



n	H
1	0
2	1
4	2
8	3
16	4

Weighted Quick-Union – Übung (1 von 5)

Was wäre bei den folgenden disjunkten Mengen das Ergebnis der folgenden Aufrufe von `union`, wenn wir Weighted Quick-Union nutzen? Zeichnen Sie den Wald in jeder Phase.



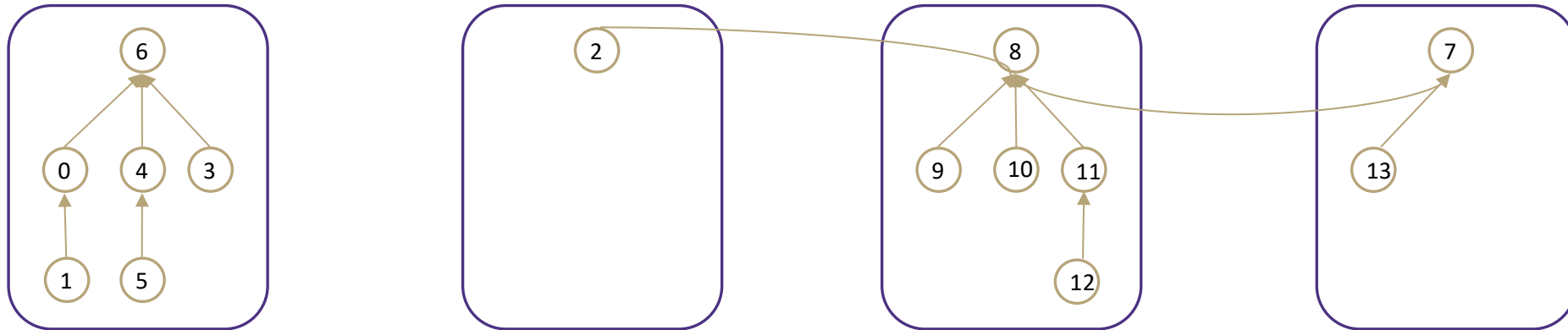
`union(2, 13)`

`union(4, 12)`

`union(2, 8)`

Weighted Quick-Union – Übung (2 von 5)

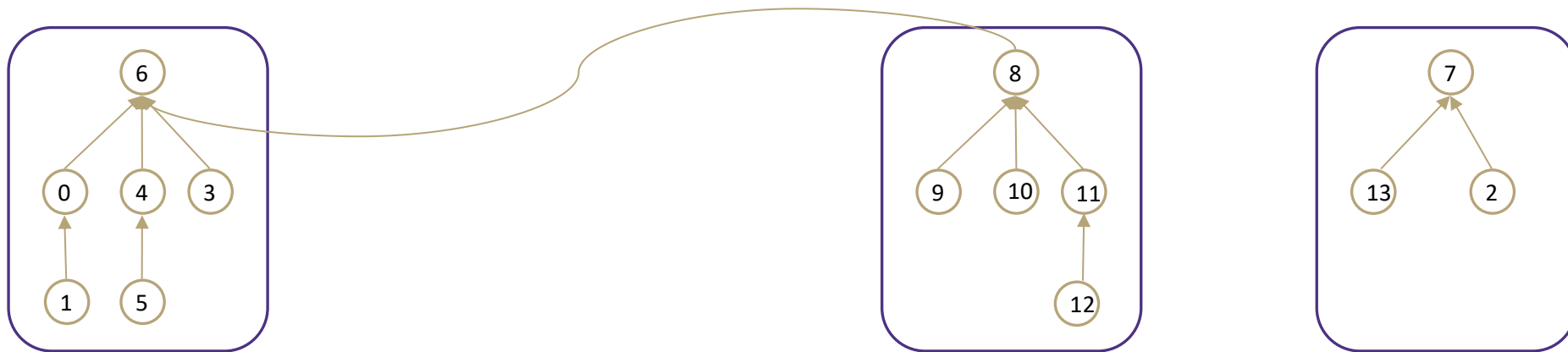
Was wäre bei den folgenden disjunkten Mengen das Ergebnis der folgenden Aufrufe von `union`, wenn wir Weighted Quick-Union nutzen? Zeichnen Sie den Wald in jeder Phase.



`union(2, 13)`

Weighted Quick-Union – Übung (3 von 5)

Was wäre bei den folgenden disjunkten Mengen das Ergebnis der folgenden Aufrufe von `union`, wenn wir Weighted Quick-Union nutzen? Zeichnen Sie den Wald in jeder Phase.

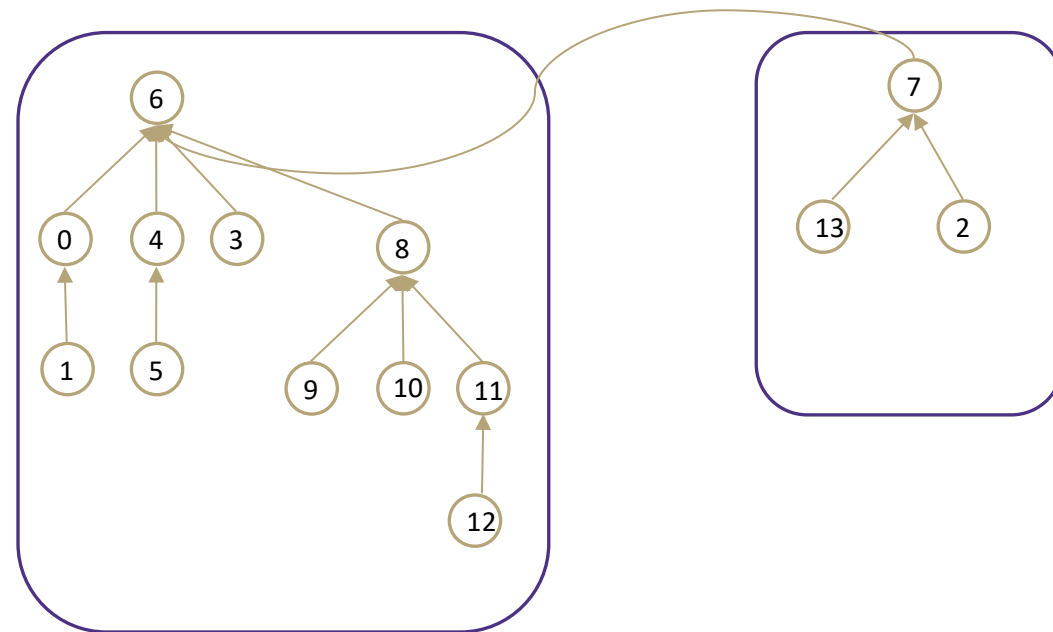


`union(2, 13)`

`union(4, 12)`

Weighted Quick-Union – Übung (4 von 5)

Was wäre bei den folgenden disjunkten Mengen das Ergebnis der folgenden Aufrufe von `union`, wenn wir Weighted Quick-Union nutzen? Zeichnen Sie den Wald in jeder Phase.



`union(2, 13)`

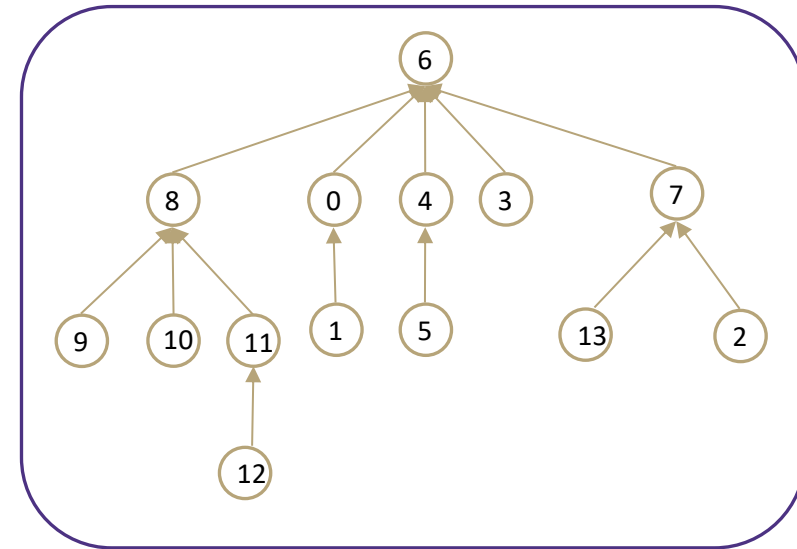
`union(4, 12)`

`union(2, 8)`

Weighted Quick-Union – Übung (5 von 5)

Was wäre bei den folgenden disjunkten Mengen das Ergebnis der folgenden Aufrufe von `union`, wenn wir Weighted Quick-Union nutzen? Zeichnen Sie den Wald in jeder Phase.

Does this improve the worst case runtime?
`find()` is more likely to be $O(\log(n))$ than $O(n)$

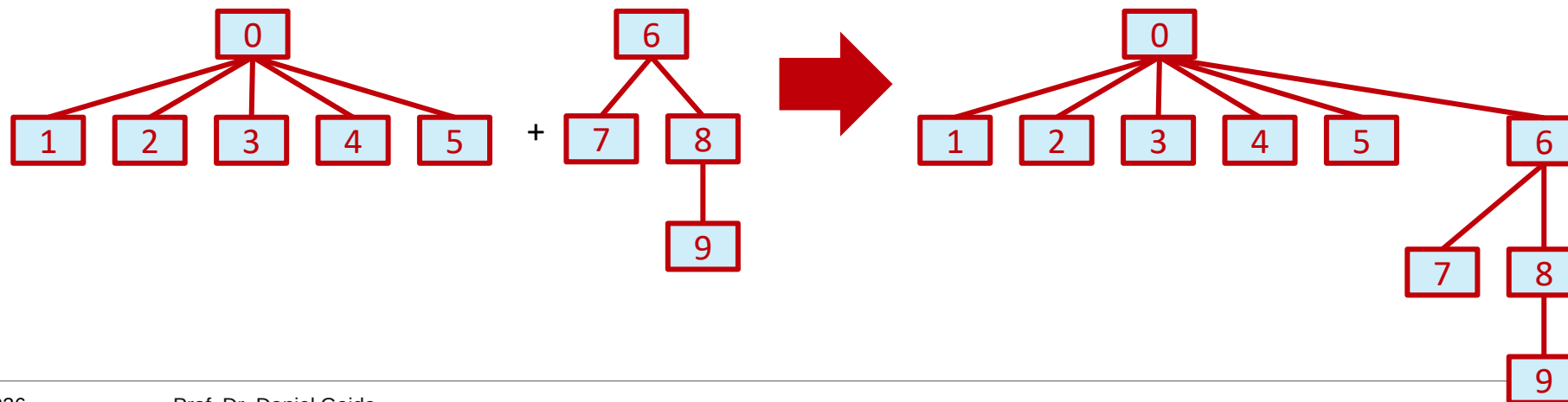


Why Weights Instead of Heights?

We used the number of items in a tree to decide upon the root

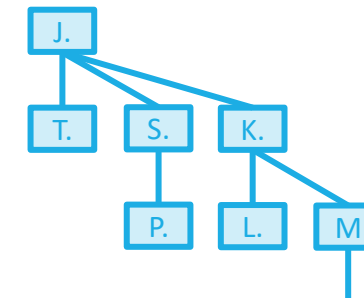
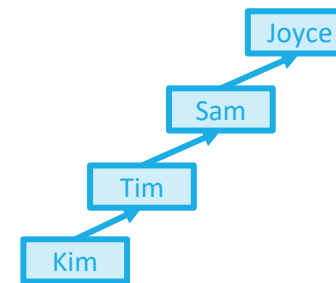
Why not use the height of the tree?

- Heighted Quick-Union's runtime is asymptotically the same: $\Theta(\log(n))$
- It's easier to track weights than heights, even though Weighted Quick-Union can lead to some suboptimal structures like this one:



Weighted Quick-Union Laufzeit

	Baseline	Quick-Find	Quick-Union	Weighted Quick-Union
makeSet(value)	$\Theta(1)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$
find(value)	$\Theta(n)$	$\Theta(1+\lambda)$	$\Theta(n)$	$\Theta(\log n)$
union(x, y) assuming root args	$\Theta(n)$	$\Theta(n)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$
union(x, y)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$



Das ist ziemlich gut! Es gibt noch eine letzte Optimierung, die wir vornehmen können: Pfadkompression

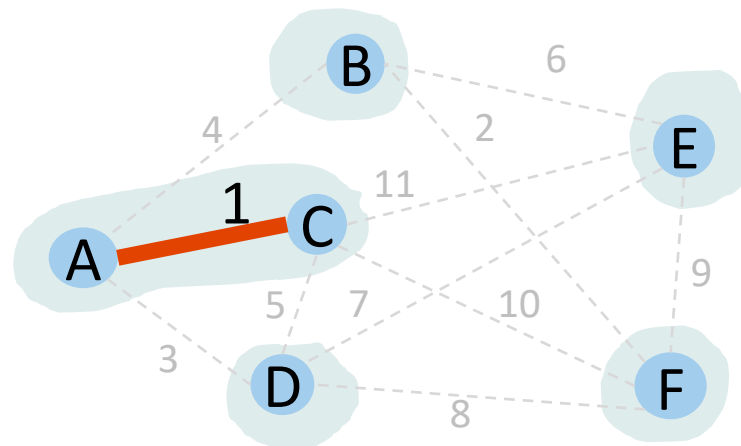
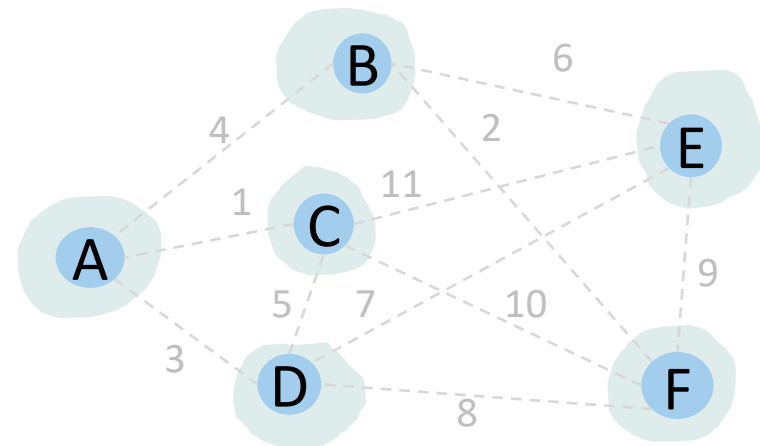
Kruskals Algorithmus - Übung

Zeichnen Sie den Wald disjunkter Bäume nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung **Weighted Quick-Union** für folgenden Graphen.

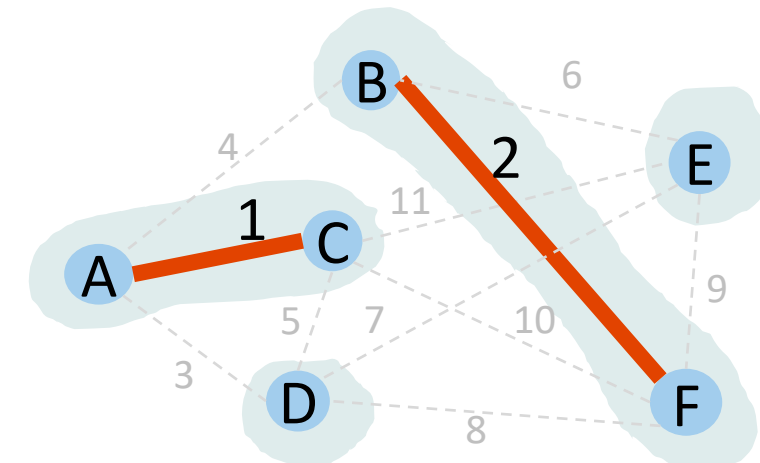
“Inseln”

“Inseln”

“Inseln”



union(A, C)



union(B, F)

A B C D E F

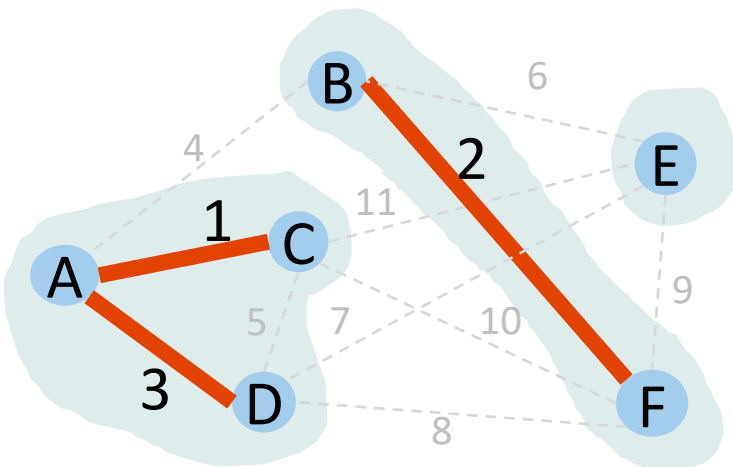
A B D E F
C → A

A B D E
C → A, F → B

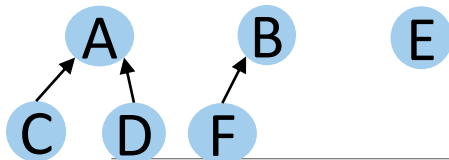
Kruskals Algorithmus - Übung

Zeichnen Sie den Wald disjunkter Bäume nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung **Weighted Quick-Union** für folgenden Graphen.

“Inseln”



union(A, D)



26.08.2026

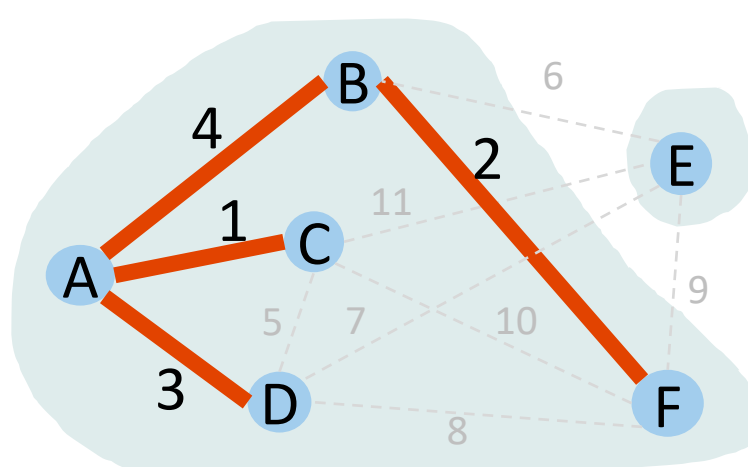
Seite 141

Prof. Dr. Daniel Gaida

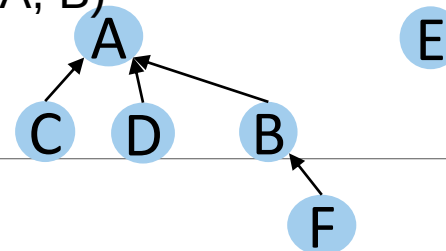
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

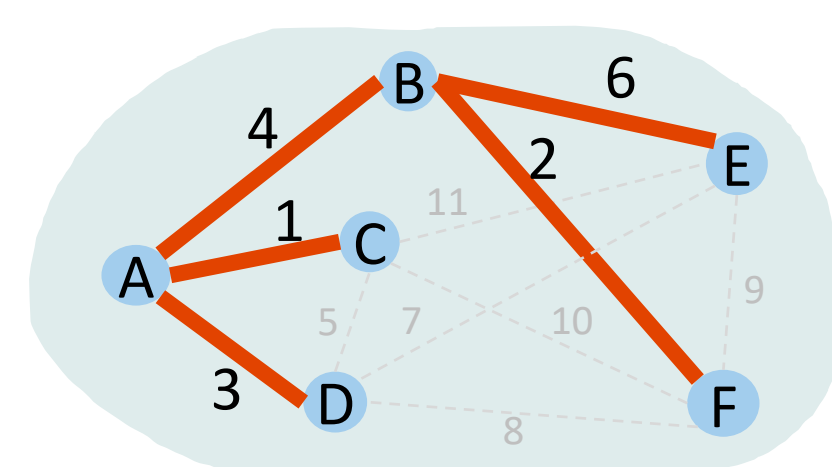
“Inseln”



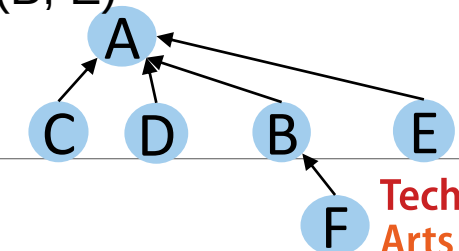
union(A, B)



“Inseln”



union(B, E)



Modifizierung von Datenstrukturen für zukünftige Verbesserungen

Die bisher untersuchten Änderungen von Datenstrukturen zielten darauf ab, Invarianten zu bewahren

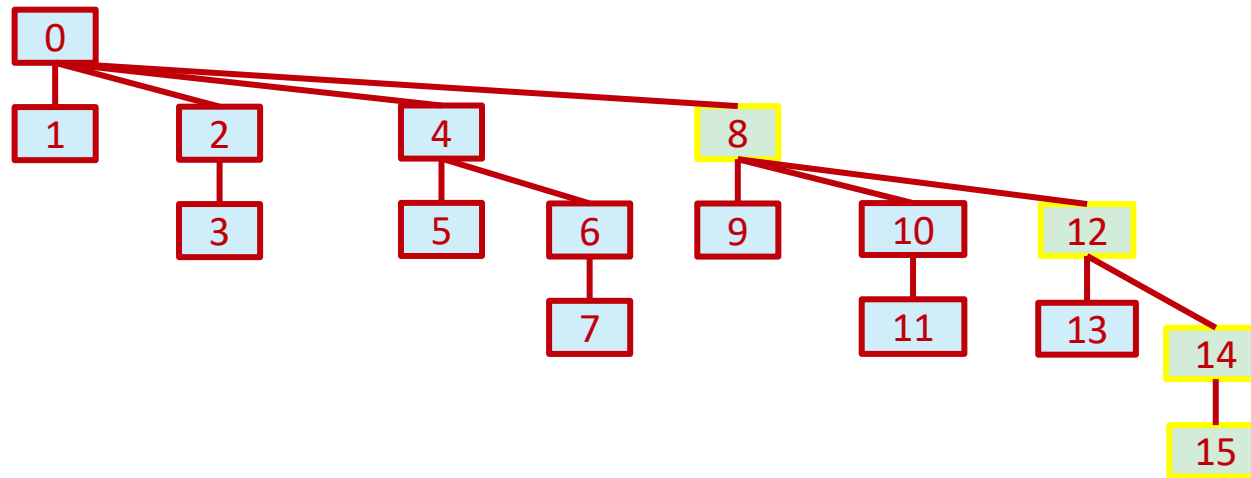
- Z.B. Durchführung von Rotationen zur Erhaltung der AVL-Invariante
- Wir verlassen uns darauf, dass diese Invarianten immer wahr sind, damit jeder Aufruf schnell ist.

Die Pfadkompression/Pfadverkürzung ist etwas ganz anderes:

- Wir ändern die Baumstruktur, um zukünftige Laufzeiten zu verbessern
- Keine Definition einer Invariante
- Der erste Aufruf von `find()` kann langsam sein. In diesem Aufruf wird die Baumstruktur verändert, so dass zukünftige Aufrufe von `find()` schnell sein können.

Weighted Quick-Union mit Pfadverkürzung - Idee

Dies ist die ungünstigste Topologie, wenn wir Weighted Quick-Union verwenden:

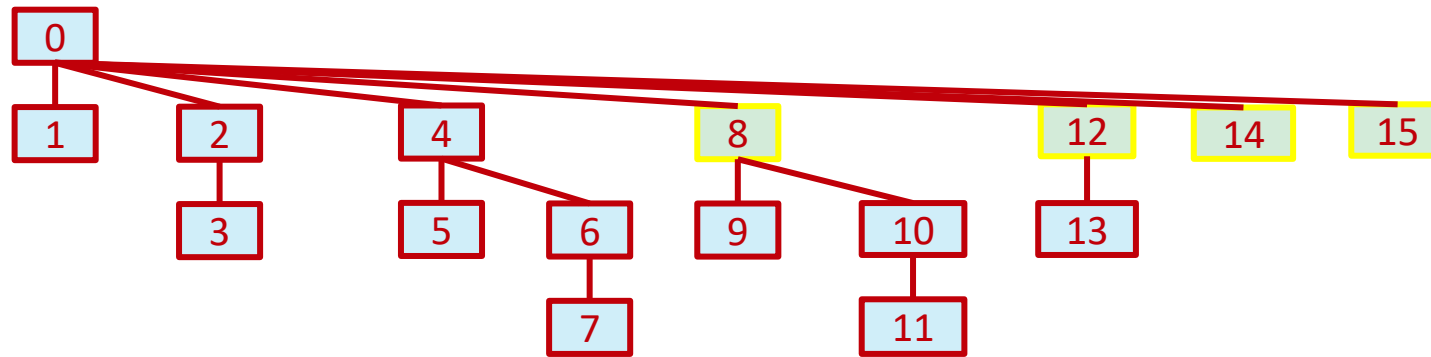


Idee: Wenn wir `find(15)` aufrufen, verschieben wir alle besuchten Knoten unter die Wurzel

- Zusätzliche Kosten sind unbedeutend (wir müssen diese Knoten ohnehin besuchen, nur konstante Laufzeit, um diese auf die Wurzel zeigen zu lassen)

Weighted Quick-Union mit Pfadverkürzung - Idee

Dies ist die ungünstigste Topologie, wenn wir Weighted Quick-Union verwenden:



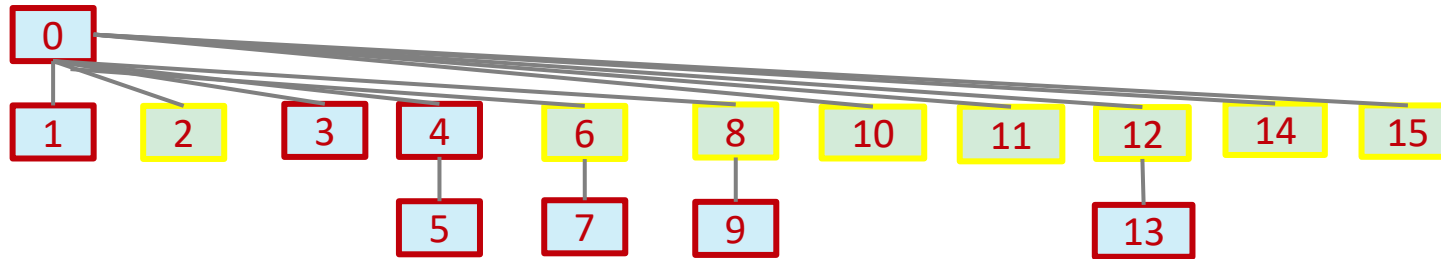
Idee: Wenn wir `find(15)` aufrufen, verschieben wir alle besuchten Knoten unter die Wurzel

- Zusätzliche Kosten sind unbedeutend (wir müssen diese Knoten ohnehin besuchen, nur konstante Laufzeit, um diese auf die Wurzel zeigen zu lassen)
- Bei jedem `find()` wird Pfadverkürzung durchgeführt, damit zukünftige Aufrufe von `find()` schneller sind!

Weighted Quick-Union mit Pfadverkürzung – Details und Laufzeit

Führen Sie die Pfadverkürzung bei jedem `find()` aus!

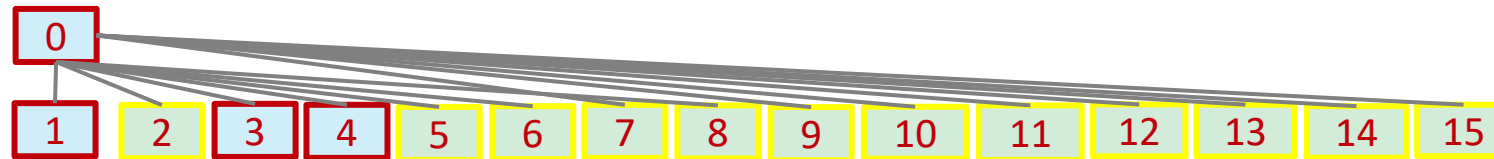
- `find(3)`, `find(6)`, `find(11)`, ...



Um die Laufzeit von $M > 1$ `find()` Aufrufe zu verstehen, ist eine amortisierte Laufzeitanalyse erforderlich

Weighted Quick-Union mit Pfadverkürzung - Laufzeit

M Aufrufe von find() bei **Weighted Quick-Union** erfordert Laufzeit von $\Theta(M \log n)$



... aber M find()s bei **Weighted Quick-Union mit Pfadverkürzung** dauert $O(M \log^* n)$!

- $\log^* n$ ist der „iterierte Logarithmus“: die **Anzahl** der Anwendungen von $\log$ auf n , bis das Ergebnis kleiner gleich 1 ist.
 - $\log_2^*(4) = 2$, da $\log_2(\log_2(4)) = 1$
- Tarjan, Robert E., and Jan Van Leeuwen. "Worst-case analysis of set union algorithms." *Journal of the ACM (JACM)* 31.2 (1984): 245-281.

Weighted Quick-Union mit Pfadverkürzung - Laufzeit

Pfadverkürzung führt zu Laufzeiten von $\text{find}()$ und $\text{union}()$, die sehr nahe an der (amortisierten) konstanten Laufzeit liegen

- $\log^*$ ist weniger als 5 für jede realistische Eingabe
- Wenn die Laufzeit von M $\text{find}()$ s/ $\text{union}()$ s auf n Knoten $O(M \log^* n)$ ist und $\log^* n \leq 5$, dann amortisiert sich Laufzeit von $\text{find}()$ / $\text{union}()$ s zu $O(1)$!

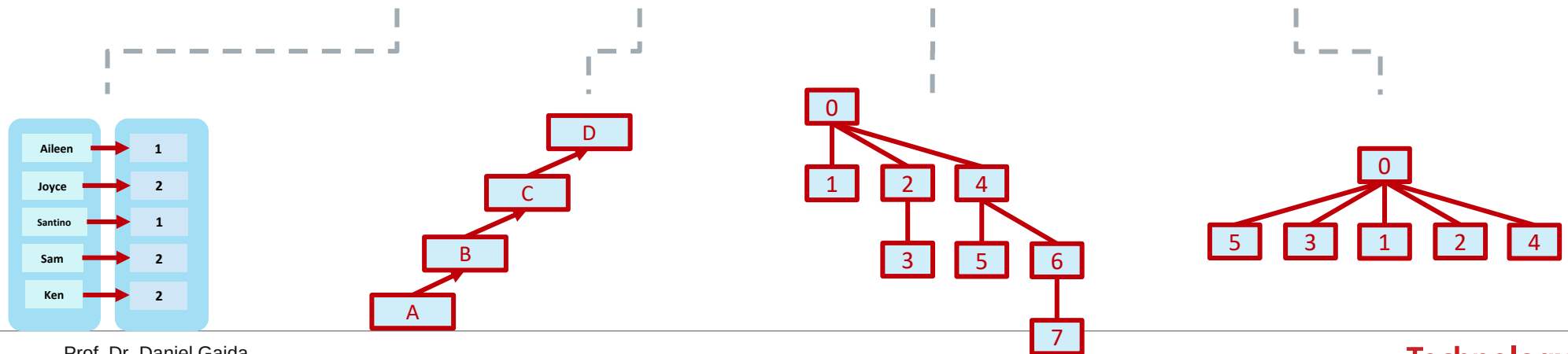
n	$\log^* n$
1	0
2	1
4	2
16	3
65536	4
2^{65536}	5

2^{16} →
 Number of atoms in the known universe is 2^{256} ish →

ADT Disjunkter Mengen - Implementierungen

Laufzeiten „In der Praxis“:

	Baseline	Quick-Find	Quick-Union	Weighted Quick-Union	WQU + Pfadverkürzung
makeSet(value)	$\Theta(1)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$
find(value)	$\Theta(n)$	$\Theta(1+\lambda)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log^* n)$
union(x, y) assuming root args	$\Theta(n)$	$\Theta(n)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$	$\Theta(1+\lambda)$
union(x, y)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log^* n)$



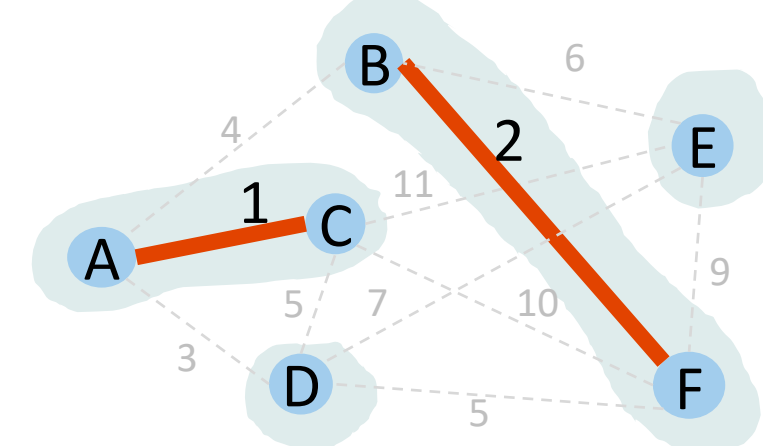
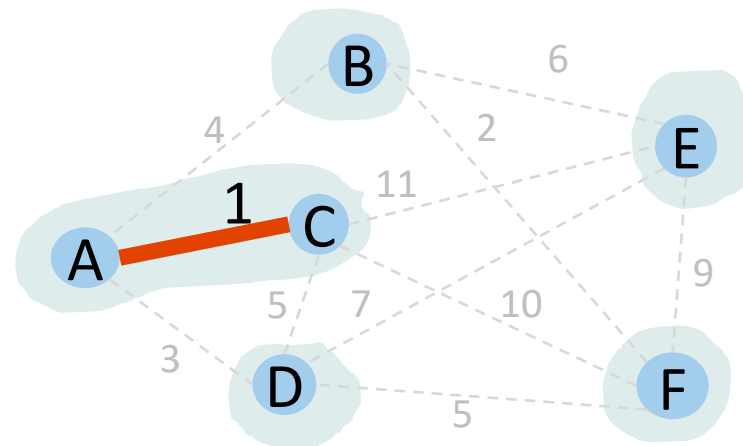
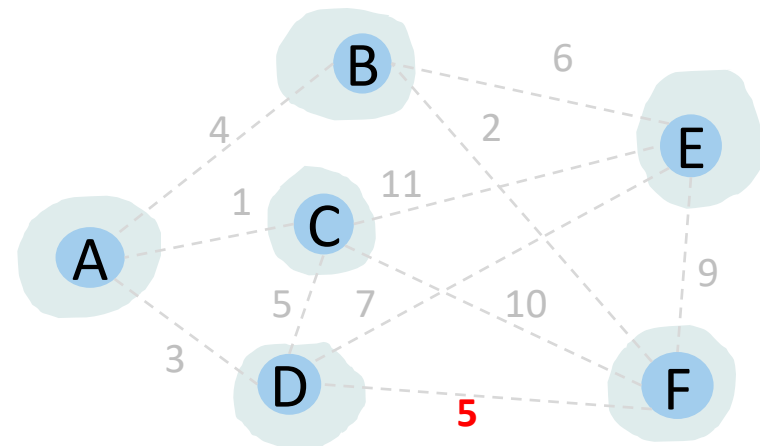
Kruskals Algorithmus - Übung

Zeichnen Sie den Wald disjunkter Bäume nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung **Weighted Quick-Union mit Pfadverkürzung** für folgenden Graphen.

“Inseln”

“Inseln”

“Inseln”



union(A, C)

union(B, F)

A B C D E F

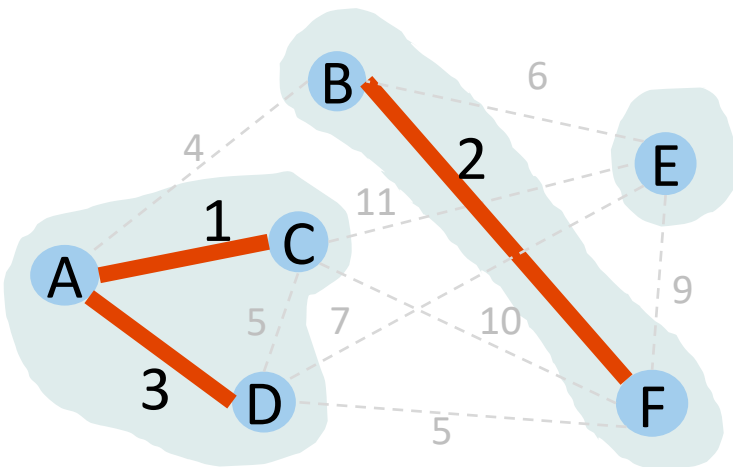
A B D E F
C → A

A B D E
C → A, F → B

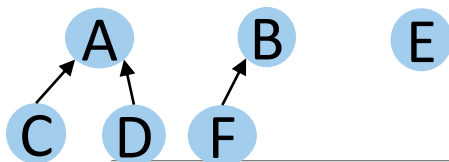
Kruskals Algorithmus - Übung

Zeichnen Sie den Wald disjunkter Bäume nach jeder Iteration des Algorithmus von Kruskal bei Nutzung der Implementierung **Weighted Quick-Union mit Pfadverkürzung** für folgenden Graphen.

“Inseln”



union(A, D)



26.08.2026

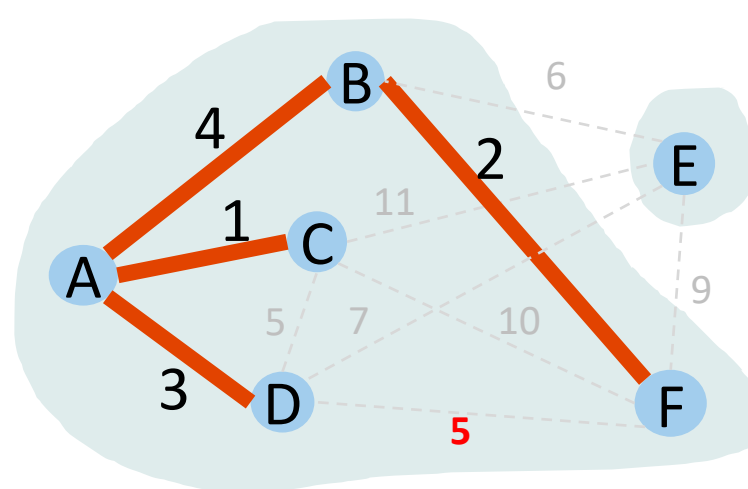
Seite 152

Prof. Dr. Daniel Gaida

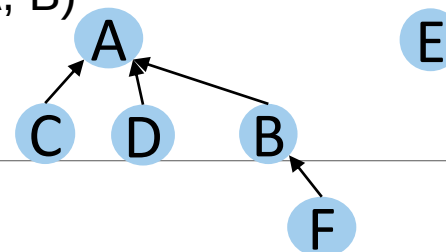
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

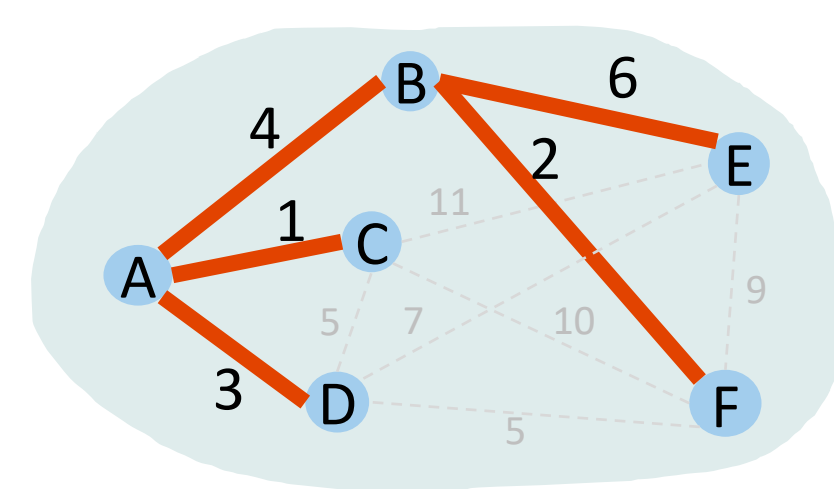
“Inseln”



union(A, B)

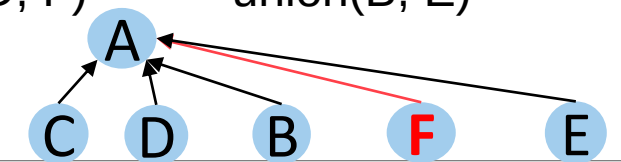


“Inseln”



union(D, F)

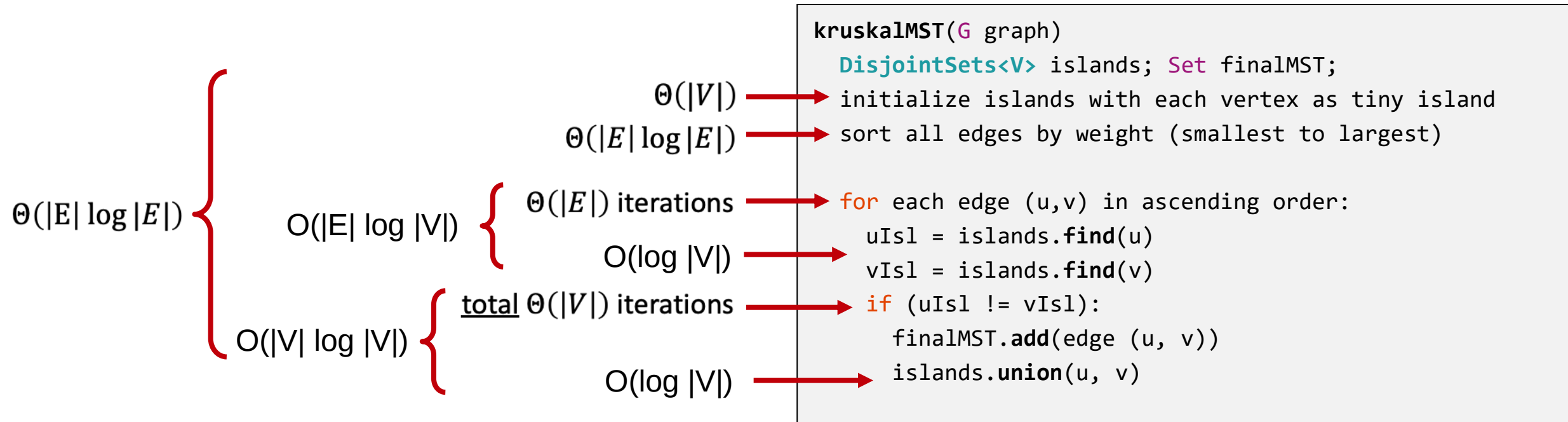
union(B, E)



Knoten F wandert unter die Wurzel, da Kante (D, F) geprüft wird

Technology
Arts Sciences
TH Köln

Laufzeit von Kruskals Algorithmus



find und union haben Worst Case Laufzeit $\log|V|$, aber amortisiert konstant „in der Praxis“

- So oder so, Laufzeit wird dominiert durch Sortieren der Kanten 😞

Damit ein MST existieren kann, darf $|E|$ nicht kleiner als $|V|$ sein, also muss die Anzahl Kanten dominieren.

Wh.: Prim's Algorithmus – Asymptotische Laufzeit

Damit ein MST existieren kann, darf $|E|$ nicht kleiner als $|V|$ sein, also muss die Anzahl Kanten dominieren:

→ $\Theta(m \log(n))$

$\Theta(|V| \log |V| + |E| \log |V|)$

Prim schneller wenn $m \gg n$.
Kruskal hat Vorteil, dass er auch für nicht zusammenhängende Graphen funktioniert:
minimal aufspannender Wald

$\Theta(|V| \log |V|)$

$\Theta(|E| \log |V|)$

```

primsShortestPath(G graph, V start)
    Map edgeTo, distTo; Set known;
    initialize distTo with all nodes mapped to  $\infty$ , except start to 0
    PriorityQueue<V> perimeter; perimeter.add(start);

    while (!perimeter.isEmpty()):
        u = perimeter.removeMin()
        known.add(u)
        for each edge (u,v) to unknown v with weight w:
            oldDist = distTo.get(v)      // previous smallest edge to v
            newDist = w // is this a smaller edge to v?
            if (newDist < oldDist):
                distTo.put(v, newDist)
                edgeTo.put(v, u)
            if (perimeter.contains(v)):
                perimeter.changePriority(v, newDist)
            else:
                perimeter.add(v, newDist)
    
```

Ihr Werkzeugkasten bis hierhin...

Abstrakte Datentypen

- Liste - Flexibilität, einfaches Verschieben von Elementen innerhalb der Struktur
- Stapel (Stack) - optimiert für First-in-Last-out-Reihenfolge
- Warteschlange - optimiert für die Reihenfolge First-in-First-out.
- Tabelle - speichert Schlüssel und Wert bei jedem Eintrag
- Prioritätswarteschlange - optimiert für Ausgabe der höchsten Priorität zuerst
- Menge – Speichern einer mathematischen Menge von Elementen
- Disjunkte Mengen – effizientes Arbeiten mit disjunkten Mengen

Implementierung von Datenstrukturen

- Array - leicht ein Element über Index zu suchen, schwer umzuordnen
- Verkettete Liste - schwer ein Element zu suchen, leicht umzuordnen
- Hashtabelle - konstante Zeit zum Suchen, keine Ordnung der Daten
- Binärer Suchbaum - effizientes Suchen, Möglichkeit eines schlechten Worst Case
- AVL-Baum - effizientes Suchen, schützt vor dem Worst Case, schwer zu implementieren
- Heap - effizient für Min oder Max Werte
- Graph – Beliebige Relationen zwischen Daten

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können den minimalen Spannbaum eines Graphen ermitteln, indem sie den Algorithmus von Prim und Kruskal effizient implementieren können und beim Algorithmus von Kruskal dabei den ADT disjunkter Mengen nutzen können, um später, mit dem minimalen Spannbaum, eine weitere Fragestellung an praktischen Graphenproblemen lösen zu können.

s. Übungsaufgaben von Aufgabenblatt 8

Nächste Vorlesung und Fragen

- Lernraum IV: Entwurf von Algorithmen
 - Sortieralgorithmen
- Schauen Sie sich bitte vorab die AP2-Videos von Prof. Kohls zu Sortieralgorithmen an:
 - <https://www.youtube.com/watch?v=V8ARyH2183Y&list=PLYohfDwqINSE6jPgHuVDKdA-9UlwhFxyQ&index=49>
- Fragen?